laSalle ENG
Ramon Llull University

Escola Tècnica Superior d'Enginyeria Electrònica i Informàtica La Salle

Trabajo Final de Máster

Máster en Programación Web de Alto Rendimiento

MeteoSalle

Alumno
David Espinosa Fariñas

Profesor Ponente
Eloi Rocamora Berlanga

Resumen:

Se quiere implementar una aplicación estilo red social orientada a compartir datos y previsiones meteorológicos. Un usuario de nuestra aplicación podrá definir una (o más) estaciones meteorológicas en las cuales podrá, o bien introducir datos manualmente o bien vincular la estación a una estación física de la que obtendrá estos datos de forma automática.

Además, se podrán consultar las diferentes estaciones meteorológicas de los demás usuarios en un mapa, así como realizar diferentes búsquedas / filtros para obtener datos de estaciones / usuarios concretos. Hemos decidido establecer en límite de nuestra aplicación a nivel España, de modo que, como máximo, se podrán realizar consultas de estaciones en todo el país.

Una estación deberá, como mínimo, proporcionar datos de temperatura, humedad y presión, siendo otros datos como la velocidad del viento opcionales. Además, una estación podrá ofrecer una predicción meteorológica para los siguientes 5 días, gracias a la recolección de datos en APIs externas y al cálculo ponderado de los mismos.

Abstract:

The aim of this project is to implement a social network style application to share weather data and forecasts. A user of our application will be able to define one (at least) weather stations in which he could either enter data manually or link that station to a physical one, from which will obtain the data automatically.

In addition, other users stations will be available to be consulted in a map, and also by doing different searches or applying filters to reach data from specific stations or users. We decided to establish our application limit on Spain so, at most, the queries allowed will be country level.

A station must provide, at least, temperature, humidity and pressure related data, being another information such as wind speed optional. Furthermore.Also, a station may offer a weather forecast for the next 5 days, thanks to the collection of data in external APIs and their weighted calculation.

1.- INTRODUCCIÓN 5
1.1. Sobre nosotros 5
1.2.- Sobre el trabajo 6
2.- OBJETIVOS 7
2.1.- Objetivos generales 7
2.2.- Objetivos técnicos 8
3.- PLANIFICACIÓN Y DESARROLLO 9
4.- REQUERIMIENTOS 10
5.- DISEÑO 11
5.1.- Tecnologías utilizadas. 13
5.2.- Diseño de la Api Restfull Middleware 24
5.2.1.- Recursos 24
5.2.2.- Definición de operaciones en términos de métodos HTTP 25
5.2.3.- Casos de uso 26
5.2.4.- Uso de HATEOAS para permitir la navegación a los recursos relacionados 27
5.2.5.- Control de versiones de la API Middleware 29
5.3.- Testing 29
5.4.- Arquitectura de software de la API Middleware 30
5.4.1.- “Interfaz de abstracción del actor” 31
5.4.2.- “Aplicación” 33
5.4.3.- “Servicios de infraestructura” 34
5.4.4.- “Dominio” 35
5.5.- Estructura general de carpetas 38
6.- RELACIÓN CON MATERIAS DEL MÁSTER 40
6.1.- Entornos web 40
6.1.1.- Vagrant & Docker 40
6.1.2.- Git 41
6.1.3.- Ansible 41
6.2.- Maquetación web 42
6.2.1.- HTML & CSS 42
6.2.2.- Flex, Grid & Bootstrap 43
6.2.3.- Responsiveness 46

6.3.- JavaScript 49
6.3.1.- Inclusión y versión 49
6.3.2.- Fetch API 51
6.3.3.- Web Storage 52
6.3.4.- Módulos y paquetes 53
6.4.- PHP 55
6.4.1.- PHP 55
6.4.2.- GO 57
6.5.- Desarrollo eficiente de software 58
6.5.2.- Arquitectura hexagonal 61
6.5.3.- Testing 63
6.5.3.1.- Selenium tests 63
6.5.3.2.- Revel TestSuite 64
6.6.- Bases de Datos 65
6.6.1.- MySQL 65
6.6.1.1.- Tipos de datos 65
6.6.1.2.- Sentencias 65
6.6.2.- MongoDb 66
6.7.- Frameworks 68
6.7.1.- ReactJS 68
6.7.2.- Symfony 70
6.7.3.- Revel 73
6.8.- SEO, Analítica web y Emprendeduría 73
6.8.1.- Arquitectura 74
6.8.1.1.- URLs, Dominios y ccTLDs 74
6.8.1.2.- Tipologías 74
6.8.1.3.- Sitemap 74
6.8.2.- Keywords 75
6.8.3.- Optimizaciones OnPage 75
6.8.3.1.- T's & M's (Titles & Metadescriptions) 76
6.8.3.2.- Contenidos 76
6.8.3.3.- Enlazado 76

6.8.3.4.- Geolocalización....77
6.8.4.- Conversión y modelo de negocio....77
6.9.- Rendimiento web....78
6.9.1.- Cache....78
6.9.2.- Paralelismo y Concurrencia....79
6.9.2.1.- Concurrencia....79
6.9.2.2.- Paralelismo....79
6.9.3.- Profiling....80
6.10.- Seguridad web....81
6.10.1.- Inyecciones....81
6.10.1.1.- Expresiones regulares....81
6.10.1.2.- Filtrado de caracteres....81
6.10.2.- Autenticación y sesiones....81
6.10.3.- Datos sensibles....83
7.- COSTE TEMPORAL....84
8.- CONCLUSIONES....86
8.1.- Conclusiones comunes....86
8.2.- Conclusión individual....86
9.- LÍNEAS DE FUTURO....88
10.- BIBLIOGRAFIA....89
11.- ANEXOS....95
11.1.- Anexo 1 Api Middleware....95

1.- INTRODUCCIÓN

1.1. Sobre nosotros

Este trabajo ha sido realizado por un equipo de tres personas, todos con experiencias diferentes relacionadas con el mundo de la informática, así como con aptitudes diversas, pero con la misma afinidad hacia el mundo de la programación web.

Marc Callejón

Marc se graduó en Ingeniería Informática en el año 2018, tras retomar unos estudios que tenía abandonados desde hace ya más de una década. Anteriormente había cursado estudios de electrónica y Telecomunicaciones, pero no había estado relacionado con el mundo del desarrollo profesional en ninguna de sus vertientes, más allá de las típicas asignaturas de la carrera y del entorno de programación de la placa Arduino. Su trayectoria profesional va a ligada a los sistemas y al mundo del hardware, pero siempre ha sido un apasionado de la tecnología, la electrónica, y de los retro-computadores, en particular.

David Espinosa

David estudió ingeniería informática además de una licenciatura de educación especializada también en informática. Durante varios años de experiencia laboral ejerció diferentes tareas, sobre todo de administración de sistemas y scripting en empresa, así como otros trabajos más relacionados con el desarrollo de forma autónoma. En la actualidad, gracias a los conocimientos adquiridos en el transcurso del máster está trabajando de desarrollador Backend en empresa.

Alfred Castells

Alfred es un desarrollador que terminó sus estudios del Grado en Ingeniería Informática en 2016 y que desde entonces ha trabajado en consultoría informática. En sus tres años de experiencia laboral adquirió conocimientos especializándose en el mundo web, concretamente en la capa Frontend, conociendo tecnologías básicas como HTML, CSS y JavaScript, y frameworks de desarrollo como React o Angular. Decidió dejar el mundo laboral para dedicarse de pleno en el Máster Universitario de Programación Web de Alto Rendimiento para complementar su formación en la programación web y así alcanzar un perfil más completo, o Full Stack, en dicha área de la programación.

1.2.- Sobre el trabajo

El trabajo que se va a presentar en este documento se sostiene en dos de las características o "necesidades" más comunes o frecuentes en la sociedad globalizada actual. La consulta de información actualizada y la compartición de ésta a nivel social.

Empezando por ésta última, se podría decir que hoy en día hay una red social para todo. TripAdvisor para turismo y sobre todo restauración, LinkedIn para aspectos laborales y curriculares, Wallapop para compraventa de segunda mano, Instagram para compartir experiencias más personales… y la lista podría ocupar el trabajo entero si se quisiera. Partiendo de esta base este trabajo quiere hacer su aportación a este mundo socialmente conectado incorporando otra red social más que cubra un aspecto diferente.

Ilustración 1 Lluvia de ideas

Y en cuanto al otro aspecto clave, el de la inculcada necesidad de poder consultar todo tipo de información actualizada prácticamente sobre cualquier aspecto, se podría hablar de las consultas de noticias, de tráfico, de resultados deportivos, de eventos, de música… De nuevo una larga lista.

Con todo esto, la propuesta que se va a implementar en este proyecto y que aúna los dos componentes expuestos es la de una red social de consultas meteorológicas. Para ello, con tal de relacionar estos dos aspectos, cada uno de los usuarios va a ser también una fuente de datos además de un perfil "consultor". Vamos a introducir el concepto de estación meteorológica para proporcionar datos de temperatura, humedad, etc.

Ilustración 2 Sobre el trabajo

En los siguientes apartados se va a desarrollar todo el proceso de análisis de requerimientos, diseño e implementación que componen nuestro proyecto, el cual se ha decidido limitar a nivel España.

2.- OBJETIVOS

En este apartado se van a enumerar y definir los diferentes objetivos que se han marcado y que se cree que mediante su alcance se obtendría una aplicación óptima en diferentes sentidos, como se verá a continuación. Están separados en dos tipos o clases, primero se verán los más relacionados con las funcionalidades en sí y más adelante se tratarán los que se podrían considerar más técnicos.

Ilustración 3 Objetivos

2.1.- Objetivos generales

- **Mapa con las estaciones:** En la página principal, una vez realizado el Log In, mostrar un mapa como elemento principal en el cual se distribuya un conjunto de marcadores, los cuales representarán las estaciones meteorológicas (en sus coordenadas correspondientes).

- **Listado con opciones de filtrado:** También en la página principal, como complemento del mapa, mostrar un listado de las estaciones que figuran en el mapa, además de ofrecer opciones para filtrarlas.

- **Información usuario:** Ofrecer una página para la visualización del perfil de un usuario, en el cual se muestre la información de éste y las diferentes estaciones de las que dispone.

- **Información estación:** Otra página, parecida a la de usuario, pero para las estaciones, mostrando la información de ésta con tablas y gráficos, incluyendo las que se obtienen de fuentes externas en el apartado de previsión.

2.2.- Objetivos técnicos

- **Interfaz adecuada:** Lograr que el Lookup general de la aplicación web constituya una interfaz intuitiva y a la vez con un diseño acorde al tema sobre el que trata (la meteorología).

- **Responsiveness:** Conseguir que la experiencia de usuario en smartphones y tablets sea como mínimo igual de agradable que en un dispositivo de escritorio.

- **Buena respuesta de rendimiento:** Al trabajar con un volumen considerable de datos, se quieren obtener buenos niveles de rendimiento, a nivel de carga y tiempos de respuesta.

- **SEO:** Que el resultado final sea una web que cumpla con todos los estándares posibles para alcanzar un buen nivel de SEO.

- **Despliegue:** Que el conjunto de componentes y servicios (Frontend, Backend y Middleware) sean lo más fáciles de levantar, partiendo de un único repositorio de Git y un documento README con todo lo necesario.

3.- PLANIFICACIÓN Y DESARROLLO

En este apartado se va a comentar un poco cómo ha sido la gestión de las tareas y la coordinación que se ha seguido para poder llevar a cabo con éxito el desarrollo del proyecto.

Para ello se ha usado **Trello**, una aplicación que provee de las herramientas necesarias para poder trabajar con una Kanban Board, uno de los sistemas organizativos más usados hoy en día y que permite, de forma visual y eficaz, tener en todo momento una fotografía del estado actual del desarrollo del proyecto.

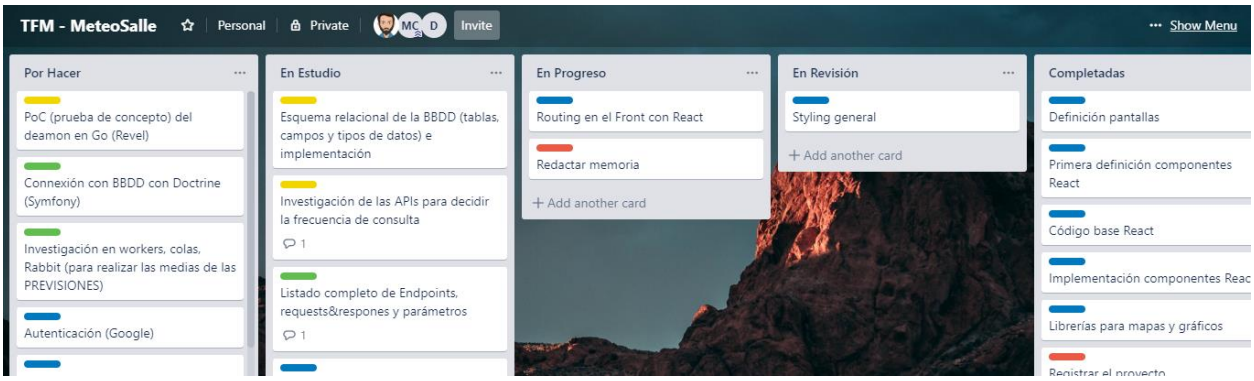


Ilustración 4 Trello

Como se puede ver en la captura, la aplicación permite la creación de columnas que representan el estado en que se encuentran las tareas, y de “tarjetas” para identificar a estas últimas. Además, a cada tarjeta/tarea se le pueden asignar etiquetas de colores para poder clasificarlas en tipos (Frontend, Backend, etc.) así como la asignación de un responsable, añadir comentarios, etc.

En el caso de este trabajo, se han definido 5 columnas: Por Hacer, En Estudio (para las tareas que requerían un análisis previo antes de empezarlas como tal), En Progreso, En Revisión (por nosotros mismos o el tutor del proyecto) y Completadas. Y para las tareas, se han creado las etiquetas o clasificación de: Frontend, Middleware, Backend, Memoria y Soporte (tareas del estilo revisión de la normativa, registro del proyecto, etc.).

Por último, hay que comentar también que, como se verá en el apartado de Diseño, la realización de este trabajo se ha separado en tres grandes bloques, Frontend-API-Backend, de modo que las tareas pertenecientes a cada una de esas áreas se han asignado automáticamente al responsable de dicha parte. Por otro lado, las correspondientes al redactado de la memoria y otras tareas de soporte se han ido repartiendo indistintamente entre los integrantes del equipo.

4.- REQUERIMIENTOS

Antes de entrar en el detalle de la implementación en sí del proyecto, se van a enumerar los diferentes requerimientos (técnicos y no tan técnicos) de los que se parte y que condicionan el diseño de la aplicación, que es el apartado que se verá a continuación. Algunos sonarán por haber definido objetivos (punto 2) con una relación bastante directa:

Ilustración 5 Requerimientos

- **Gestión de usuarios:** Se pide efectuar la típica implementación de gestión de usuarios; registro, login con autenticación, etc.
- **Variables meteorológicas mínimas:** Como mínimo las estaciones meteorológicas deben contemplar valores de temperatura, humedad y presión atmosférica.
- **Correspondencia temporal en los datos:** Cualquier dato/valor introducido relativo a un valor meteorológico, debe tener asociado un timestamp.
- **Mapas y gráficos:** Las estaciones y sus valores (la evolución temporal) se deben poder plasmar en un mapa y diversos gráficos, respectivamente.
- **Diseño responsive:** La aplicación web debe adaptar su interfaz a tablets y smartphones.
- **Frontend y Backend diferenciado:** Se deben implementar por separado estas dos capas como mínimo, permitiendo eso el uso de tecnologías y frameworks diferenciados.
- **API REST:** El Frontend debe obtener los datos a representar en la aplicación (en nuestro caso, desde la capa Middleware) a través de una API REST.
- **PHP:** Se requiere el uso del lenguaje de programación PHP, al menos para la capa Middleware.
- **Caché de 30 minutos:** Se debe implementar alguna técnica de cacheado de datos con una caducidad de 30 minutos, pasados los cuales dichos datos dejan de tener validez.
- **GIT:** El proyecto se debe versionar en un repositorio de GIT con tal de poder realizar una mejor supervisión y seguimiento.
- **Ansible:** Para el despliegue de la aplicación se debe usar Ansible para efectuar el aprovisionamiento de máquinas virtuales / componentes de sus servicios o dependencias correspondientes al menos en una de las partes de la aplicación.
- **Server Salle:** Una vez terminado el proyecto, éste debe permanecer funcionando en un servidor propiedad de La Salle.

5.- DISEÑO

Como ya se ha comentado en un par de ocasiones, éste trabajo ha sido llevado a cabo en equipo, y en este punto explicaremos cómo se han repartido las diferentes tareas de desarrollo. Por suerte, como se verá a continuación, el proyecto en sí ya "traía" definidas tres capas bastante diferenciadas: Frontend, Middleware y Backend.

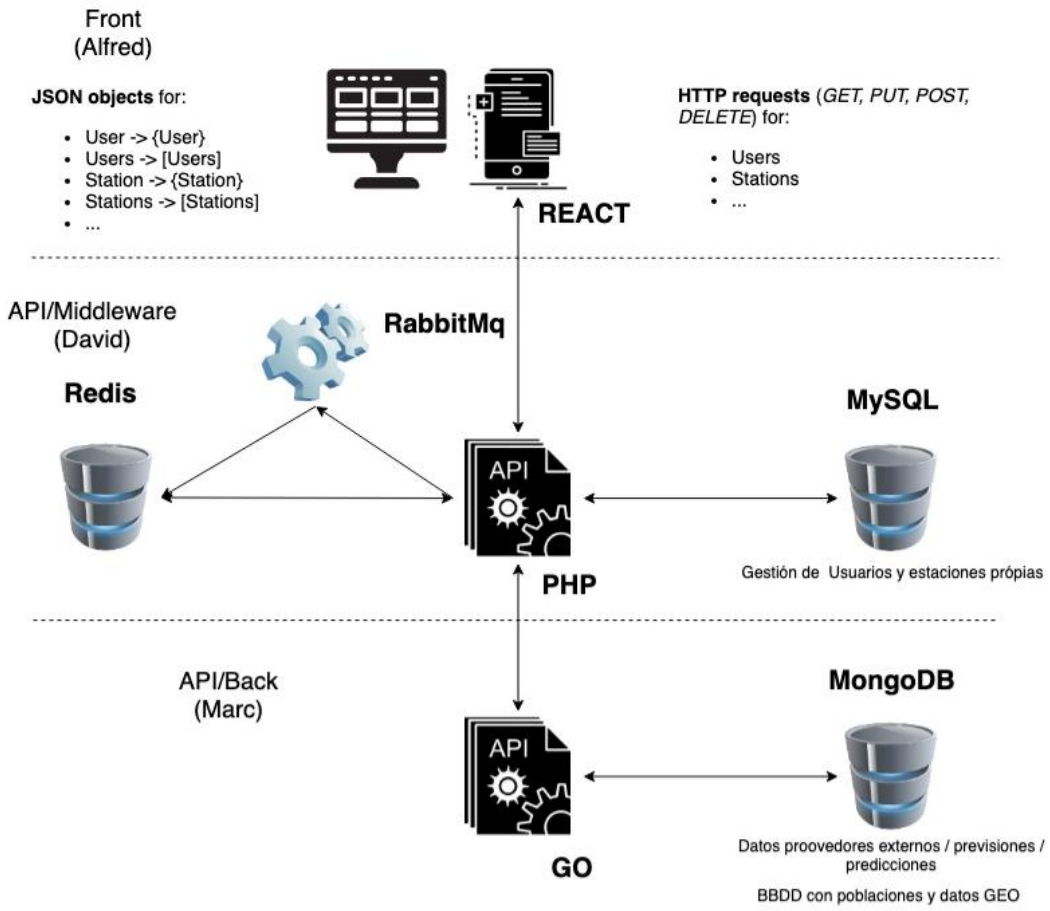


Ilustración 6 Diagrama de comunicaciones

- Capa **Frontend**: Alfred se ha encargado de desarrollar el frontal de la aplicación web, es decir, todo el conjunto de pantallas y elementos que componen la parte visible de ésta. Esto engloba todo el trabajo de identificación de elementos, estilos y comportamientos, siendo gran parte de esta última tarea toda la parte de comunicación con el Middleware; el envío y la gestión y almacenaje de los datos con los que trabajar. Como podemos ver en el diagrama, la tecnología principal para el desarrollo de esta capa ha sido ReactJS.

- Capa **Middleware**: David se ha encargado de construir la API principal de la aplicación, que se encarga de gestionar todas las peticiones que llegan desde el frontal, ya sean de lectura o de escritura, y de solicitar o traspasar esos datos a la capa Backend. Podríamos definir este componente intermedio como una especie de traductor de datos e información

entre los dos extremos de la web representa la capa de abstracción del frontend para consumir y gestionar los datos. De nuevo, como se puede ver representado en el esquema de capas, la API ha sido desarrollada en PHP con el frameworks Symfony y se pudiera catalogar como de nivel 3 debido a que implementa el principio **HATEOAS (Hipertexto como motor del estado de la aplicación),** cada solicitud devuelve la información necesaria para encontrar los recursos relacionados y cómo operar sobre ellos.

- Capa **Backend**: La gestión de la BBDD principal de la aplicación ha sido cosa de Marc, quien ha llevado a cabo del desarrollo del componente de persistencia de los datos relativos a la información proporcionada por las distintas API. Para ello, se ha usado un sistema gestor de Base de datos cómo es MongoDB y una API desarrollado en GO bajo el frameworks Revel, la unión de estas dos tecnologías, por su agilidad, brindan un rendimiento exquisito.

Obviamente, el hecho de haber realizado esta repartición tan exclusiva de las distintas partes de las que se compone el trabajo no implica que no haya habido colaboración y comunicación en todo momento. Y por supuesto, al final, todos han acabado cooperando y contribuyendo en el desarrollo de las capas ajenas, por el simple hecho de que no todos los componentes han seguido un ciclo de desarrollo equivalente (a nivel de ritmo) a lo largo de las distintas etapas de la implementación.

A continuación, se muestra el diseño de la API intermedia **Middleware**, que resuelve el problema de estandarizar las peticiones del frontend, separando las solicitudes de datos externas del modelo de negocio de meteosalle en general, entiéndase como modelo de negocio el sitio social de meteosalle, que para su funcionamiento necesita nutrirse de datos generales que se obtienen de recursos externos a los dominios de la aplicación; estos datos externos son recopilados y almacenados temporalmente por el backend. La capa middleware que en estos momentos nos compete; su tarea principal es la de gestionar y almacenar la información de usuarios, así como sus datos relacionados con las estaciones y los datos de estaciones externas que estandariza para el modelo de negocio de la aplicación, además de implementar la lógica necesaria para cubrir los supuestos casos de usos previstos por el frontend. Para la solución de este problema se ha implementado una interfaz de programación de aplicaciones o API como comúnmente se conoce; específicamente una API Rest porque utiliza protocolos estándares de comunicación que pudieran ser empleadas para seguir expandiendo la aplicación además de simplificar el desarrollo y brindando una interfaz común que será consumida por el resto de las aplicaciones, siendo agnóstica a cualquier lenguaje de programación.

5.1.- Tecnologías utilizadas.

Para el desarrollo de la API se han utilizado varias tecnologías, entre ellas:

Ilustración 7 PHP 7

PHP (acrónimo en inglés para **Hypertext Preprocessor**) como lenguaje de programación fundamental para su desarrollo debido a que es un lenguaje de programación de propósito general, orientado a objetos, sin costo de licencias y cuenta con una gran comunidad y documentación, robusto, confiable y seguro.

Ilustración 8 Symfony

Se ha utilizado además un frameworks que respeta los principios SOLID que no te fuerza al acoplamiento como otros frameworks y que permite automatizar los procesos comunes del desarrollo, su código, y el de todos los componentes y librerías que incluye, se publican bajo la licencia MIT de software libre. Se ha escogido principalmente por su fácil utilización y la capacidad de incluirles componentes y paquetes para el desarrollo de la API, necesarios para agilizar el proceso de desarrollo. Los componentes y bundles utilizados principales son:

Componentes

symfony/console: Permite crear comandos de línea de comandos. Los comandos de la consola se pueden usar para cualquier tarea recurrente, como cronjobs, importaciones u otros trabajos por lotes. Se ha utilizado para crear el comando que ejecuta el **worker** que se mantiene a la escucha de eventos de actualización de estaciones, así como para crear la cache inicial en Redis.
symfony/dotenv: analiza los archivos .**env** para que las variables de entorno almacenadas en ellos sean accesibles a través de $ _ENV o $ _SERVER. Se ha utilizado para cargar las variables de entornos de la aplicación como son, la configuración de dirección de servidores y configurar los estados de la aplicación, refiérase a en estado de producción o de desarrollo.
symfony/http-client: El componente HttpClient es un cliente HTTP de bajo nivel con soporte para envolturas de flujo PHP y cURL[4]. Proporciona utilidades para consumir API y admite operaciones síncronas y asíncronas. En la aplicación se ha utilizado para consumir la API del backend 2, se ha preferido este componente antes que otro porque es altamente configurable en las peticiones pudiendo especificar al tipo de versión HTTP a utilizar, la cantidad de conexiones simultaneas que se desean realizar, así como especificar que tecnología utilizar para realizar peticiones, cURL o PHP me he decantado por cURL principalmente porque es un software especializado en la transferencia de archivos.

symfony/http-foundation: El componente HttpFoundation define una capa orientada a objetos para la especificación HTTP. Según mi criterio uno de los componentes estrella. En PHP, la solicitud está representada por algunas variables globales ($ _GET, $ _POST, $ _FILES, $ _COOKIE, $ _SESSION, ...) y la respuesta es generada por algunas funciones (echo, header (), setcookie (), ...)
El componente Symfony HttpFoundation reemplaza estas variables y funciones globales PHP predeterminadas, por una capa orientada a objetos lo que la hace magnifica para el desarrollo de la API Rest. Se ha utilizado en el proyecto para manejar todas las peticiones y respuestas de la API a través del objeto **Request** y del objeto **JsonResponse** exceptuando las respuestas a rutas que no existen que veremos en otro componente.
Symfony/Routing El componente de enrutamiento asigna una solicitud HTTP a un conjunto de variables de configuración. Se utiliza para construir sistemas de enrutamiento para aplicaciones web donde cada URL está asociada con algún código para ejecutar. Se ha utilizado para indicar la función a ejecutar al realizar una llamada a una URL del dominio de la API, a través de Annotation\Route, se habilita para un controller una URL determinada, se ha preferido definir las rutas como annotations debido a que se puede identificar y asignar más fácil la función de un controller para una ruta determinada.

Bundles

Un bundle (paquete) es similar a un plugin en otro software, pero aún mejor. Las características principales enmarcadas en Symfony se implementan con paquetes (FrameworkBundle, SecurityBundle, DebugBundle, etc.) También se usan para agregar nuevas características a la aplicación a través de paquetes de terceros. En el desarrollo de la API se han utilizado varios, como son:
Symfony/maker-bundle: Symfony Maker automatiza el proceso de creación de comandos, controladores, test, etc. Evitando tener que repetir continuadamente la creación de código análogo como pudiera ser la creación individual de controladores. Se ha utilizado en la fase de desarrollo para crear todos los controladores y comandos.
NelmioCorsBundle: Permite enviar encabezados de uso compartido de recursos de origen cruzado con una configuración por URL de estilo ACL. Se ha utilizado para permitir las peticiones por parte del frontend a la API, para evitar los ataques de tipo Cross site scripting.

Otros

symfony/Flex: Aunque su trabajo no es “visualmente” reconocido cabe destacar su importancia, según sus propias palabras en https://flex.symfony.com/.
“Symfony Flex es la forma de administrar las aplicaciones de Symfony” …, …” son un conjunto de instrucciones automatizadas para integrar paquetes de terceros en aplicaciones de Symfony”.
internamente, Symfony Flex es un “complemento” de Composer que modifica el comportamiento de los comandos require y update. Al instalar o actualizar dependencias en una aplicación habilitada para Flex, Symfony puede realizar tareas antes y después de la ejecución de las tareas de Composer.

Ilustración 9 Composer

Composer es un administrador de paquetes a nivel de aplicación para el lenguaje de programación PHP que proporciona un formato estándar para administrar las dependencias del software y las bibliotecas requeridas. Fue desarrollado por Nils Adermann y Jordi Boggiano, quienes continúan administrando el proyecto. Herramienta fundamental para la agilización del proyecto. Se ha utilizado para instalar todas las librerías externas como:
predis/predis utilizada para la comunicación con la base de datos de Redis y gestionar los datos almacenados en memoria refiérase a actualizaciones de datos, inserciones, modificaciones y eliminaciones.
php-amqplib/php-amqplib: Esta biblioteca es una implementación pura de PHP del protocolo AMQP. Utilizada en el proyecto para explotar las funcionalidades de rabbit.
A través de este software se ha automatizado todo el proceso de despliegue de la aplicación y los requerimientos de aplicaciones externas.

Ilustración 10 Mysql

Mysql es un sistema de gestión de bases de datos relacional desarrollado bajo licencia dual: Licencia pública general/Licencia comercial por Oracle Corporation y está considerada como la base de datos de código abierto más popular del mundo, sobre todo para entornos de desarrollo web. Es utilizada en el proyecto para mantener los datos y persistirlos en el tiempo de las estaciones, usuarios y la relación entre el código postal y la provincia a la que pertenece una estación además del histórico de las estaciones de los usuarios.

A continuación, se muestra una imagen con el esquema del modelo relacional de la base de datos utilizada.

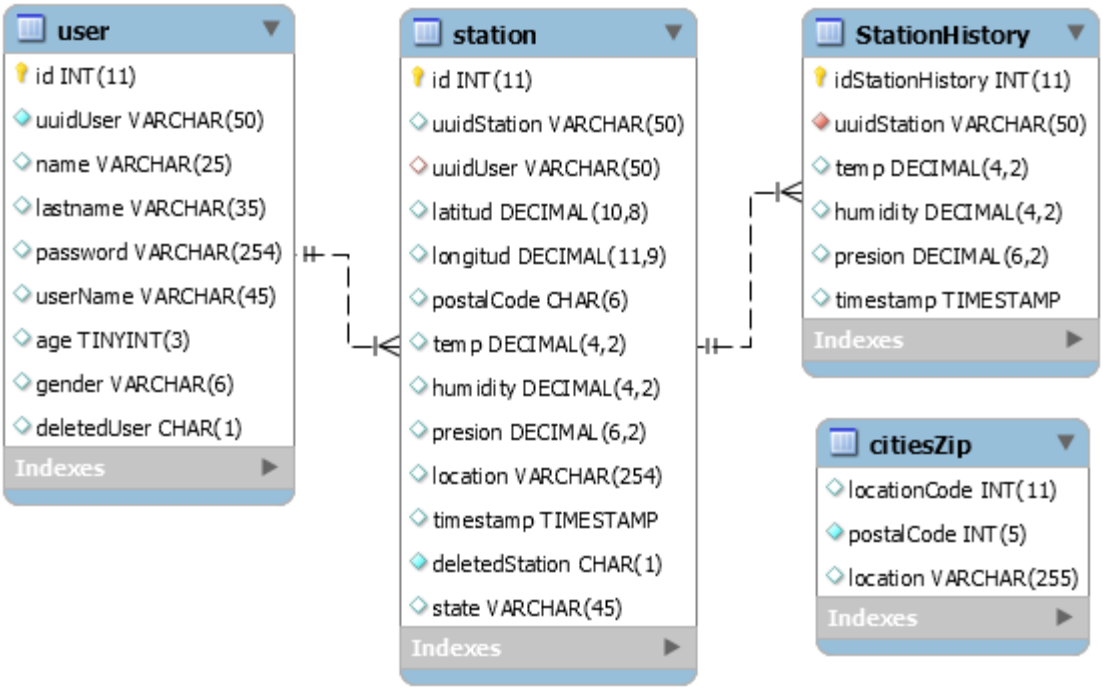


Ilustración 11 Diagrama relacional de la base de datos meteosalle

imagen1

Como se puede observar se han definido varios índices que agilizarán el proceso de búsqueda de datos para las peticiones que se le realicen, los tamaños de campos y los tipos de datos se han escogido teniendo en cuenta los valores que se persistirán, se ha escogido la codificación de caracteres utf8 que es capaz de reconocer cualquier carácter Unicode, teniendo en cuanta que la aplicación Meteosalle en general podrá ser accedida por cualquier persona y los caracteres del idioma no son iguales en todos los estándares.

Table User

Representa los datos pertenecientes a un usuario, mantiene una relación de uno a muchos con la tabla Station, debido a que un usuario podrá “tener” muchas estaciones, sin embargo, una estación no podrá existir sin un usuario por eso se establece la relación a través de una llave foránea que mantenga la consistencia de la base de datos, el mismo criterio se aplica para la tabla de StationHistory y su dependencia con Station.

Table Station

Representa los datos de una estación, el campo location no guarda relación con la tabla citiesZip (ver más abajo), debido a que todos los campos de esta tabla son modificables por el usuario, excepto el identificador de estación y el de usuario.

Table StationHistory

Representa los datos históricos de una estación, mantiene una dependencia con la tabla Station teniendo una relación de muchos a uno, es decir una estación podrá contener muchos datos históricos.

Table CitiesZip

Tabla de “soporte logístico” contiene la información necesaria para poder consultar adecuadamente la API del backend 2 desde el punto de vista que la API externa se consulta en base a códigos de localidad (por detalles de optimización que agilizan el proceso de búsqueda), sin embargo, las peticiones de clientes externos (Frontend) se basan en códigos postales.

Ilustración 12 RabbitMq

RabbitMQ es un software de negociación de mensajes de código abierto. Implementa el estándar Advanced Message Queuing Protocol (AMQP) es un protocolo de estándar abierto en la capa de aplicaciones de un sistema de comunicación. Las características que definen al protocolo AMQP son la orientación a mensajes, encolamiento ("queuing"), enrutamiento (tanto punto-a-punto como publicación-subscripción), exactitud y seguridad. El código fuente está liberado bajo la licencia Mozilla Public License.
En el proyecto se utiliza principalmente para crear un proceso paralelo que actualice los valores de las estaciones en la base de datos de **Redis** brindando agilidad a la hora de responder a la aplicación cliente, básicamente cuando se modifica el estado de una estación, se publica un “evento”, un mensaje en la cola “onUpdate” definida en el “worker” “StationEventListenerWorkerCommand”, este evento se lanza en el caso de uso que está modificando u obteniendo el dato que contiene una estación “Station”, debido a que su principal función es actualizar la base de datos de Redis**.** Poniendo en contexto: cuando se busca una estación el caso de uso FindStation primero va a Redis e intenta encontrar la estación, de no encontrarla va a Mysql y el objeto devuelto lo introduce en un evento llamado OnUpdateStation, este evento lleva por nombre, el nombre de la cola donde será introducido y luego es insertado a través de “TailsRepositoryRabbit” que es el que tiene la implementación concreta para conectarse al servicio “Rabbit”; en el otro extremo de la cola se encuentra el worker que se encarga finalmente de escribir en “Redis”.

A continuación, un ejemplo de su implementación.

Caso de uso FindStation

```
    $this->tailMessageRepository = $tailMessageRepository;
}

public function __invoke($uuidStation): Station
{
    $response = $this->cacheDataRepository->find($uuidStation); //Redis
    if (!empty($response)) {
…
    } else {
        $station = $this->repository->findStation($uuidStation); //Mysql

        $event = new OnUpdateStation($station); //evento
        $this->tailMessageRepository->publishEvent($event); //publicación del evento
    }
    return $station;
```

Interfaz (“contrato”).

```
interface MeteosalleEvents
{
    public function __toString(): string;

    public function getDataAsJson(): string;
}
```

Implementación concreta del servicio externo.

```
public function publishEvent(MeteosalleEvents $event): void
{
    $this->channel->exchange_declare((string)$event, 'fanout', false, false, false);
    $data = $event->getDataAsJson();

    $msg = new AMQPMessage($data);

    $this->channel->basic_publish($msg, (string)$event);
```

Worker escuchando el evento.

```
private function listenerEventForUpdateCache(CacheDataRepositoryRedis $cacheData): void
{
…
    $channel->exchange_declare('onUpdate', 'fanout', false, false, false);
…
    $callback = static function ($msg) use ($cacheData) {

        $message = json_decode($msg->body, true);
        $cacheData->insert($message['uuidStation'], $message); …
```

Ilustración 13 Redis

Redis es un motor de base de datos en memoria, basado en el almacenamiento en tablas de hashes de tipo (clave/valor) pero que opcionalmente puede ser usada como una base de datos durable o persistente. Está liberado bajo licencia BSD por lo que es considerado software de código abierto. En el proyecto se ha utilizado para agilizar los procesos de búsqueda de datos, los datos consultados se almacenan en Redis y son actualizados según las operaciones que se vayan realizando en la aplicación, de esta forma se evita retrasar las respuestas al cliente; devolviendo los datos que ya hayan sido consultados, disminuye los tiempos de respuesta de 3,88 segundos a 1500 milisegundos.

A continuación, un ejemplo de su utilización en el caso de uso FindUser.

```
public function __invoke($uuidUser): user
{
    $response = $this->cacheDataRepository->find($uuidUser);
    if (!empty($response)) {
        $user = new User
        (
            $response['uuidUser'],
            $response['name'],
…
    } else {

        $user = $this->repository->findUser($uuidUser);

        $this->cacheDataRepository->insert($uuidUser, $user->getUserLikeArray(),
$_SERVER['TIME_TO_LIVE_CACHE']);
    }
    return $user;
```

Implementación concreta.

```
final class CacheDataRepositoryRedis implements CacheDataRepository

public function find($valueToFind)
{
    $key = md5($valueToFind);

    if ($this->redis->exists($key)) {
        return json_decode($this->redis->get($key), true);
    }
…
```

```
public function insert(string $valueToInsert, array $dataToCache, $timeExpire = null): void
{
    if ($timeExpire !== null) {
        $timeToLive = $timeExpire;
    } else {

        $timeToLive = $_SERVER['TIME_TO_LIVE_CACHE'];
    }

    $jsonEncode = json_encode($dataToCache, false);
    $key = md5($valueToInsert);

    $this->redis->set($key, $jsonEncode);//if key exist update values
    $this->redis->expire($key, $timeToLive);
```

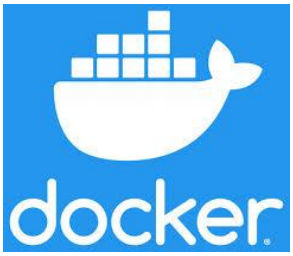


Ilustración 14 Docker

Docker es un proyecto de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción y automatización de virtualización de aplicaciones en múltiples sistemas operativos. Se ha utilizado en el proyecto para el despliegue de la aplicación debido a su versatilidad y adaptación en varios sistemas operativos además de la agilidad que ofrece para el despliegue; la configuración se realiza a través de un fichero escrito en YAML[1], interpretado por la aplicación Docker-compose.

Muestra del fichero de configuración empleado.

```
version: '3'
services:
  mysql:
    image: mysql:5.7
    container_name: mysql
    environment:
      MYSQL_DATABASE: 'meteosalle'
      MYSQL_USER: 'user'
      MYSQL_PASSWORD: 'password'
      MYSQL_ROOT_PASSWORD: 'password'
…
 nginx:
```

[1] YAML es un formato de serialización de datos legible por humanos inspirado en lenguajes como XML, C, Python, Perl. Wikipedia.

```
...
 ports:
   - "80:80"
```

El fichero está compuesto por “tags” jerárquicos, dentro de cada “tag” se establecen los detalles para ese “tag”, por ejemplo: en la capa superior encontramos el tag “versión”, este indica a la aplicación como interpretar las instrucciones a continuación; en dependencia de la versión tendrá disponibles algunas que otras propiedades. Debajo está definido “services”, dentro de este tag se declaran los servicios por individual que se han de utilizar por ejemplo “mysql” y dentro, los parámetros específicos para cada servicio, cada servicio por separado representa una instancia de Docker un “Docker container”. La propiedad “image” especifica la imagen virtual del contenedor que queremos utilizar; “enviroment” especifica valores adicionales que se introducirán para personalizar el servicio.

{ REST }

REST La transferencia de estado representacional (en inglés representational state transfer) es un estilo de arquitectura de software para sistemas hipermedia[2] distribuidos como la World Wide Web. El término se originó en el año 2000, en una tesis doctoral sobre la web escrita por Roy Fielding, uno de los principales autores de la especificación del protocolo HTTP[3] y ha pasado a ser ampliamente utilizado por la comunidad de desarrollo.
Es utilizado en el proyecto para la comunicación con el resto de los servicios de la aplicación Meteosalle, tanto el frontend como el servicio de búsqueda de información de datos externos.

A continuación, se muestra el diagrama de flujo.

[2] El término hipermedia sirve para designar al conjunto de métodos o procedimientos para escribir, diseñar o componer contenidos que integren soportes tales como: texto, imagen, video, audio, mapas y otros soportes de información emergentes, de tal modo que el resultado obtenido, además, tenga la posibilidad de interactuar con los usuarios.
[3] El Protocolo de transferencia de hipertexto Http es el protocolo de comunicación que permite las transferencias de información en la World Wide Web. Wikipedia.

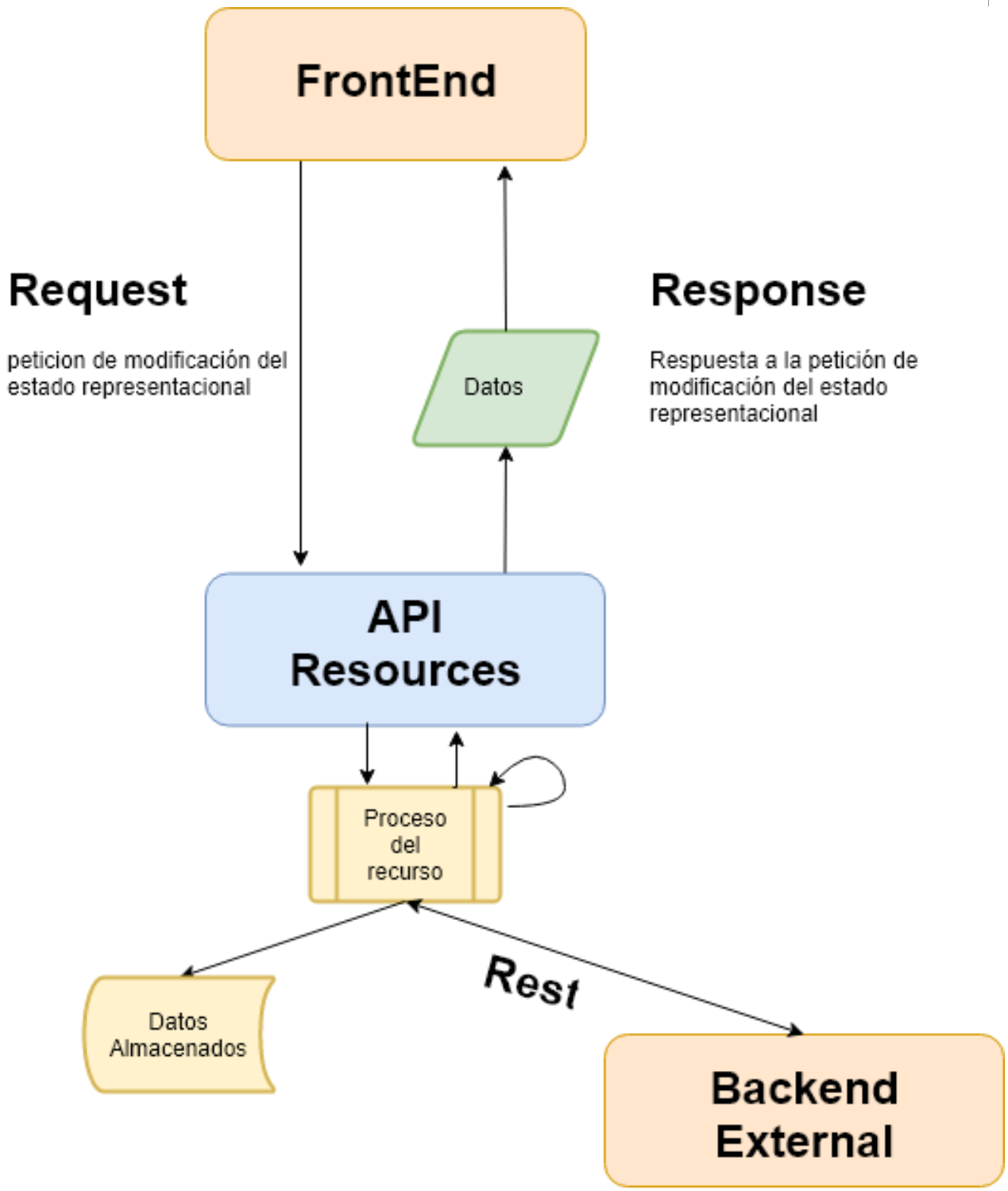


Ilustración 15 Diagrama de flujo entre los servicios

Ilustración 16 Json

JSON (acrónimo de JavaScript Object Notation, «notación de objeto de JavaScript») es un formato de texto sencillo para el intercambio de datos. Se trata de un subconjunto de la notación literal de objetos de JavaScript, aunque, debido a su amplia adopción como alternativa a XML, se considera (año 2019) un formato independiente del lenguaje se ha utilizado en el proyecto para responder con este tipo de datos a todas las llamadas que se le realicen a la Api, además el intercambio con el backend 2 también se realiza utilizando este tipo de dato un ejemplo. Dentro de la API toda la información y el flujo de datos es gestionado con tipos de datos propios de php, exceptuando las respuestas de la API, las inserciones en la base de datos de Redis y cuando se insertan los eventos de actualizaciones en RABBITMQ. Básicamente todo lo que no permita trabajar con un objeto propio de php se serializa en este formato para intercambiar información. El objeto y la función utilizada para esto es JsonResponse que pertenece al componente HTTTP Foundation de Symfony y para las “serializaciones” internas jsonEncode y jsonDecode, ambas funciones del lenguaje de programación utilizado en el proyecto PHP.

Ejemplo de la utilización de Json Response:

```
$jsonResponse = new JsonResponse(array_merge(['station'=>$station-
>getStationLikeArray(),'links'=>$stationsLinks->getLinksForGET($uuidStation)]), 200,
    array(
        'Content-Type' => 'application/json',
        'User-Agent' => 'MeteoSalleMiddel',
        'Access-Control-Allow-Origin'=>'*',
    ));

$jsonResponse->setEncodingOptions(400);
return $jsonResponse;
```

Ejemplo de la utilización de json_decode() cuando se escucha el evento de actualizar una estación:

```
$callback = static function ($msg) use ($cacheData) {

    $message = json_decode($msg->body, true);
    $cacheData->insert($message['uuidStation'], $message);
    echo 'Update cache for ‘. $message['uuidStation'];
};
```

Ejemplo de la utilización de json_encode() cuando se escucha el evento de actualizar una estación:

```
public function getDataAsJson(): string
{
```

```
    return json_encode($this->payload);
}
public function publishEvent(MeteosalleEvents $event): void
{
    $this->channel->exchange_declare((string)$event, 'fanout', false, false, false);
    $data = $event->getDataAsJson();

    $msg = new AMQPMessage($data);

    $this->channel->basic_publish($msg, (string)$event);
```

5.2.- Diseño de la Api Restfull Middleware

En el año 2000, Roy Fielding propuso la transferencia de estado representacional (REST) como enfoque de arquitectura para el diseño de servicios web. REST es independiente de cualquier protocolo subyacente y no está necesariamente unido a HTTP. Sin embargo, en las implementaciones más comunes de REST se usa HTTP como protocolo de aplicación, y la API Middleware centra su diseño en REST para HTTP. Uno de los principales propósitos que se esconden detrás de REST es que debe ser posible navegar por todo el conjunto de recursos sin necesidad de conocer el esquema de URI. Cada solicitud HTTP debe devolver la información necesaria para encontrar los recursos relacionados directamente con el objeto solicitado mediante los hipervínculos que se incluyen en la respuesta, y también se le debe proporcionar información que describa las operaciones disponibles en cada uno de estos recursos. Este principio se conoce como HATEOAS, del inglés Hypertext as the Engine of Application State (Hipertexto como motor del estado de la aplicación). La API middleware es realmente una máquina de estado finito, y la respuesta a cada solicitud contiene la información necesaria para pasar de un estado a otro, brindando la posibilidad de un rápido aprendiza de la API.

5.2.1.- Recursos

La API Middleware está diseñada en torno a recursos que se identifican inequívocamente por su URI[3], pidiendo una determinada URI corresponderá a la petición de un servicio de la aplicación Meteosalle, el frontend interactuará con la API a través del intercambio de representaciones de recursos y utilizando verbos HTTP para realizar operaciones sobre los recursos, estos recursos también están agrupados en colecciones para adoptar una nomenclatura coherente que permita al desarrollador de la aplicación cliente una curvatura lineal de aprendizaje de su uso, se ha tomado en cuenta la relación entre los diferentes tipos de recursos y el cómo se exponen sus asociaciones. La estructura de la URL se ha definido de la forma siguiente.

Dirección del servidor /apiv1/colección/recurso/colección

A continuación, un ejemplo de cómo se ha implementado a nivel de código utilizando el bundle Annotations de Symfony en uno de los controladores.

```
* @Route("/apiv1/stations/{uuidStation}", name="get_one_station", methods={"GET"})

public function index($uuidStation): JsonResponse
```

cada uno de los recursos tiene una 'función' asociada que es la que se encarga de la lógica necesaria para operar sobre el recurso solicitado, tomando en cuenta el método HTTP solicitado. Como se puede observar para el intercambio de datos representacionales se ha utilizado el formato JSON.

5.2.2.- Definición de operaciones en términos de métodos HTTP

El protocolo HTTP define una serie de métodos que asignan significado semántico a una solicitud Rest. Los métodos HTTP utilizados para las operaciones sobre los recursos de la API Middleware se han escogido cuidadosamente respetando la idempotencia de cada uno de ellos y son:

- **GET** para recuperar una representación del recurso en el URI especificado, tanto de predicciones, usuarios o estaciones. El cuerpo del mensaje de respuesta en formato JSON contiene los detalles del recurso solicitado.

- **POST** crea un nuevo recurso en el URI especificado, tanto de usuarios como de estaciones y el histórico de las estaciones. El cuerpo del mensaje de solicitud proporciona los detalles del nuevo recurso.

- **PUT** actualiza el recurso en el URI especificado tanto de usuarios como de estaciones. El cuerpo del mensaje de solicitud especifica el recurso que se va a actualizar.

- **DELETE** elimina el recurso en el URI especificado para la aplicación cliente, en la Api Middleware, el recurso no es eliminado, lo que se decidió hacer fue la modificación del campo deletedStation o deletedUser estableciendo un valor que lo identifica como eliminado.

Sus posibles respuestas, así como ejemplos concretos de las peticiones y respuestas (casos de uso) se relacionan en el **Anexo 1 API Middleware** de la memoria, que se podrá utilizar como información técnica para desarrolladores de clientes para la API.

5.2.3.- Casos de uso

El efecto de una solicitud específica en la API depende de si el recurso es una colección o un elemento individual. En la tabla siguiente se resumen las convenciones adoptadas para la implementación de la API REST.

Recurso	POST	GET	PUT	DELETE
/stations/	Crear una nueva estación	Recupera todas las estaciones	Error	Error
/stations/1	error	Recupera los detalles de la estación 1	Actualiza los detalles de la estación 1 si existe	Elimina la estación 1
/stations/1/history	Crear un nuevo registro histórico para la estación 1	error	error	error
/stations/postalcode/2	error	Recupera las estaciones con el código postal 2	error	Error
/user/	Crear un nuevo usuario y un token de session	error	Error	Error
/user/1	error	Recupera los detalles de usuario 1	Actualiza los detalles de usuario 1 si existe	Elimina usuario 1
/user/1/stations	error	Recupera las estaciones de usuario 1	error	error
/login	Crea el token de autenticación para el resto de las peticiones			
/ predictions/08720	error	Recupera las predicciones para el código postal 08720	error	error
/status	error	Recupera el estado de la API, mostrando los recursos que pueden ser utilizados	error	error

Como ya se comentó los ejemplos concretos de las peticiones y respuestas, se encuentran en el **Anexo 1 API Middleware.**

5.2.4.- Uso de HATEOAS para permitir la navegación a los recursos relacionados

Como ya se mencionó, uno de los principales propósitos que se esconden detrás de REST es que debe ser posible navegar por todo el conjunto de recursos sin necesidad de conocer el esquema de URI. Cada solicitud HTTP devuelve la información necesaria para encontrar los recursos relacionados directamente con el objeto solicitado y otras operaciones que pueden realizarse. Los valores {uuidStation}, {postalCode} y {uuidUser} son modificados en tiempo de ejecución según sea el caso para este fin.

Observación:
Actualmente no hay ninguna norma o especificación que defina cómo modelar el principio HATEOAS. Los ejemplos mostrados en esta sección muestran la solución desarrollada para cumplimentar este principio, la llamada al recurso /status devuelve la información sobre los recursos disponibles, así como información relacionada al uso del recurso.

{
"message": "Success, the api status is up!!!",
"links": [
{
"href": "http://meteosalle.local/apiv1/user/{uuidUser}",
"action": "GET ",
"types": [
"multipart/form-data",
"application/x-www-form-urlencoded"
]
},
{
"href": "http://meteosalle.local/apiv1/stations/",
"action": "GET ",
"types": [
"headers: Authorization"
]
},
{
"href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
"action": "DELETE ",
"types": [
"headers: Authorization"
]
},
{
"href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
"action": "PUT ",
"types": [
"headers: Authorization",
"multipart/form-data",
"application/x-www-form-urlencoded"
]
},
{
"href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
"action": "GET ",
"types": [
"headers: Authorization"
]
},
{
"href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
"action": "POST ",
"types": [
"headers: Authorization",
"multipart/form-data",
"application/x-www-form-urlencoded"
]
},
{
"href": "http://meteosalle.local/apiv1/stations/{uuidStation}/history",
"action": "POST ",
"types": [
"headers: Authorization",
"multipart/form-data",

"application/x-www-form-urlencoded"
]
},
{
"href":
"http://meteosalle.local/apiv1/stations/postalcode/{postalCode}",
"action": "GET ",
"types": [
"headers: Authorization"
]
},
{
"href":
"http://meteosalle.local/apiv1/predictions/{postalCode}",
"action": "GET ",
"types": [
"headers: Authorization"
]
},
{
"href":
"http://meteosalle.local/apiv1/user/{uuidUser}",
"action": "GET ",
"types": [
"headers: Authorization"
]
},
{
"href":
"http://meteosalle.local/apiv1/user/{uuidUser}",
"action": "DELETE ",
"types": [
"headers: Authorization"
]
},
{
"href":
"http://meteosalle.local/apiv1/user/{uuidUser}",
"action": "PUT ",
"types": [
"headers: Authorization",
"multipart/form-data",
"application/x-www-form-urlencoded"
]},
{
"href":
"http://meteosalle.local/apiv1/user/{uuidUser}",
"action": "GET ",
"types": [
"headers: Authorization"
]
},
{
"href":
"http://meteosalle.local/apiv1/user/{uuidUser}",
"action": "POST ",
"types": [
"multipart/form-data",
"application/x-www-form-urlencoded"
]
},
{
"href":
"http://meteosalle.local/apiv1/user/login/",
"action": "POST ",
"types": [
"headers: Authorization"
]
}
]
}

5.2.5.- Control de versiones de la API Middleware

Es muy poco probable que las API web en general permanezcan estáticas, sin modificaciones en su código conforme los requisitos empresariales cambien, se pueden agregar nuevas colecciones de recursos, las relaciones entre los recursos pueden cambiar y la estructura de los datos de los recursos puede rectificarse. Si bien la actualización de una API para controlar los requisitos nuevos o diferentes es un proceso relativamente sencillo, se ha tenido en cuenta los efectos que tendrán dichos cambios en las aplicaciones cliente que utilizan la API Middleware. La premisa principal que se ha tenido en cuenta es la de permitir que las aplicaciones cliente existentes sigan funcionando sin cambios y al mismo tiempo dejar que las nuevas aplicaciones cliente aprovechen las ventajas de nuevas características y recursos que se pudieran generar a raíz del proyecto, para cumplimentar esta premisa se ha añadido en la ruta principal de cada recurso una versión de la API de la forma **/apiv1/**… El control de versiones permite que una API web indique las características y recursos que expone para que una aplicación cliente pueda enviar solicitudes que se dirijan a una versión específica de una característica o un recurso. Cada vez que se modifique la API o cambie el esquema de recursos, se agregará un número de versión al URI para cada recurso. Los URI ya existentes seguirán funcionando como antes y devolverá, los recursos conforme a su esquema original.

5.3.- Testing

Cada vez que escribe una nueva línea de código, también se pueden agregar nuevos errores. Para crear aplicaciones mejores y más confiables, se debe probar el código utilizando pruebas al menos unitarias.
Symfony integra una librería independiente llamada PHPUnit, la cual envuelve con el componente PHPUnit Bridge component, para brindar más funcionalidades. PHPUnit está configurado por el archivo phpunit.xml.dist en la raíz del frameworks. A continuación, un ejemplo de su utilización.

```
/**
 * @test
 */

public function shouldReturnExceptionOnFindAllUseCase()
{
    $this->expectException(Exception::class);

        $stations = [];
        $stationRepository = new InMemoryStationRepository();

            $cacheData = new InMemoryCacheDataRepositorys();

        $remoteRepository = new StationRemoteRepositoryApi();
        $tails = new TailsRepositoryRabbit($_SERVER['HOST_RABBIT']);

        $findAllStations = new FindAllStations($stationRepository, $remoteRepository,
```

```
$cacheData, $tails);
    $allStations = $findAllStations();

}
```

Se espera una excepción en findAllStation debido a que InMemoryStationRepository lanza una excepción en esa función.

5.4.- Arquitectura de software de la API Middleware

En los inicios de la informática, la programación se consideraba un arte y se desarrollaba como tal debido a la dificultad que entrañaba para la mayoría de las personas, pero con el tiempo se han ido descubriendo y desarrollando formas y guías generales, con base a las cuales se puedan resolver los problemas. A estas, se les ha denominado arquitectura de software. En el libro "An introduction to Software Architecture", David Garlan y Mary Shaw definen que la arquitectura es un nivel de diseño que hace foco en aspectos "más allá de los algoritmos y estructuras de datos de la aplicación; el diseño y especificación de la estructura global del sistema es un nuevo tipo de problema y durante algunos años se han lanzado varias ideas como:

- Hexagonal Architecture (a.k.a. Ports and Adapters) by Alistair Cockburn and adopted by Steve Freeman, and Nat Pryce in their book Growing Object Oriented Software
- Onion Architecture by Jeffrey Palermo
- BCE by Ivar Jacobson, en su libro Object Oriented Software Engineering: A Use-Case Driven Approach

Aunque todas estas arquitecturas varían algo en sus detalles, son muy similares en sus preceptos y conceptos. Todas tienen el mismo objetivo, que es la separación de las funciones. Todas logran esta separación dividiendo el software en capas. Cada uno tiene al menos una capa para las reglas de negocio y otra para las interfaces. Robert C. Martin (Uncle Bob) ha acuñado estos tipos de arquitectura como “Clean Arquitecture” debido a que cada uno de estos tipos de arquitecturas devuelven una aplicación:

- **Independiente del frameworks.** La arquitectura no depende de la existencia de alguna biblioteca de software. Esto permite utilizar dichos frameworks como herramientas y no como el software que siempre deberemos de cargar para poder ejecutar nuestra aplicación.
- **Testeable** Las reglas de negocio se pueden probar sin la interfaz de usuario, la base de datos, el servidor web o cualquier otro elemento externo a nuestro dominio de la aplicación.
- **Independiente de la interfaz** de usuario o cualquier aplicación externa que consuma los servicios de la aplicación.
- **Independiente de la base de datos**. Puede cambiar, MySQL o Oracle o SQL Server por Mongo, BigTable, CouchDB u otra. Las reglas de nuestro modelo de negocio no están vinculadas a la implementación concreta de la base de datos.
- **Independiente de cualquier dependencia externa**. De hecho, las reglas de negocio simplemente no saben nada sobre el mundo exterior por lo que las modificaciones u actualizaciones externas, no conllevaran una modificación de nuestro dominio en la aplicación.

Para el desarrollo de la API Middleware hemos seguido las pautas de la arquitectura hexagonal, centrando nuestro diseño de software en el dominio, en el núcleo de nuestra aplicación, que a fin de cuentas es la que traduce el problema concreto en las políticas necesarias del aplicativo para la solución concreta. El dominio no es lo único que se necesita para dar solución al problema concreto de la vida real, aunque implemente toda lógica necesaria del modelo de negocio; es una capa a la que se puede acceder desde las capas superiores, nunca a la inversa, que es lo que propone este tipo de arquitectura en su esencia y se logra aplicando el principio de inversión de dependencia (SOLID)[5]. La API Middleware está estructurada en capas abstractas, conceptualmente hablando, que agrupan las "funciones comunes" o relacionadas en concepto. Esta arquitectura no define la cantidad de capas a utilizar, y su premisa fundamental es que las capas superiores del aplicativo no conozcan las implementaciones concretas de una capa superior, de esta forma se logra mantener la complejidad accidental que se genera en extensos proyectos controlada. Se han definido cuatro capas.

5.4.1.- "Interfaz de abstracción del actor"

La capa de "Interfaz de abstracción del actor", en esta capa se definen las implementaciones concretas de los recursos de la API utilizando el frameworks Symfony, se encarga de invocar los servicios de infraestructura, así como los casos de uso que pueda tener el recurso, es la que permite abstraer a la aplicación cliente o actor sobre los "mecanismos" utilizados para crear usuarios, estaciones, etc. Es el punto de entrada de las peticiones externas y el punto de retorno a esa petición. Aquí se definen los controladores.

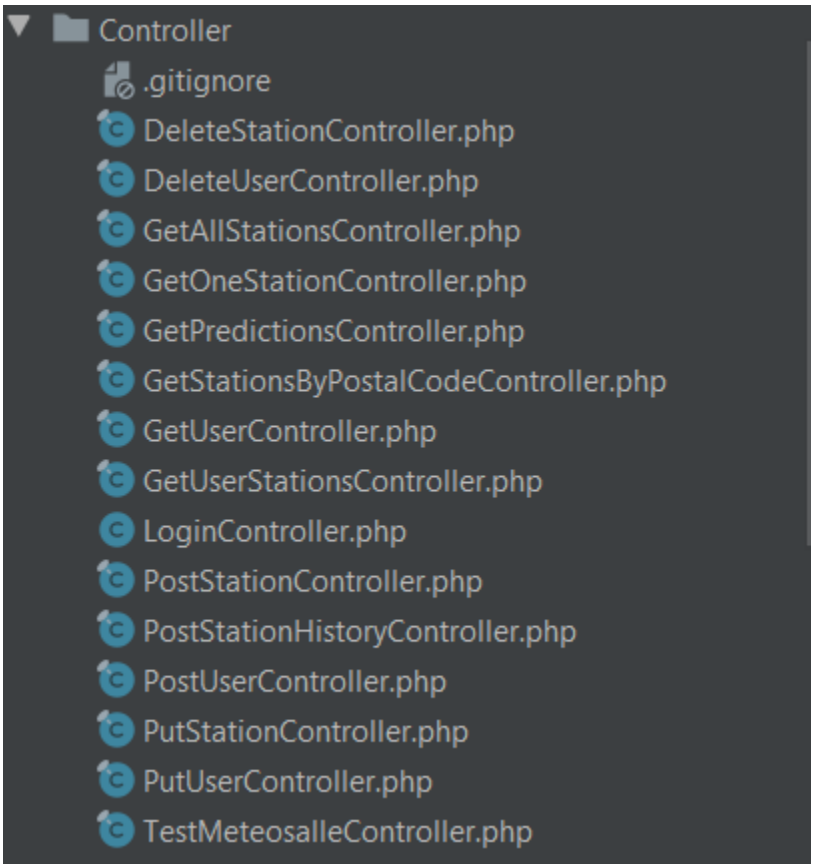


Ilustración 17 Controladores

Ejemplo de la implementación en esta capa GetOneStationController.

```
public function index($uuidStation): JsonResponse
{

    try {

        $stationRepository = new StationRepositoryMysql($_SERVER['HOST_MYSQL']);
        $cacheData = new CacheDataRepositoryRedis($_SERVER['HOST_REDIS']);
        $tails = new TailsRepositoryRabbit($_SERVER['HOST_RABBIT']);

        $findStation = new FindStation($stationRepository, $cacheData, $tails);
        $station = $findStation($uuidStation);

        $stationsLinks = new StationsLinks();
        $jsonResponse = new JsonResponse(array_merge(['station'=>$station-
>getStationLikeArray(),'links'=>$stationsLinks->getLinksForGET($uuidStation)]), 200,
            array(
                'Content-Type' => 'application/json',
                'User-Agent' => 'MeteoSalleMiddel',
        ...
            ));
        $jsonResponse->setEncodingOptions(400);
        return $jsonResponse;
```

Notar como se inicializan los objetos de capas inferiores, primero los servicios de infraestructura y luego el caso de uso al que se le inyectan las dependencias.

5.4.2.- “Aplicación”

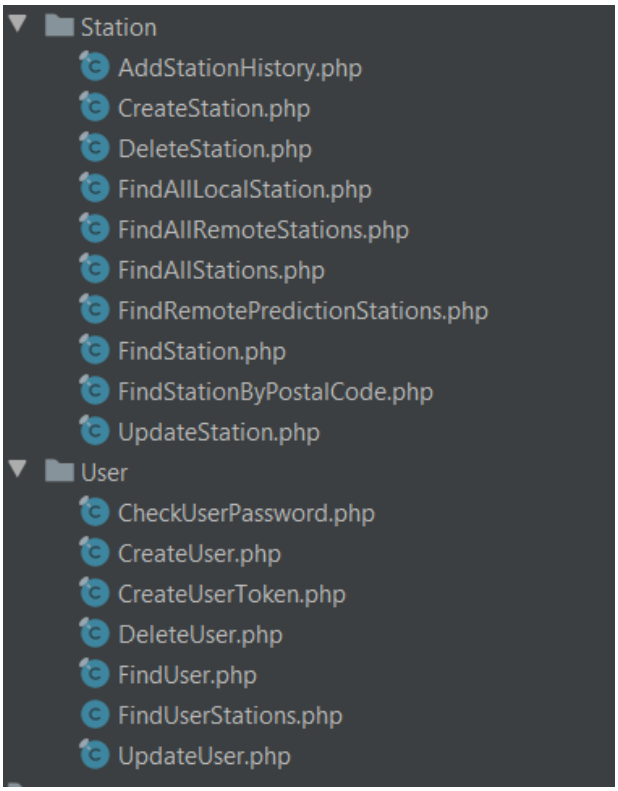


Ilustración 18 Aplicación

La capa “**Aplicación**” contiene los casos de uso de la API, las acciones y prestaciones de la aplicación en general. Encapsula e implementa todas las prestaciones de la aplicación. Estos casos de uso organizan el flujo de datos hacia y desde las entidades, y ordenan a esas entidades que usen sus reglas de negocio para lograr los objetivos del caso de uso a través de interfaces.

Siguiendo la línea del ejemplo anterior, se muestra la parte fundamental del caso de uso.

```
public function __construct
(
    StationRepository $repository,
    CacheDataRepository $cacheDataRepository,
    TailMessageRepository $tailMessageRepository
)
{
    $this->repository = $repository;
    $this->cacheDataRepository = $cacheDataRepository;
    $this->tailMessageRepository = $tailMessageRepository;
}
public function __invoke($uuidStation): Station
{
    $response = $this->cacheDataRepository->find($uuidStation);
    if (!empty($response)) {
    ...
    } else {

        $station = $this->repository->findStation($uuidStation);
```

```
        $event = new OnUpdateStation($station);
        $this->tailMessageRepository->publishEvent($event);

    }
    return $station;
```

Como se puede observar, aquí se establece la política el “contrato” que han de cumplir los controladores para utilizar este caso de uso. Notar que se han inicializado tres servicios de la capa de infraestructura en el controlador y se le han inyectado al caso de uso, sin embargo, el caso de uso desconoce lo que realmente se le está inyectando, por lo tanto no se encuentra acoplado a la implementación concreta del servicio y cumplimos la premisa de la arquitectura hexagonal en cuanto a que una capa inferior no conocerá las implementaciones de una capa superior y con los principios SOLID abierto y cerrado, inversión de dependencias y segregación de interfaces.

5.4.3.- “Servicios de infraestructura”

La capa “**Servicios de infraestructura**” en esta capa se encuentran los algoritmos necesarios para obtener los datos de las diferentes bases datos y elementos externos, generalmente se debe devolver un objeto de dominio.

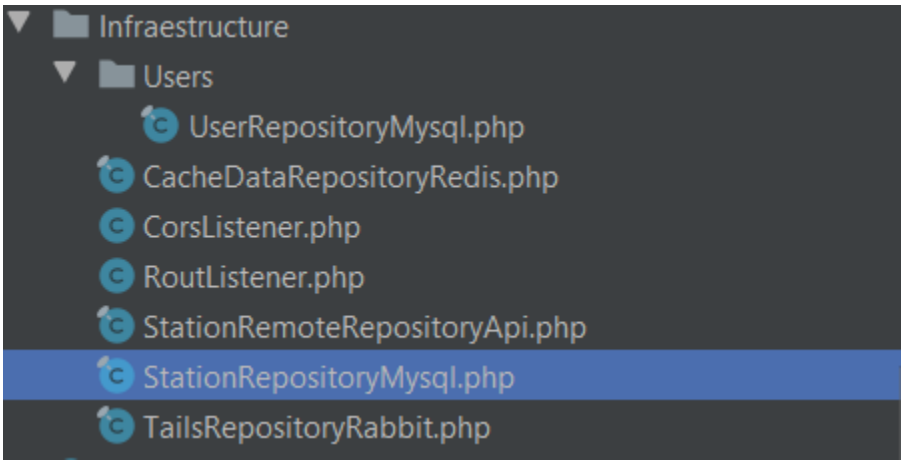


Ilustración 19 Infraestructura

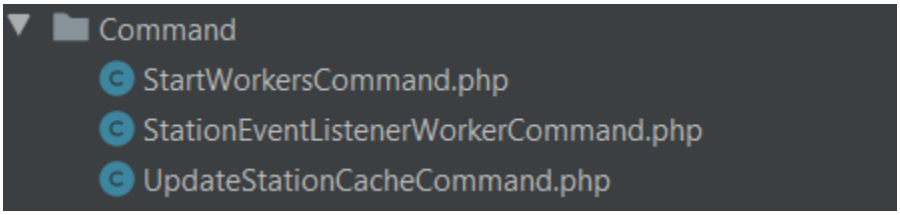


Ilustración 20 Comandos

Siguiendo con el ejemplo se muestra la implementación en el servicio. Notar como implementa la interfaz StationRepository

```
final class StationRepositoryMysql implements StationRepository
…
public function findStation(string $uuidStation): Station
{
    $response = $stmt->fetch(PDO::FETCH_ASSOC);
    if (!empty($response)) {
        $station = new Station
        (
            $response['uuidStation'],
            $response['uuidUser'],
…
        $station->setHistoric($this->findHistorycStation($uuidStation)); …
        return $station;
```

5.4.4.- “Dominio”

La capa “**Dominio**” contiene toda la lógica de la API Middleware y define como las capas exteriores pueden interactuar con esta y sus elementos a través de interfaces o “contratos”.

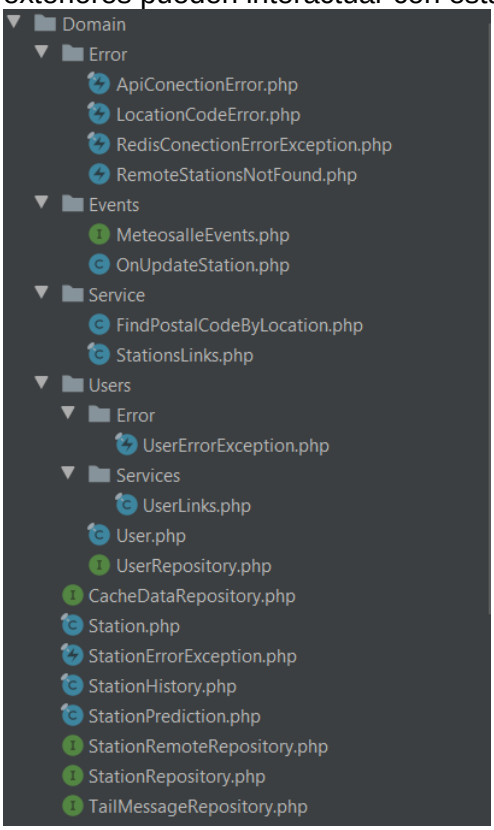


Ilustración 21 Dominio

Siguiendo la línea de ejemplos

```
interface StationRepository
{
…
public function findStation(string $uuidStation): Station;
…
}
final class Station
…
public function __construct(
    $uuidStation,
    $uuidUser,
…

{
    $this->uuidStation = $uuidStation;
    $this->uuidUser = $uuidUser;
…

public function __toString()
{
    return $this->uuidStation;
 …
public function getStationLikeArray(): array
{
    return get_object_vars($this);
}
…
public function setTimestamp($timestamp): void
{
    if (strtotime($timestamp) === false) {
        $this->timestamp = $timestamp;
    } else { …
```

Notar la interfaz, en ella se declara lo que cualquier implementación debe cumplir para poder implementarla, en este caso la implementación es realizada por un elemento de la infraestructura, de esta forma logramos invertir las dependencias y poner en un nivel superior el servicio de infraestructura StationRepositoryMysql, no importa cuál sea la base de datos en el futuro o si la extensión PDO cambie por otra, ninguno de estos cambios conllevara una modificación de las reglas de negocio establecidas en la API Middleware.

La imagen a continuación resume e ilustra con más claridad las capas y sus elementos funcionales dentro de esta.

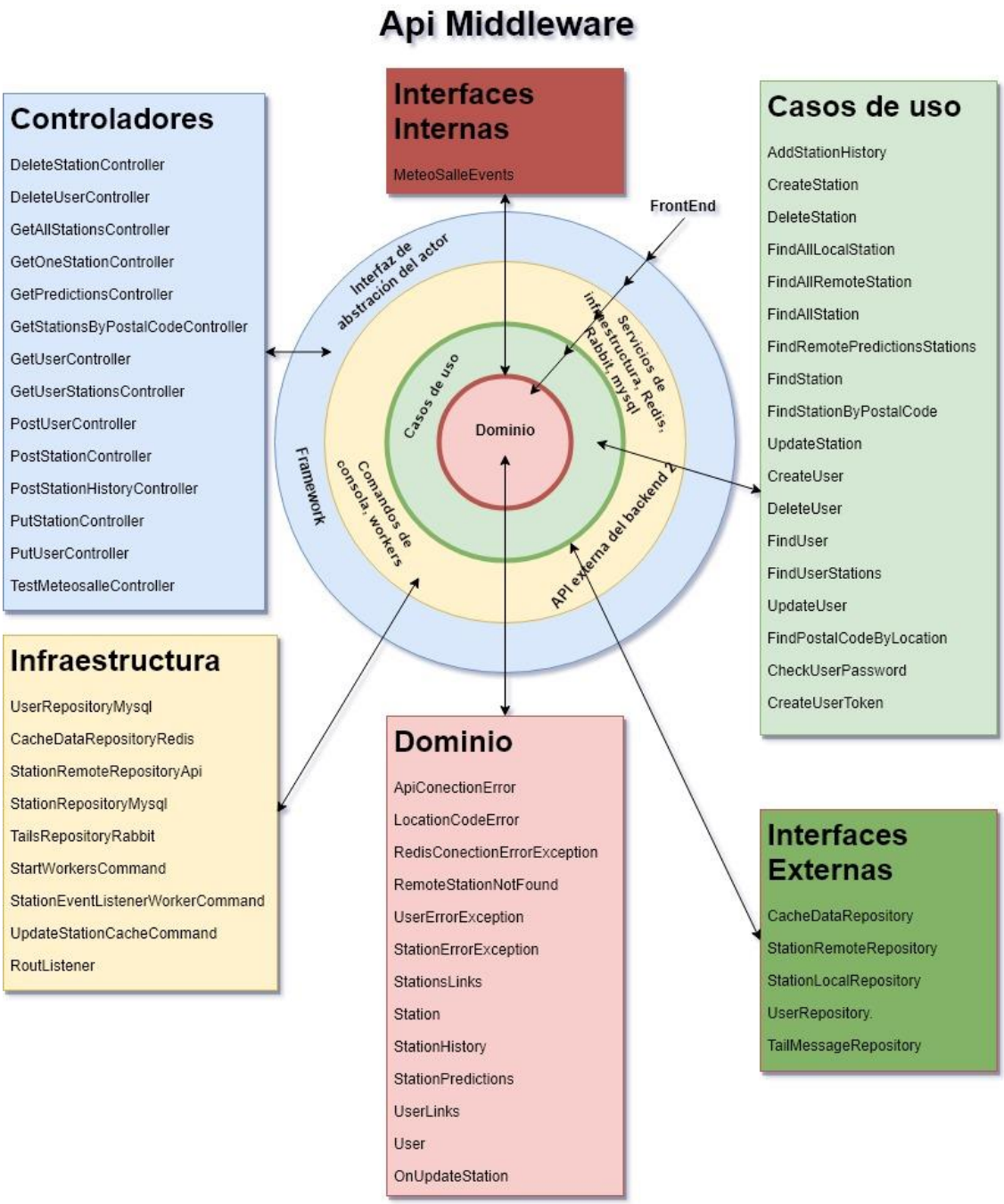


Ilustración 22 Diagrama de arquitectura de la API

5.5.- Estructura general de carpetas

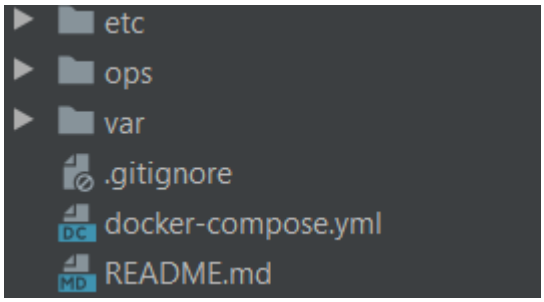


Ilustración 23 Estructura de carpetas 1

- **etc.:** contiene la configuración particular de los contenedores de Docker, meteosalle.sql, configuración de nginx y php.ini

- **Ops:** contiene las particularidades de los docker file, sus personalizaciones para cada servicio, en nuestro caso los comandos especificos que se han de ejecutar en la creacion de la imagen de PHP.
- **Var:** Contiene los datos que se han mapeado de los contenedores como las bases de datos de mysql y redis, así como los logs de nginx.

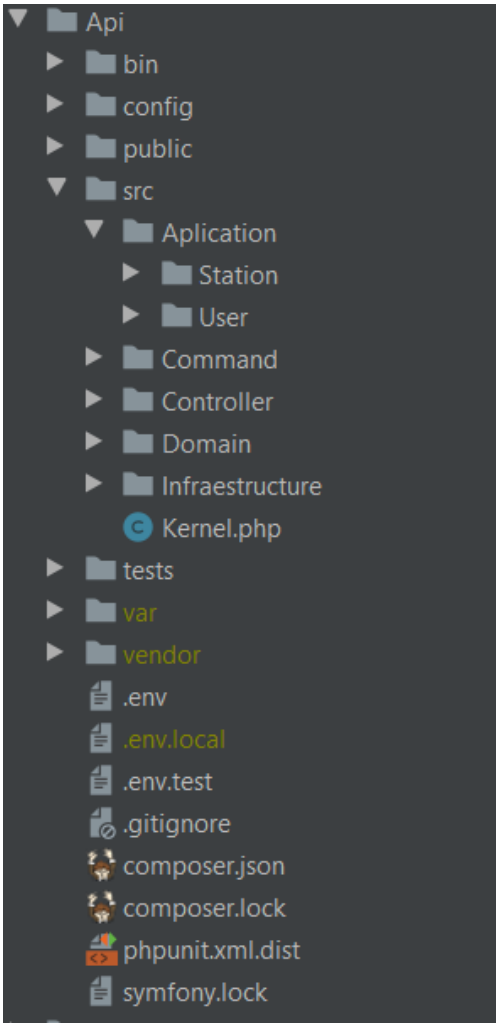


Ilustración 24 Estructura de carpetas 2

- **Bin:** contiene los comandos propios de symfony

- **Config:** Contiene las configuraciones de los bundles, servicios, paquetes y rutas a los controladores, propias del frameworks.

- **Public:** Punto público de la aplicación, a esta carpeta se apunta en la configuración del interprete web que se utilice, en nuestro caso Nginx.

- **Src:** contiene toda la lógica de la aplicación, sus carpetas contenidas reflejan en cierto modo las capas abstractas conceptuales definidas en el apartado "**arquitectura de software de la API Middleware"**, se encuentra además el árbol abierto del contenido de las carpetas**.**
  - **Aplication:** contiene por separado los casos de uso de la aplicación.
  - **Command:** contiene los comandos de consola creados para recrear la base de datos en memoria, así como el "listener" de eventos.
  - **Controller:** Contiene todos los puntos de entrada y salida de la aplicación.
  - **Domain:** Contiene todos los ficheros que componen el dominio de la aplicación, en esta carpeta radica la esencia del aplicativo, la lógica contenida que moldea el modelo de negocio de la Api Middleware.
  - **Infraestructure:** Contiene la implementación concreta de los servicios externos.
- **Test:** Contiene las pruebas que verifican el funcionamiento adecuado de la aplicación.

- **fichero .env:** Contiene las configuraciones que se pueden modificar para adaptar la aplicación a diferentes entornos o cambiar el comportamiento configurable de alguna parte o de toda la aplicación.
- **fichero .env.local:** Contiene las configuraciones personalizadas generalmente al entorno de desarrollo, así como configuraciones que no han de estar en .env cuando se comparte la aplicación.
- **fichero .env.test:** Al igual que las anteriores adaptadas para el tiempo de ejecución de test.
- **fichero .gitignore:** Contiene la referencia de todos los ficheros y carpetas que no se desean publicar en GitHub.
- **fichero .composer.json y Composer.lock:** es el fichero de configuración donde se gestión las dependencias del proyecto.
- **Phpunit.xml:** fichero de configuración de los test**.**
- **Fichero symfony.lock:** fichero mantenido por Flex donde se guardan las referencias de las "Recipes" instaladas.

Estructura de los test

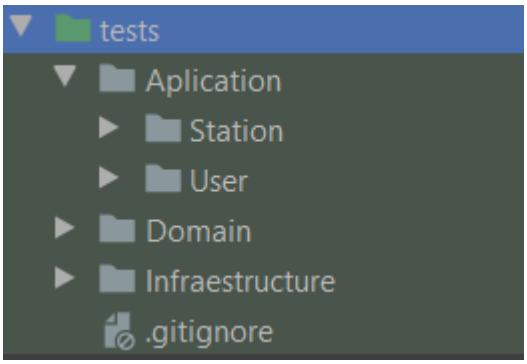


Ilustración 25 Estructura de carpetas 3

La estructura de carpetas de los test simula la estructura de la carpeta src.

- Aplication: Contiene los test para los casos de Station y de User.

- Domain contiene los ficheros para el dominio de la aplicación.

- Infraestructure: contiene los facke que implementan las interfaces que sirven de soporte para realizar los test.

6.- RELACIÓN CON MATERIAS DEL MÁSTER

En este apartado se va a intentar, de alguna manera, justificar el trabajo desarrollado en este proyecto con relación al contenido y al aprendizaje que se ha podido extraer de cursar este máster. Se va a ir asignatura por asignatura, entrando a ver el detalle del granito de arena que ha aportado cada una y ha hecho posible que se construya la aplicación Meteosalle.

6.1.- Entornos web

La primera asignatura que se ve es la de Entornos web, en la cual se definen las herramientas y tecnologías vistas en clase que se aplican a la creación y configuración de los entornos de desarrollo y producción para Meteosalle.

6.1.1.- Vagrant & Docker

Para el tema de la virtualización, es decir, el levantamiento de máquinas virtuales e instancias para la ejecución de la aplicación, se han usado Vagrant y Docker.

Vagrant es un software que permite configurar y definir las propiedades y características de nuestras máquinas virtuales creadas bajo un hypervisor como VirtualBox, VMware, etc. Entre las propiedades más comunes que se pueden “setear” desde el Vagrantfile (el fichero principal de configuración de Vagrant) podemos encontrar la IP, los puertos, los directorios compartidos… Este es un ejemplo de Vagrantfile usado en el proyecto:

```
Vagrant.configure("2") do |config|

    config.vm.box     = "centos/7"
    config.ssh.forward_agent = true

    config.vm.synced_folder "../ansible", "/ansible"
    config.vm.synced_folder "../share/www", "/var/www",
        owner: "vagrant",
        group: "vagrant",
        mount_options: ["dmode=777,fmode=777"]

    config.vm.provider "virtualbox" do |v|
        v.customize ["modifyvm", :id, "--natdnshostresolver1", "on"]
        v.customize ["modifyvm", :id, "--natdnsproxy1", "on"]
        v.customize ["modifyvm", :id, "--memory", "2048", "--cpus", "4"]
        v.name = "ansibleMpwarTfm"
    end

    config.vm.define "web", primary: true do |web|
      web.vm.network :private_network, ip: "192.168.33.56"
      web.vm.network :forwarded_port, guest: 80, host: 8080, auto_correct: true
      web.vm.network :forwarded_port, guest: 443, host: 8081, auto_correct: true
      web.vm.network :forwarded_port, guest: 3306, host: 3306, auto_correct: true
      web.vm.network :forwarded_port, guest: 3000, host: 3000, auto_correct: true
      web.vm.network :forwarded_port, guest: 5000, host: 5000, auto_correct: true
```

Y por su lado, Docker es otro software con fines parecidos pero que en lugar de definir máquinas virtuales permite definir varios contenedores de software en los cuales podemos configurar diferentes servicios en múltiples sistemas operativos. Se ha utilizado en el proyecto para el despliegue de la aplicación debido a su versatilidad y adaptación en varios sistemas operativos además de la agilidad que ofrece para el despliegue.

Ilustración 26 Docker 1

6.1.2.- Git

En segundo lugar, como sistema de control de versiones, como no podía ser de otra manera, se ha usado Git (se trata de la herramienta más conocida para este fin), para así poder gestionar el repositorio con el código y sus constantes cambios y actualizaciones.

Ilustración 27 GitHub

Durante el máster se han ido viendo diferentes servicios para usar repositorios de Git, como GitHub, GitLab, Bitlocker… En concreto, se ha usado GitHub, dado que con sus últimas versiones permite repositorios privados gratuitos para hasta 5 miembros, de modo que con nuestro equipo de tres personas más el tutor estas prestaciones nos iban perfectamente.

Así pues, aún y desarrollando cada uno su propio componente de la aplicación, gracias a este repositorio de GitHub se ha podido realizar la implementación de forma segura y escalable, además de poder tener información completa y constante acerca de los cambios que se hacían, y disponiendo de métodos para revertir cualquier cambio indeseado, sin dejar de lado el poder estar al día de los cambios de los demás.

6.1.3.- Ansible

Y finalmente tenemos Ansible, otra herramienta que se complementa perfectamente por ejemplo con Vagrant, pues permite el aprovisionamiento de máquinas virtuales en cuanto a los paquetes y configuraciones que éstas requieren.

Ansible funciona fundamentalmente a base de roles y tareas, los cuales pueden representar desde la instalación de un servicio a un cambio en un fichero de configuración. Por suerte, hoy en día, hay multitud de roles en Internet para la instalación y configuración automática de gran parte de los principales servicios y paquetes que se utilizan.

Al final, el conjunto de roles y tareas necesarias para la configuración de un entorno se agrupan en uno o varios ficheros llamados playbooks, que vendrían a ser como una especie de manifiesto de configuración. En la siguiente captura se puede contemplar un ejemplo de playbooks usado para aprovisionar un entorno en Meteosalle:

```
roles:
  - role: geerlingguy.ntp
    ntp_timezone: Europe/Madrid

  - { role: geerlingguy.apache }
  - { role: geerlingguy.firewall }
  - { role: geerlingguy.git }

  - role: geerlingguy.repo-remi
    when: ansible_os_family == 'RedHat'

  - { role: geerlingguy.php-versions }
  - { role: geerlingguy.php }
  - { role: geerlingguy.composer }
  - { role: geerlingguy.nodejs }

  - { role: geerlingguy.mysql }
```

Ilustración 28 Roles Ansible

6.2.- Maquetación web

Esta asignatura ha sido naturalmente importante para el desarrollo de la capa frontal de la aplicación, pues de ella se han usado los conocimientos extraídos para la construcción de las diferentes pantallas e interacciones que constituyen la interfaz con el usuario, parte vital de la web.

6.2.1.- HTML & CSS

El primer elemento o concepto para destacar de la asignatura de maquetación web han sido, obviamente, los lenguajes básicos como son HTML y CSS. Tal y como se ha explicado y justificado en el apartado de evaluación tecnológica del componente FrontEnd, son las herramientas estándar para el desarrollo web (en la parte frontal) y se han usado sus últimas versiones “estables”, HTML5 y CSS3.

Sobre el **HTML**, simplemente comentar que, debido a la elección de React como frameworks de desarrollo JavaScript (detalle en las asignaturas de JavaScript/Frameworks), su implementación ha sido un poco atípica. En lugar de tener uno o varios ficheros HTML con las diferentes etiquetas de marcado correspondientes a los diferentes elementos que componen la interfaz, éstas se han ido repartiendo entre los diferentes componentes React, por lo que se obtiene algo así:

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">

    <link type="text/css" rel="stylesheet" href="styles.css">
    <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css"
          integrity="sha384-Gn5384xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/dAiS6JXm" crossorigin="anonymous"
          >
    <link rel="stylesheet" href="https://use.fontawesome.com/releases/v5.5.0/css/all.css"
        integrity="sha384-B4dIYHKNBt8Bc12p+WXckhzcICo0wtJAoU8YZTY5qE0Id1GSseTk6S+L3BlXeVIU" crossorigin="anonymous">
    <title>MeteoSalle</title>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

Ilustración 29 Vista del html

Como se puede observar, el fichero index.html solamente contiene un elemento div que representa la raíz (root) de nuestra aplicación. Más adelante, se asocia esta raíz HTML con la raíz JavaScript, y a partir de esta última se va creando el árbol de componentes, renderizando cada uno su propia porción de código HTML:

```javascript
class StationComponent extends React.Component {

    constructor(props) {
        super(props);
    }

    render() {
        console.log('PROPS STATION', this.props);
        return (
            <div id='station-main' class="container">
                <div class="row">
                    <StationInfoComponent info={this.props.station}/>
                    <StationDataComponent
                        data={this.props.station}
                        openDeleteStationModal={this.props.openDeleteStationModal}
                        openEditStationModal={this.props.openEditStationModal}
                    />
                </div>
            </div>
        );
    }
}
```

Ilustración 30 Ejemplo de componentes de React

En cuanto a **CSS**, se ha optado por el formato tradicional consistente en juntar todo el código de estilos en un fichero externo y luego enlazar este archivo al index.html, como se puede observar en la primera imagen de este subapartado.

6.2.2.- Flex, Grid & Bootstrap

No obstante, la aportación de CSS no termina aquí, pues creemos que merece la pena destacar un conjunto de módulos y frameworks relacionados con el lenguaje que nos han ayudado mucho, sobre todo en tareas de disposición de layouts, adaptación a dispositivos, etc.

- **Flex**: CSS Flex box es un módulo de CSS que nos ayuda a, como bien indica su nombre, diseñar layouts relativamente complejos y adaptados de una forma bastante simple. Mediante un conjunto de reglas o sentencias que implementa este módulo, se pueden ubicar los diferentes elementos que componen la interfaz con la disposición que se necesite en cada caso (centrado, espaciado, al principio, etc.) siguiendo dos ejes direccionales, los llamados main axis y cross axis. Veamos un ejemplo:

```
#login-form {
  padding: 20px 20px;
  height: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: space-around;
  font-family: Arial;
  background-color: lightgrey;
}
```

Ilustración 32 CSS muestra

```
▼<div id="login-page">
  ▼<div id="login-wrapper">
    ▼<form id="login-form">
      ▶<div id="login-header">…</div>
      ▶<div class="login-form-line">…</div>
      ▶<div class="login-form-line">…</div>
      ▶<div class="login-form-buttons">…</div>
      </form>
    </div>
  </div>
```

Ilustración 33 Html muestra

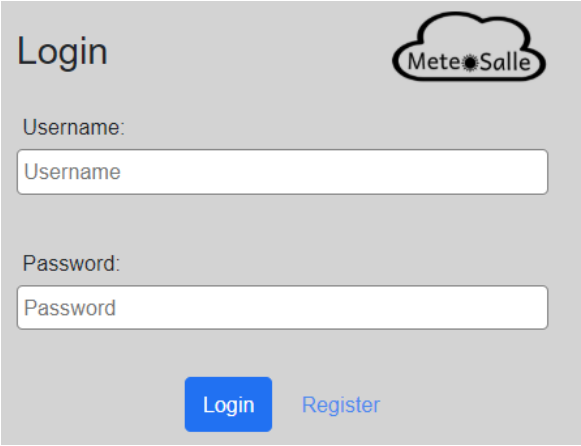


Ilustración 31 Formulario de login

En las capturas superiores podemos ver cómo se aplica Flex box al elemento login-form, especificando lo siguiente:

- flex-direction: column -> Se quiere que los elementos subyacentes a login-form se coloquen uno debajo del otro.
- align-items: center -> Se quiere que los elementos estén centrados respecto a su cross axis (en este caso, el horizontal, pues al decirle que queríamos disposición de columna hemos establecido que el main axis es el vertical).
- justify-content: space-around -> Se quiere que automáticamente se aplique espaciado entre los elementos subyacentes a login-form, respecto a su main axis (vertical).

- **Grid**: CSS Grid es otro módulo de CSS bastante parecido a Flex box, en el sentido que también sirve para establecer una disposición determinada en nuestro layout, pero en este caso con un propósito mucho más específico. Con grid, como también se puede deducir a través de su nombre, se puede definir una “parrilla” para disponer los elementos de la interfaz, indicando el número de columnas y filas, tamaño, espaciado, etc. Vamos con un ejemplo:

```
#form-items {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: 1fr 1fr 1fr;
}
```

Ilustración 35 CSS Grid muestra

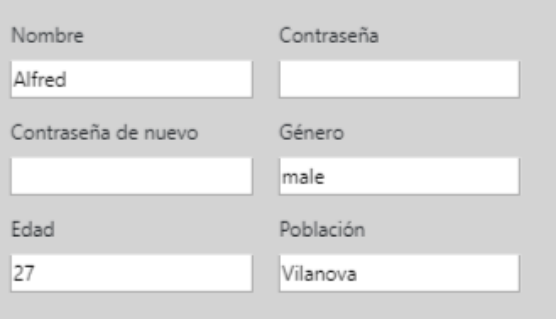


Ilustración 34 Vista de registro

Como podemos ver, basta con indicar el número de columnas y filas deseado (2 y 3 en el ejemplo superior.

Para dispositivos pequeños (móviles) tendríamos que adaptar la disposición a algo así:

```
#form-items {
  display: grid;
  grid-template-columns: 1fr;
  grid-template-rows: 1fr 1fr 1fr 1fr 1fr 1fr;
}
```

Ilustración 37 CSS de otros dispositivos

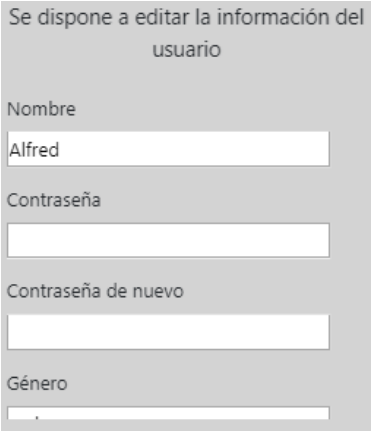


Ilustración 36 Vista de registro, otro dispositivo

- **Bootstrap**: Bootstrap es un frameworks de CSS muy utilizado y con mucha variedad de componentes y funcionalidades. Ofrece plantillas para la creación de formularios, botones, barras de navegación… pero en este proyecto se ha usado una de sus funcionalidades más interesantes que son las columnas de Bootstrap, muy útiles para el aspecto responsive de una web, para la adaptación a diferentes dispositivos. Mediante una serie de clases que se pueden aplicar a nuestros elementos HTML, se puede indicar qué número de columnas (anchura) se quiere que éste ocupe en una determinada anchura (dispositivo), siguiendo este formato: col-{tamaño}-{número de columnas}. Veamos un ejemplo:

```
return (
    <div className='container stations-list-item col-12 col-sm-6 col-md-12 col-lg-12 col-xl-6'>
        <h4>
            <Link to="/station" onClick={() => this.props.selectStation(this.props.station.id)}>
                Station -{this.props.station.id}-
            </Link>
        </h4>
        <table class="table station-table">
```

Se puede ver que se está indicando el número de columnas (en este caso del elemento StationListItemComponent) para todos los tamaños de Bootstrap (cada uno de los cuales tiene asociada unos límites máximo y mínimo de píxeles). En este caso se define que cada elemento de estación ocupe 6 columnas (la mitad) en el tamaño más grande, que ocupe toda la anchura en los dos siguientes, de nuevo la mitad en el penúltimo y el total otra vez en el tamaño mínimo.

Éste sería el resultado en un par de casos (xl y lg):

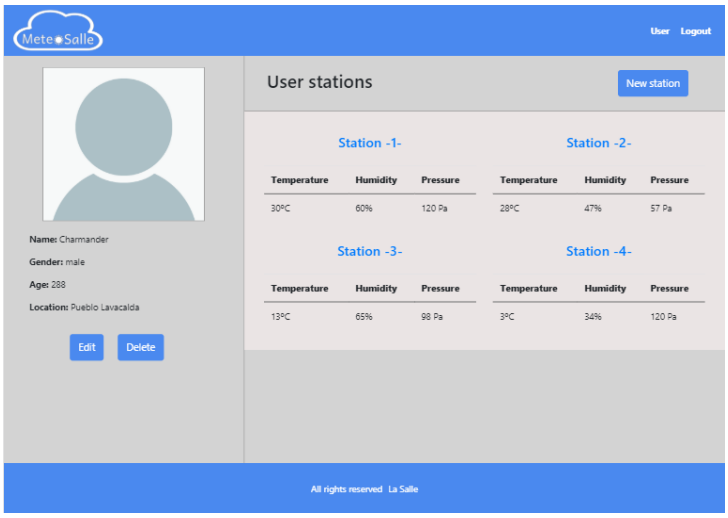


Ilustración 39 Vista de estaciones

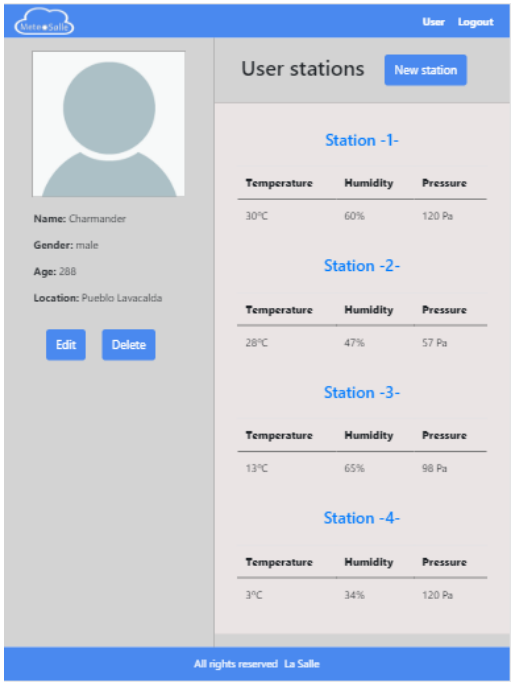


Ilustración 38 Vista de estaciones pequeños dispositivos

6.2.3.- Responsiveness

Para terminar el apartado relacionado con la materia de maquetación web, se hablará un poco de responsiveness, uno de los aspectos más importantes (sobre todo hoy en día) a la hora de desarrollar la capa frontal de una aplicación web, ya que el hecho de que una web se adapte a los diferentes dispositivos desde los cuales se puede acceder es un requisito absolutamente básico.

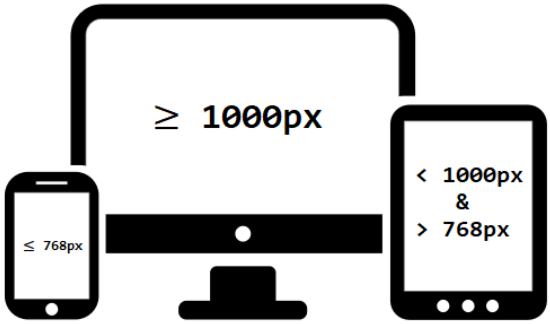


Ilustración 40 Dimensiones de las vistas

En este proyecto se ha tenido muy presente y se han definido, dejando de lado los límites entre tamaños que nos ofrece Bootstrap y que hemos utilizado en casos puntuales, 3 puntos de ruptura, definiendo así 3 layout o disposiciones diferentes con tal de poder mostrar el contenido de la interfaz de forma óptima según el dispositivo en el que se plasme, y así obtener una mejor experiencia de usuario. A continuación, se ofrece una captura para cada una de ellas:

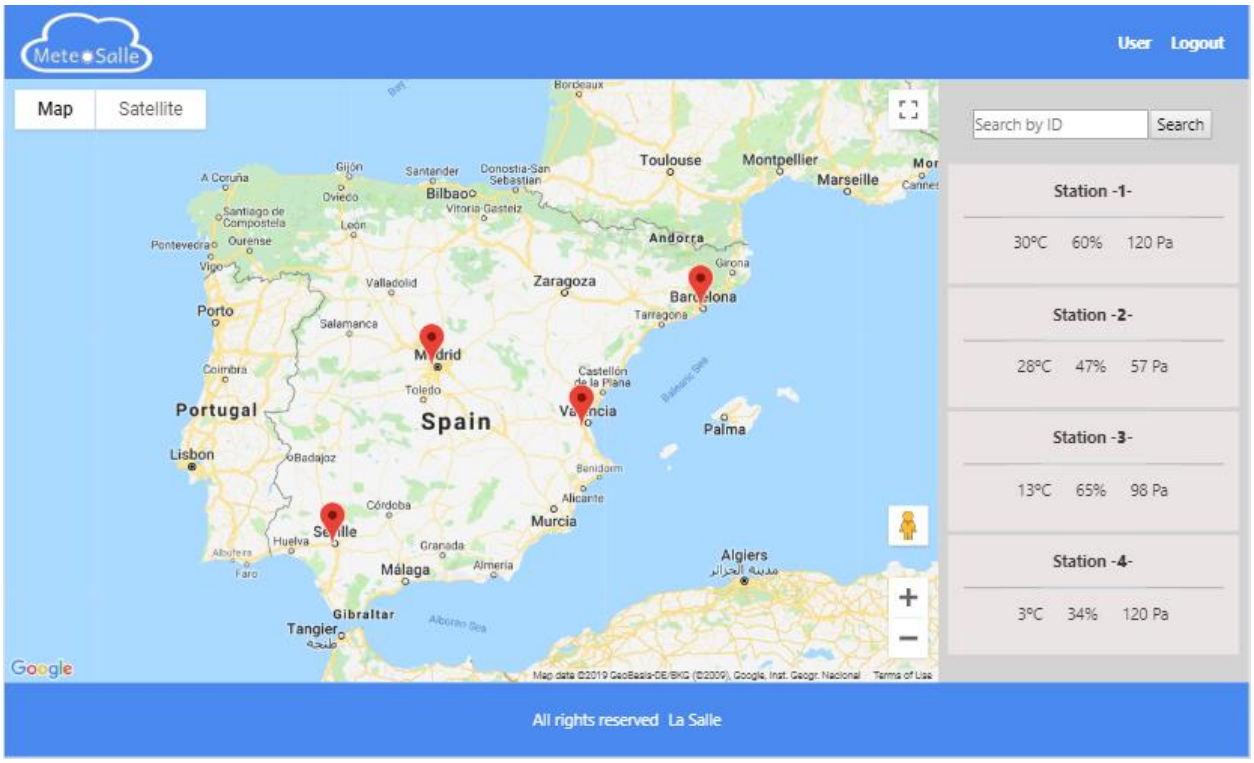


Ilustración 41 Página de inicio. Tamaño "Desktop"

En la captura superior se puede contemplar un ejemplo de la disposición de layout correspondiente al típicamente llamado tamaño desktop, el más grande de los que hemos

contemplado y el escenario base de nuestro desarrollo. Para los formatos tablet y smartphone tendríamos las disposiciones que aparecen en las siguientes figuras:

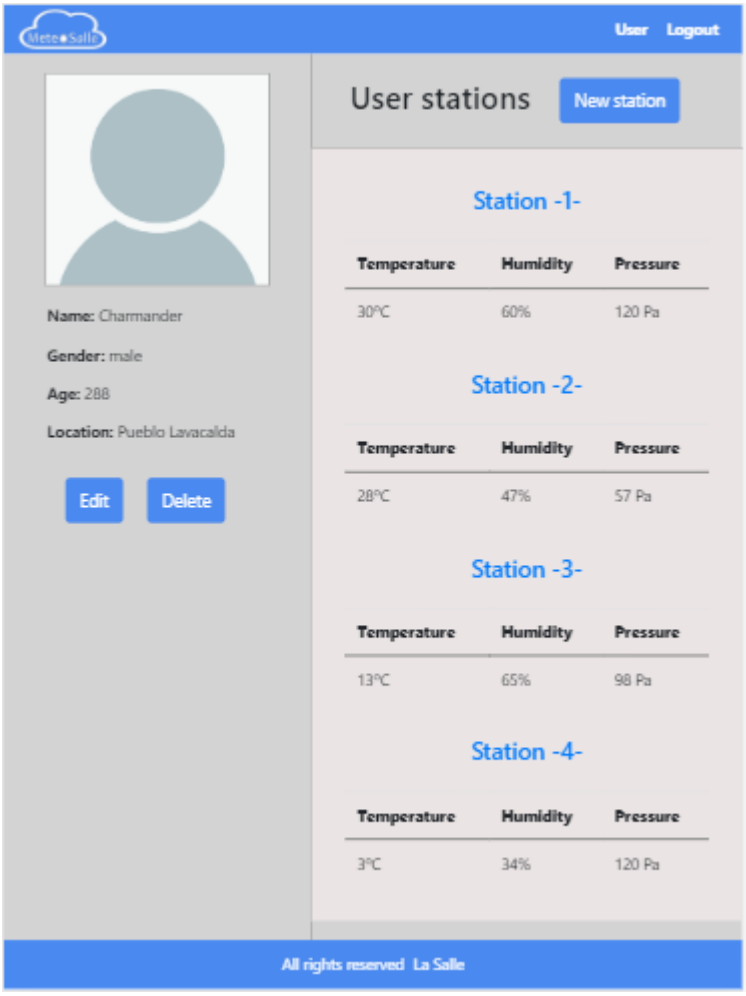


Ilustración 43 Página de inicio. Tamaño "Tablet"

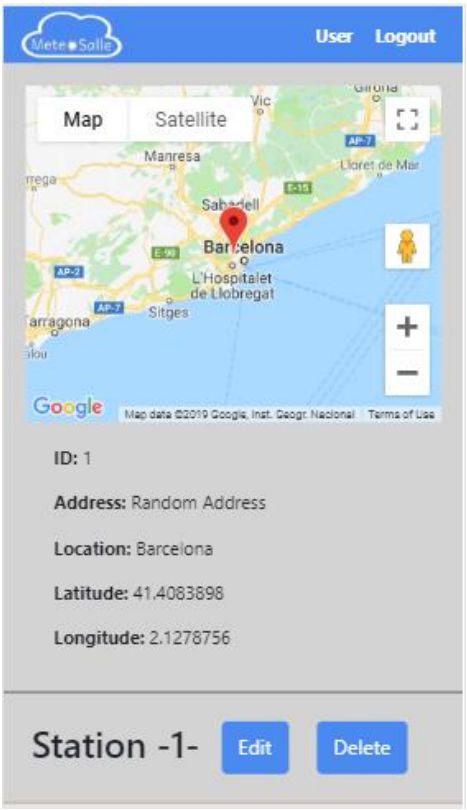


Ilustración 42 Página de inicio. Tamaño "Smart phone"

6.3.- JavaScript

En cuanto a JavaScript, así como sucede con maquetación, es una asignatura que ha sido evidentemente clave para el desarrollo del Frontend de MeteoSalle, permitiendo construir toda la lógica de comportamiento relacionada con la interfaz de la aplicación y con la comunicación con el componente Middle.

6.3.1.- Inclusión y versión

El primer aspecto para comentar sobre JavaScript es el nexo o la conexión mediante la cual se ha asociado el elemento básico del DOM (HTML) con tal de poder asociar un comportamiento a los elementos de la interfaz:

En la asignatura anterior (HTML) se hablaba de la asociación entre la raíz HTML y la raíz JavaScript con tal de efectuar esta conexión, y en la captura lateral se puede observar cómo se lleva a cabo, enlazando el elemento del DOM con ID root al componente App, que como se puede ver en el apartado del diseño de la capa frontal, representa el componente básico de la aplicación en cuanto a React.

```
import Register from './components/Register'
import rootReducer from './reducers'

export const store = createStore(rootReducer)

render(
  <Provider store={store}>
    <BrowserRouter>
        <App />
    </BrowserRouter>
  </Provider>,
  document.getElementById('root')
)
```

Ilustración 44 Asociación Html-Javascript

Así pues, a partir de esta unión ya se puede empezar a desarrollar la implementación de la lógica para que nuestra aplicación funcione. Como también se ha comentado en el apartado anterior, aunque en esta conexión simplemente veamos que estamos enlazando con un div “vacío”, más adelante cada componente irá aportando su granito de arena a la aplicación, no sólo a nivel de HTML, sino también su comportamiento (obviamente JavaScript):

```
class ListComponent extends React.Component {

    constructor(props) {
        super(props);
        this.getRenderStations = this.getRenderStations.bind(this);
    }

    getRenderStations() {
        let component = this;
        return this.props.stations.map(function (station) {
            return <ListItemComponent
                station={station}
                animateStation={component.props.animateStation}
                unAnimateStation={component.props.unAnimateStation}
                selectStation={component.props.selectStation}
            />;
        });
    }

    render() {
        const renderStations = this.getRenderStations();
        return (
            <div id='main-list' class="container col-12 col-sm-12 col-md-4 col-lg-4 col-xl-3">
```

Ilustración 45 Comportamiento

También en el apartado de diseño de la versión del Frontend se comentaba que la versión usada de JavaScript ha sido ECMAScript6, la cual aporta opciones interesantes y mejoras respecto a sus versiones anteriores, algunas de las cuales se comentan a continuación:

- import/export: Posibilidad de importar o exportar funciones, clases, variables, etc. desde o a otros componentes:

```
import React from 'react'

const Footer = () => (
  <div id='footer'>
    <nav id='footer-links'>
      <a id="rights-link" class="footer-link" href="#">All rights reserved</a>
      <a id="vendor-link" class="footer-link" href="#">La Salle</a>
    </nav>
  </div>
)

export default Footer
```

Ilustración 46 Javascript import/export

- Clases: Posibilidad de definir objetos de una clase determinada, con su constructor, métodos, etc.:

```
import React from 'react'
import { Router, NavLink, Link } from 'react-router-dom'
import logo from '../img/logo_white.png';
import * as api from '../utils/apiCalls'

class Header extends React.Component {

    constructor(props) {
        super(props);
    }

    render() {
```

Ilustración 47 Javascript objetos

- Promesas: Mejor y más cómoda gestión de llamadas asíncronas (API típicamente) y actuar en respuesta:

```
for (let param in params) {
    url = url + (param + '=' + params[param]);
}
store.dispatch(actions.setSpinnerMode(true));
fetch(url)
  .then(res => res.json())
  .then(
    (result) => {
      console.log('RESULT', result);
      store.dispatch(actions.setFilteredStations({
```

Ilustración 48 Javascript promesas

- Arrow functions: Sintaxis más reducida y (al acostumbrarse) más sencilla para implementar funciones:

```
<h4>
  <Link to="/station" onClick={() => this.props.selectStation(this.props.station.id)}>
    Station -{this.props.station.id}-
  </Link>
</h4>
<table class="table station-table">
```

Ilustración 49 Arrow functions

6.3.2.- Fetch API

Como se comentaba en la introducción de esta asignatura, uno de los principales cometidos de la capa frontal y que se consiguen gracias a JavaScript, es el de la comunicación con la API del componente Middle. Para ello, se ha hecho uso de una API de JavaScript llamada Fetch API que se parece mucho a la forma tradicional de realizar estas llamadas (con XMLHttpRequest) pero que ofrece un conjunto de características más flexible.

Tiene una única función o componente, fetch(), que puede efectuar llamadas con cualquier método HTTP y se integra muy bien con el ya comentado y tan útil aspecto de ECMAScript 6, las promesas. He aquí algunos ejemplos de llamadas mediante esta API:

```
export function deleteStation($id) {
    fetch("http://192.168.33.50/tfm/Middle/public/index.php/deleteStation/" + $id, { method: 'DELETE' })
        .then(
            (result) => {
              console.log('RESULT DELETE STATION', result.status);
              if (result.status === 200) {
              }
            },
            (error) => {
              console.log('ERROR DELETING', error);
            }
        )
}
```

Ilustración 50 Fetch Api

```
        method: 'POST',
        headers: {
            'Content-Type': 'application/x-www-form-urlencoded'
        },
        body: "lat=" + $latitude + "&long=" + $longitude
    }).then(
        (result) => {
          console.log('RESULT CREATING', result.status);
          if (result.status === 200) {
          }
        },
        (error) => {
          console.log('ERROR CREATING', error);
        }
      )
}
```

Ilustración 51 Fetch Api, método POST

6.3.3.- Web Storage

Otro elemento de JavaScript que se ha usado y que merece la pena comentar es el Web Storage, que no es otra cosa que la posibilidad de almacenar datos localmente en el navegador, ofreciendo posibilidades útiles a las aplicaciones web.

Dentro de esta posibilidad hay dos opciones, usar el local storage, almacenando datos sin fecha de expiración, o el session storage, que almacena los datos solamente durante la sesión (hasta que no se cierre el navegador).

Para este proyecto se ha hecho uso del session storage precisamente para almacenar y gestionar diferentes valores relacionados con la sesión y los tokens de autenticaciones:

```
    method: 'POST',
    headers: {
        'Content-Type': 'application/x-www-form-urlencoded'
    },
    body: "username=" + $username + "&password=" + $password
}).then(res => res.json())
  .then(
    (result) => {
      console.log('RESULT', result);
      sessionStorage.setItem("session", result);
      if (result !== -1) {
        setUserInfo(result);
        getAllStations({ user: result });
      }
      store.dispatch(actions.setSpinnerMode(false));
    },
    (error) => {
```

Ilustración 52 Web Storage

Como se puede observar en estas capturas, el Web Storage almacena información al estilo diccionario, manteniendo los datos estructurados en parejas clave-valor.

```
renderRedirect() {
    if (parseInt(sessionStorage.session) > 0 && typeof sessionStorage.session !== "undefined") {
        console.log('SHOULD GO TO HOME!');
        return (
            <Redirect to='/home'/>
        );
    }
}
```

Ilustración 53 Web Storage 1

```
    if (parseInt(sessionStorage.session) < 1 || typeof sessionStorage.session === "undefined") {
        console.log('MUST LOGIN!');
        return (
            <Redirect to='/login'/>
        );
    }
}
```

Ilustración 54 Web Storage 2

6.3.4.- Módulos y paquetes

Y para terminar este apartado de JavaScript, se comentarán dos aspectos clave en la construcción de aplicaciones web como son la **modularización** y la **paquetización**, dos elementos que definen cómo se reparte el trabajo en nuestra aplicación de forma interna y qué funcionalidades o elementos se adquieren desde fuera.

- Modularización: Es importante en cualquier aplicación que se pueda considerar grande o relativamente grande definir una estructura de módulos o componentes para poder garantizar una construcción equilibrada y escalable, así como para establecer un orden y un sistema organizativo.

  En lo que a nuestro código JavaScript se refiere, tal y como se ha comentado en diferentes momentos a lo largo de esta memoria, se ha optado por usar un frameworks de desarrollo llamado **React** que refuerza este concepto de modularización, no sólo a nivel de JavaScript, también a nivel de HTML, como hemos podido observar en apartados anteriores.

  Así pues, la interfaz de MeteoSalle ha sido construida a base de módulos (llamados componentes) que han ayudado a una elaboración planificada y equilibrada para obtener el resultado final. A continuación, podemos ver un esquema de la composición de la interfaz:

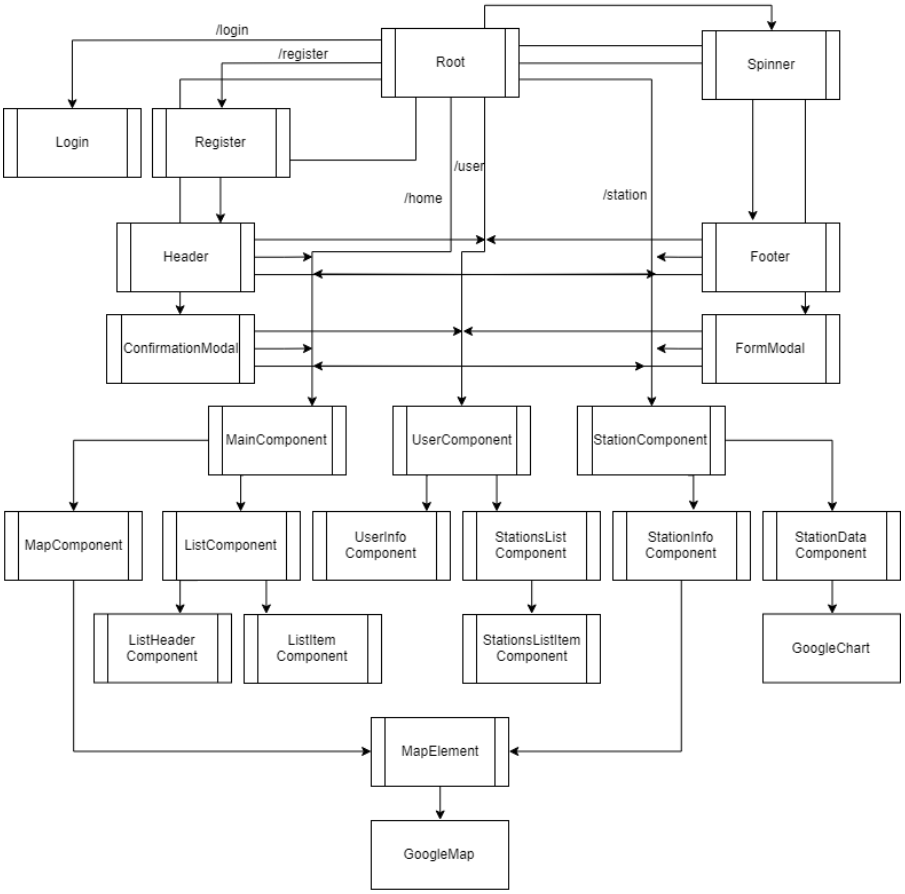


Ilustración 55 Diagrama de componentes

- Paquetización: Y así como debe haber una repartición de lógica y tareas de forma interna en la aplicación, es importante también saber identificar qué parte del trabajo podemos evitar, ya sea porque podemos usar un componente que nos lo implementa, o bien porque podemos encontrar una librería o frameworks que nos facilita mucho las cosas (React, por ejemplo).

  Sea como sea, es necesario algún sistema para la gestión de estos paquetes o dependencias que queramos adquirir. Para este proyecto se ha hecho uso de **npm**, el gestor de paquetes de Node, que hoy en día es el más usado y extenso que existe. Este software incluye en el proyecto un fichero de configuración (package.json) en el que se definen las dependencias a instalar, así como sus versiones. A continuación, se adjunta captura del de la capa frontal de MeteoSalle:

```
"dependencies": {
  "bcryptjs": "^2.4.3",
  "chai": "^4.2.0",
  "chromedriver": "^74.0.0",
  "i18next": "^14.0.1",
  "i18next-browser-languagedetector": "^3.0.3",
  "mocha": "^6.1.4",
  "prop-types": "^15.6.1",
  "react": "^16.8.1",
  "react-dom": "^16.8.1",
  "react-google-charts": "^3.0.11",
  "react-google-login": "^5.0.4",
  "react-google-maps": "^9.4.5",
  "react-i18next": "^10.0.0",
  "react-redux": "^5.0.7",
  "react-router-dom": "^5.0.0",
  "redux": "^3.5.2",
  "selenium-webdriver": "^4.0.0-alpha.1"
},
```

Ilustración 56 Dependencias web

6.4.- PHP

La asignatura de PHP ha sido esencialmente fundamental para este proyecto. En primer lugar, porque era uno de los requerimientos del proyecto el uso de este lenguaje, y, sobre todo, porque la implementación del componente intermedio, la API, se ha efectuado con este lenguaje de programación.

6.4.1.- PHP

La mayor parte de la asignatura de PHP fue dedicada precisamente a estudiar y analizar las principales características de este lenguaje, y a continuación se expondrán las que se han aplicado de forma más explícita junto con algún ejemplo:

- **Orientación a objetos:** La orientación a objetos constituye uno de los paradigmas de programación más modernos y usados hoy en día, y PHP no deja este aspecto de lado en sus versiones más recientes. La definición de clases, atributos y métodos ha permitido modelar de forma eficiente los conceptos que requerían de tratamiento en el Middleware, como era el caso de estaciones meteorológicas y usuarios.

```php
use \Exception;
use Throwable;

final class ApiConectionError extends Exception
{

    private const MESSAGE = 'Api conection error exception';

    public function __construct($message = self::MESSAGE, $code =
    {
        parent::__construct($message, $code, $previous);
    }
}
```

Ilustración 57 Excepción de conexiones externas

- **Gestión de códigos HTTP:** Al tratarse de una API REST, han sido necesarias las nociones básicas de códigos de estado HTTP, para la óptima manipulación de respuestas hacia la capa frontal. De este modo, otro de los conceptos introducidos y tratados en la asignatura de PHP ha sido de alta utilidad.

```php
$cacheData = new CacheDataRepositoryRedis($_SERVER['HOST_REDIS']);
$tails = new TailsRepositoryRabbit($_SERVER['HOST_RABBIT']);

$findStation = new FindStation($stationRepository, $cacheData, $tails);
$station = $findStation($uuidStation);

$jsonResponse = new JsonResponse($station->getStationLikeArray(), 200,
    array(
        'Content-Type' => 'application/json',
        'User-Agent' => 'MeteoSalleMiddel',
```

Ilustración 58 Ejemplo de código de respuesta http

- **Gestión de bases de datos:** La manipulación directa de una base de datos es otra de las características usuales que se puede dar en una aplicación web hoy en día, y PHP dispone de una librería built-in, llamada PDO, que permite, definidos los parámetros básicos de entrada y conexión, realizar comunicaciones bidireccionales a nivel de datos para poder introducir y recuperar datos de un sistema gestor.

```php
try {
    $this->conect = new PDO(
        "mysql:host=$host;dbname={$_SERVER['DB_MYSQL']}",
        $_SERVER['USER_MYSQL'],
        $_SERVER['PASS_MYSQL']
    );

} catch (PDOException $exception) {
    throw new StationErrorException('Internal Server error', 500);
}
```

Ilustración 59 Conexión a base de datos

- **Composer:** Finalmente, y no por ello menos importante, usualmente no todo lo que se desea hacer se puede hacer con las capacidades básicas de PHP, por lo que puede ser necesario recurrir a librerías externas que sí nos solventen esos requerimientos. Para ello, existe Composer, un gestor de paquetes para PHP (como lo era NPM para JavaScript) que nos abstrae de la incorporación e invocación de elementos externos a la propia aplicación.

```
use Predis\Client;
use Predis\Connection\ConnectionException;

final class CacheDataRepositoryRedis implements CacheDataRepository
{

    private $redis;

    public function __construct(string $host)
    {
```

Ilustración 60 Predis librería externa instalada con composer

6.4.2.- GO

Dentro de la asignatura de PHP se dedicaron algunas sesiones para conocer el lenguaje de programación GO y este, ha sido una pieza clave en el desarrollo de la API del Backend.

GO es un lenguaje de código abierto, adoptado por Google poco después de su creación y, además, es mantenido por esta misma compañía, con actualizaciones semestrales.

Entre sus características principales cabe destacar la gestión de tareas de forma concurrente, el hecho de que su diseño se lleva a cabo teniendo en cuenta las capacidades tecnológicas actuales como la computación en red o multinúcleo, además, consigue combinar la facilidad de programación de un lenguaje interpretado de forma dinámica, con la eficiencia y seguridad de un lenguaje compilado de escritura estática.

Como hemos comentado anteriormente, GO es el motor principal de la API del backend y gracias a su gestión del paralelismo, es capaz de recibir y enviar peticiones al mismo tiempo y a múltiples fuentes, tal y como se puede apreciar en la siguiente imagen que muestra consola de la API:

Cabe destacar también que aunque Go es un lenguaje joven, es mantenido por una comunidad muy activa y existen multitud de paquetes, IDE's de desarrollo y frameworks, que lo hacen muy atractivo y que se verán más adelante en el apartado de diseño.

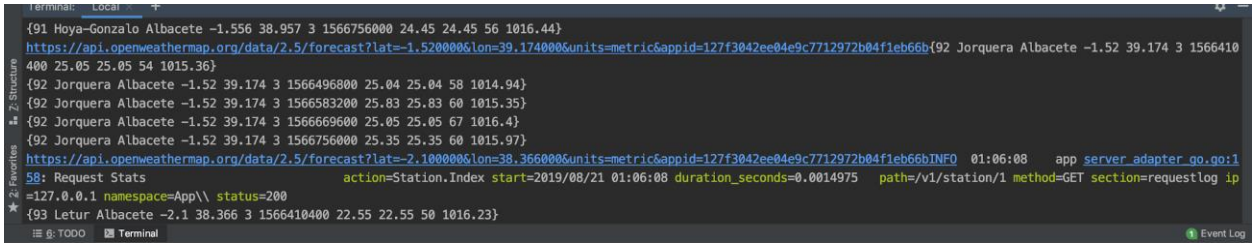


Ilustración 61 Petición de Api externa

6.5.- Desarrollo eficiente de software

La asignatura de Desarrollo eficiente de software es una de las que podríamos definir como de las más interdisciplinares en cuanto al desarrollo de este proyecto, pues se han extraído nociones y metodologías que se han usado en la implementación de las diferentes capas de la aplicación. Vayamos paso por paso viendo los diferentes conceptos aplicados.

6.5.1.- SOLID

Se empezará esta asignatura repasando los principios SOLID y viendo ejemplos de su seguimiento pertenecientes a componentes y módulos de MeteoSalle:

- **S**ingle responsability: Este primer principio nos habla de la distribución lógica de carga en cuanto a los componentes que conformen nuestras aplicaciones. Asignar una única responsabilidad a un módulo/objeto/clase, con tal de obtener un esquema más sencillo a nivel conceptual y un sistema más modular.

  Un buen ejemplo de este punto de vista podría ser la implementación que se ha seguido en la capa frontend con ReactJS. El hecho de trabajar con componentes ayuda a modularizar el sistema y a asignar responsabilidades específicas a cada uno de ellos. En la siguiente imagen podemos ver cómo se realiza esta distribución en los componentes que componen la interfaz de la pantalla principal (mapa y lista de estaciones):

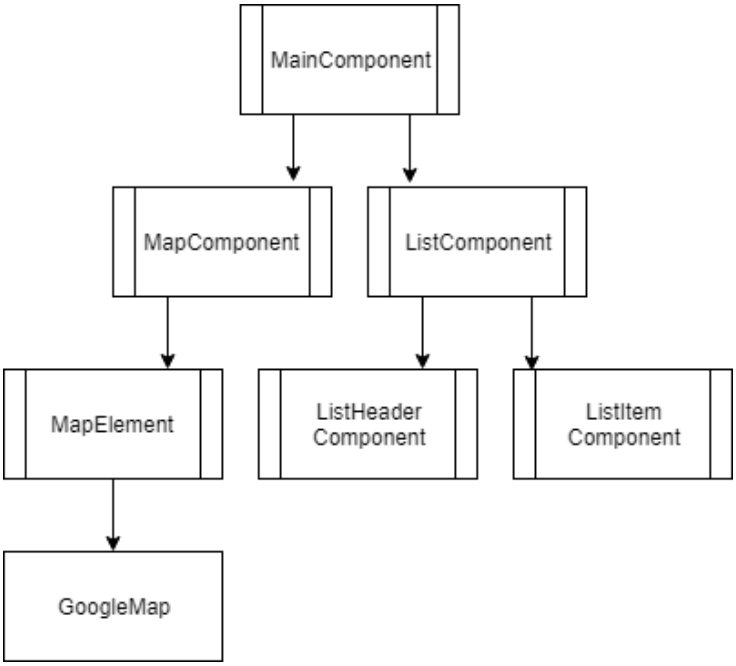


Ilustración 62 Componentes pantalla principal

- **O**pen/Close: Tratar siempre de tener una implementación que esté abierta a la extensión (añadir funcionalidades o nuevos componentes) pero cerrada a la modificación (sin tener que realizar grandes cambios para poder llevar a cabo dichas extensiones).

  Se podría encontrar un ejemplo de la aplicación de este principio en la herramienta de gestión de idiomas y literales que se implementa en el Frontend de la aplicación, pues en caso de querer añadir un nuevo idioma, bastaría con incluir los literales en el repositorio único donde están los de los demás idiomas:

```
const resources = {
  en: {
    translation: {
      "Language": "en",
      "Login": "Login",
      "Register": "Register",
      "Username": "Username",
      ...
      "NumericRegex": "Introduce a valid value (integer)"
    }
  },
  es: {
    translation: {
      "Language": "es",
      "Login": "Login",
      "Register": "Registro",
      "Username": "Usuario",
      ...
      "NumericRegex": "Introduzca un valor válido (entero)"
    }
  },

  // MORE LITERALS SET LANGUAGES HERE

};
```

Ilustración 63 Gestión de idiomas

- Sustitución de **L**iskov: Poder sustituir implementaciones que hereden de una misma clase sin que se produzcan efectos colaterales inesperados.

  Un ejemplo para este principio lo podríamos encontrar en el componente Middle, donde se han modelado diferentes tipos de errores y excepciones que extienden de Exception, y que por supuesto respetan el comportamiento de la clase de la que heredan.

```
use \Exception;
use Throwable;

final class ApiConectionError extends Exception
{

    private const MESSAGE = 'Api conection error ex

    public function __construct($message = self::ME
    {
        parent::__construct($message, $code, $previ
    }
}
```

Ilustración 64 Ejemplo de implementación de Liskov

- **S**egregación de interfaces: Extracción de interfaces de forma natural como parte del flujo del desarrollo, en lugar de forzar a sus clientes a implementar métodos que no per toquen.

  De nuevo se puede encontrar un ejemplo en la capa Middleware, donde para acceder por ejemplo a los usuarios del sistema, al disponer de diferentes tipos de repositorios (MySQL, Redis, Rabbit) se ha generado una interfaz la cual abstrae cada implementación especificando los métodos “a su manera”, brindando un adaptador y desacoplando el código:

```
interface UserRepository
{
    public function createUser(User $User): void;

    public function findUser(string $uuidUser): User;

    public function updateUser(User $user): void;

    public function deleteUser($uuidUser): void;
}
```

Ilustración 65 Interfaz de desacople del repositorio de usuarios

- Inversión de **D**ependencias: Desacoplamiento. Recibir dependencias en forma de interfaces para no depender de detalles de implementación.

  Muy en relación con el ejemplo anterior, los casos de uso que necesitan acceso a los repositorios de usuarios, en lugar de recibir las implementaciones concretas de dichos repositorios, especifican en su contrato la recepción de la interfaz UserRepository, con tal de dotar de flexibilidad y agilidad a los servicios de la API.

```
public function __construct(UserRepository $repository, C
{
    $this->repository = $repository;
    $this->cacheDataRepository = $cacheDataRepository;
}

public function __invoke($uuidUser)
{
    $query = md5($uuidUser);
    $this->repository->deleteUser($uuidUser);
```

Ilustración 66 Ejemplo de inversión de dependencias

6.5.2.- Arquitectura hexagonal

Para hablar de arquitectura hexagonal hay que centrarse en el Middleware, en la API. Se trata de un paradigma que divide las aplicaciones en capas, de exterior a interior, con el objetivo de aislar el dominio, su premisa fundamental es que una capa inferior no puede conocer nada de una capa superior, lo que se modela en el core de la aplicación.

A continuación, se muestra como se ha seguido este patrón de diseño en el componente Middleware, con ejemplos y detalle para cada capa:

- **Infraestructura**: Capa para los componentes típicamente llamados controladores, usualmente para gestionar entrada y salida de datos, así como otros servicios implementados de forma ligada a alguna tecnología (cómo por ejemplo, un repositorio

para MySQL).

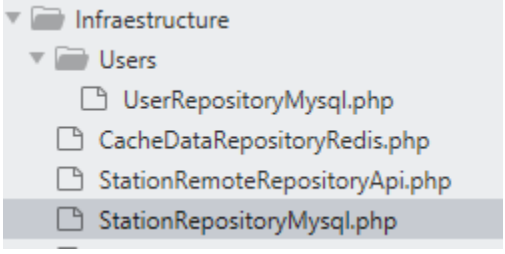


Ilustración 68 Capa de insfraestructura

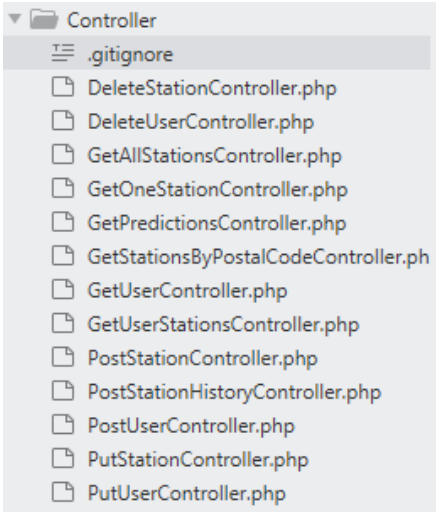


Ilustración 67 Capa de insfraestructura Controladores

- **Aplicación**: Capa para los llamados casos de uso, los propósitos/acciones de la API. Son servicios que conocen los modelos del dominio y orquestan, no mantienen la lógica de negocio de la aplicación.

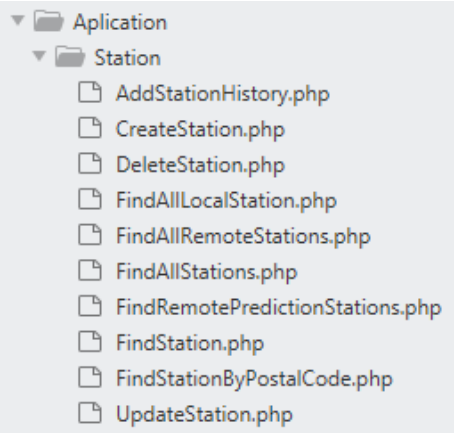


Ilustración 69 Capa de aplicación

- **Dominio**: Los modelos en sí, las representaciones con las que trata la API. En nuestro caso serían usuarios y estaciones, así como errores, eventos, representa el núcleo de la aplicación, donde se mantiene toda la lógica del modelo de negocio:

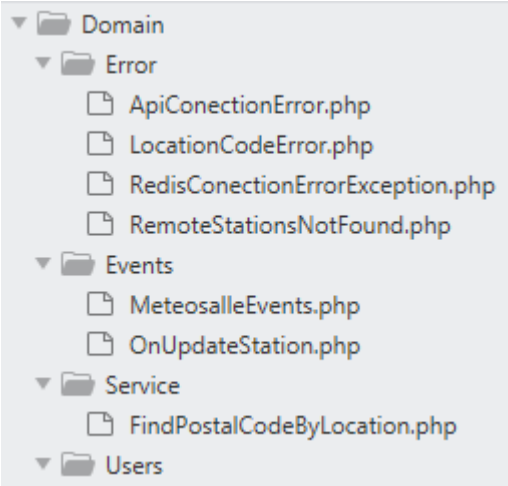


Ilustración 70 Capa de dominio

6.5.3.- Testing

La implementación de tests es otro de los aspectos interesantes a comentar relacionados con el desarrollo eficiente de software. En las diferentes partes de la aplicación se han implementado diferentes tipos de tests; unitarios, de comportamiento, de aceptación… Veámoslos:

6.5.3.1.- Selenium tests

Para la implementación de tests en la capa frontal se ha usado Selenium Web driver como herramienta principal. Esta herramienta permite automatizar una serie de procesos para que el navegador los vaya recorriendo, de modo que junto a otras librerías como Mocha y Chai se ha podido implementar un conjunto de tests bastante extenso para cada una de las pantallas de la interfaz: Login, Registro, Home, User y Station.

A continuación, se muestran algunos ejemplos de librerías y módulos que forman parte de esta herramienta y que han sido muy útiles de cara a testear el Frontend:

- await, until: Uno de los aspectos básicos de estos tests es saber jugar con la latencia, pues el tiempo que tarda en mostrarse un contenido en una página o en renderizar un elemento en concreto no es exacto ni constante. Con esto en mente, Selenium web driver incluye un par de librerías interesantes como await y until, que nos permiten establecer esperas a determinados eventos (por ejemplo, esperar a que una imagen esté presente en la página, esperar que haya desaparecido, etc.). Todo esto obviamente con un valor de TimeOut para no tener esperas infinitas.

```
it('should go to MeteoSalle Home page and check the header logo', async () => {
    await driver.get('http://localhost:3000');
    await driver.wait(until.stalenessOf(driver.findElement(By.id('spinner-img'))));
```

Ilustración 71 Test

- findElement, by: Como complemento interesante a los módulos anteriores y como pieza fundamental de estos tests, la posibilidad de encontrar elementos a través de diferentes propiedades: ID, clase, propiedades de CSS, etc. Con estas librerías y una buena estrategia de IDs y clases realizar los tests de interfaz puede llegar a ser bastante sencillo.

```
it('should go to MeteoSalle Login page and check the form inputs', async () => {
    await driver.get('http://localhost:3000');
    const usernameInput = await driver.findElement(By.id('username'));
    const passwordInput = await driver.findElement(By.id('password'));
```

Ilustración 72 Test 1

- sendKeys, clic: Estos otros dos módulos son los que básicamente nos permiten interactuar con la interfaz, ya sea mediante la introducción de texto en un input de formulario o realizando clicks en botones, enlaces, imágenes, etc.

```
it('should go to MeteoSalle Register page and perform a bad Register', async () => {
    await driver.get('http://localhost:3000');
    await driver.findElement(By.id('link-button')).click();
    await driver.findElement(By.id('username')).sendKeys(badCredentials.username);
    await driver.findElement(By.id('password')).sendKeys(badCredentials.password);
    await driver.findElement(By.id('password-again')).sendKeys(badCredentials.passwordAgain);
```

Ilustración 73 Test 2

- expect, equal: Ya entrando en el terreno de Chai, al final el objetivo de los tests es realizar comprobaciones y aserciones de valores, y una vez localizados los elementos sobre los cuales queremos indagar (con getText o getAttribute) ya se puede realizar este paso final.

```
await driver.wait(until.stalenessOf(driver.findElement(By.id('spinner-img'))));
const errorMessage = await driver.findElement(By.id('register-form-error'));
const visibility = await errorMessage.getAttribute('className');
expect(visibility).to.equal('');
```

Ilustración 74 Test 3

6.5.3.2.- Revel TestSuite

Para la parte del Backend donde se ha hecho uso del lenguaje de programación Go, se ha optado por utilizar una herramienta propia del Framework de desarrollo. En este caso, Revel proporciona un módulo de pruebas denominado testrrunner, que permite realizar acciones de escritura y ejecución habituales en el desarrollo de una Api de forma rápida.

Se basa en una estructura de pruebas denominada TestSuite que proporciona ayuda para la emisión de peticiones a su solicitud y para afirmar cosas (aserciones) acerca de la respuesta. Este suite de testing cuenta además con la llamada dos funciones especiales, after() y before() que se invocan después o antes de la ejecución de cada método de prueba, si están presentes.

Ejecución de los tests:

- **Interactivamente**: Desde el navegador web, esta función es muy útil en la fase de pruebas en desarrollo, ya que permite acceder a un panel que muestra el resultado de las pruebas y permite su ejecución, de forma interactiva:

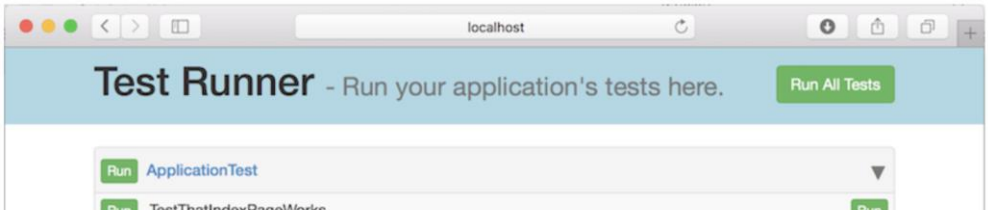


Ilustración 75 Test 4

- **No interactivamente:** Desde la línea de comandos, en este caso usaremos esta funcionalidad para realizar pruebas automatizadas en producción.

6.6.- Bases de Datos

La asignatura de Bases de Datos estuvo especialmente focalizada en MySQL (en un 80%), mientras que al final se vieron algunas pinceladas de MongoDB (20%). Ambos sistemas gestores de bases de datos han sido usados en esta implementación (en la API y en el Backend respectivamente) lo cual dota al proyecto de riqueza y variedad pues se trata de dos herramientas muy distintas (relacional vs. no relacional). Además, se ha usado también otro sistema, Redis. Los tres se detallan a continuación:

6.6.1.- MySQL

Así pues, para la base de datos del componente Middleware se ha usado este sistema gestor, uno de los más comunes y usados y de tipo relacional, es decir, basado en las relaciones entre objetos o registros de diferentes tablas. Se va a entrar en el detalle de las dos características vistas en clase más estrechamente aplicadas en este proyecto; la elección de tipos de datos y los diferentes tipos de sentencias:

6.6.1.1.- Tipos de datos

Con tal de ofrecer variedad, a continuación, se detalla un conjunto de tipos de datos modelados en las tablas de la base de datos y su justificación para la elección de dicho tipo:

- Temperatura (estación): **DECIMAL 4,2**. Los datos se almacenan en grados Celsius, con que con la reserva de esos caracteres es suficiente
- Timestamp (registro): **TIMESTAMP**. Tipo de dato optimizado y ya preparado para almacenar estos valores temporales.
- Age (usuario): **TINYINT**. El mínimo valor numérico es suficiente para la edad.
- Deleted user (usuario, campo auxiliar): **CHAR(1)**. Campo fijo, siempre se usará un carácter para indicar si un usuario está eliminado o no.
- Gender **VARCHAR(6)**. Campo de texto para ‘male’, ‘female’ o ‘other’.

6.6.1.2.- Sentencias

Lo mismo con sentencias, a continuación se muestran ejemplos de diferentes tipos de sentencias presentes en la API (desde la perspectiva de la interfaz PDO para PHP):

- Inserción: Ejemplo de inserción de una estación con binding de parámetros.

```
$fields = 'INSERT INTO `station`(
    uuidStation,uuidUser,latitud,longitud,postalCode,temp,humidity,presion,location,state)';
$values = ' VALUES (?,?,?,?,?,?,?,?,?,?)';
$query = $fields . $values;

$statment = $this->conect->prepare($query);

$statment->bindParam(1, $uuidStation);
$statment->bindParam(2, $uuidUser);
...
```

Ilustración 76 Ejemplo de consulta a base de datos

- Consulta: Ejemplo de consulta de una estación a partir del binding de su ID.

```
$select = 'select uuidStation, uuidUser, latitud, longitud, postalCode,  temp, humidity, presion, location, state,
    timestamp';
$from = ' from `station` where `station`.`uuidStation` = ? and deletedStation = ' . '0' . '';
$query = $select . $from;

$stmt = $this->conect->prepare($query);
$stmt->bindParam(1, $uuidStation);
$stmt->execute();
```

Ilustración 77 Base de datos búsqueda por ID

- Actualización: Ejemplo de actualización de una estación.

```
$update = "UPDATE `station` SET ";
$values = "uuidUser = :uuidUser, latitud =:latitud, longitud=:longitud, postalCode=:postalCode, temp=:temp,
    humidity=:humidity, state=:state";
$where = " WHERE uuidStation = :uuidStation";

$query = $update . $values . $where;
$statment = $this->conect->prepare($query);
```

Ilustración 78 Base de datos actualización

6.6.2.- MongoDb

Como vimos en clase, MongoDB es una base de datos orientada a documentos. Lo que significa, que a diferencia de las bases de datos relacionales, en lugar de guardar los datos en tablas y registros, los guarda en documentos u objetos.

Estos documentos se almacenan utilizando los tipos BSON, que funciona como una representación binaria de JSON.

Otra de las diferencias sustanciales que lo diferencian de las bases de datos relacionales, es que no es necesario seguir un esquema de relaciones. Los documentos que comparten una colección, algo así como las tablas en mysql, pueden tener esquemas distintos, es decir, con

diferentes campos.

Este es un ejemplo de uno de los objetos con conforman nuestra aplicación

```
{
    "_id" : ObjectId,
    "station" : int32,
    "location" : string,
    "state" : int32,
    "latitude" : float,
    "longitude" : float,
    "postalCode" : int32}
```

MongoDb se engloba dentro del conjunto de las Bases de Datos no relacionales, o NoSQL. Estas bases de datos tienen algunas características particulares como la ausencia de Joins, o la no existencia de las transacciones, lo que compromete la integridad y atomicidad de los datos.

A pesar de ello, MongoDb es altamente recomendado, en aplicaciones Web ya que permite el almacenamiento de gran cantidad de datos semi estructurados, así como en entornos que requieren escalabilidad, con capacidades de replicación y sharding de manera rápida.

6.7.- Frameworks

A menudo no es necesario implementar desde cero absolutamente todo lo necesario para la construcción de una aplicación web. Afortunadamente, hay disponibles multitudes de frameworks que nos abstraen de una tarea en concreto, o nos ayudan a desarrollarla mediante el aprovisionamiento de funciones y clases que ya están enfocadas a la consecución de nuestro objetivo.

En este proyecto se han usado principalmente tres frameworks, uno para cada capa o componente, así que a continuación se entrará un poco en el detalle de cada uno.

6.7.1.- ReactJS

Para la capa frontal, como ya se ha comentado en varios apartados llegados a este punto, se ha hecho uso de un frameworks llamado ReactJS (o simplemente React). Para no repetirnos demasiado, y puesto que la descripción de sus características principales, así como la justificación de la elección de este frameworks está disponible en el apartado de diseño del Frontend, aquí simplemente se va a recordar una definición breve y básica del frameworks y se mostrarán casos prácticos que ejemplifiquen un poco sus propiedades/capacidades:

```
class StationComponent extends React.Component {

    constructor(props) {
        super(props);
    }

    renderRedirect() {
        if (parseInt(sessionStorage.session) < 1 || typeof sessionStorage.session === "undefined") {
            return (
                <Redirect to='/login'/>
            );
        }
    }
                                                                                    Lógica
    render() {
        sessionStorage.setItem("station", this.props.station.id);
        return (
            <div id='station-main' className="container">
                {this.renderRedirect()}
                <div className="row">
                    <StationInfoComponent
                        user={this.props.user}
                        info={this.props.station}
                        openDeleteStationModal={this.props.openDeleteStationModal}
                        openEditStationModal={this.props.openEditStationModal}
                    />
                    <StationDataComponent
                        user={this.props.user}
                        data={this.props.station}
                        openEditStationModal={this.props.openEditStationModal}
                    />
                </div>
            </div>
        );
    }                                                                               Aspecto
}
```

Ilustración 79 Framework React

ReactJS, pues, es un frameworks de JavaScript que nos ayuda a la construcción de interfaces dinámicas para nuestras aplicaciones mediante la modularización de éstas en componentes diferenciados, en cada uno de los cuales se define su aspecto y su comportamiento. De este modo, se obtiene una interfaz muy modularizada y eficiente en temas de rendimiento.

- Componentes: La principal característica de React es, pues, su claro enfoque a la separación/modularización de las interfaces en componentes. En la imagen superior se puede ver un ejemplo de un componente, con una apariencia muy parecida a las clases de los lenguajes orientados a objetos, y como en el método render se define el aspecto de la parte de la interfaz de la que se ocupa.

- props: Otro de los aspectos principales y necesarios a dominar para la programación de interfaces con React. Las llamadas props (abreviatura de properties) no son más que los parámetros con los que un componente puede renderizar a otro (composición de componentes). Suponen, pues, una de las principales vías tanto de configuración como de comunicación entre componentes. Con un ejemplo se entenderá mejor:

  En la imagen lateral tenemos el método render del componente que renderiza tanto el mapa como la lista de la pantalla principal. Como se puede ver en lo que vendrían siendo los atributos de los tags de los componentes, se está realizando un traspaso de información. Por un lado, el array de estaciones que cada uno debe renderizar (a su manera) y por otro, funciones para que ambos puedan utilizar a modo de callback a determinados eventos. Como se decía antes, las props constituyen la principal forma de comunicación entre componentes, y en este ejemplo podemos ver que esta comunicación puede ser bidireccional (objetos con datos y funciones de callback con la posibilidad de recibir datos como parámetros.

```
render() {
    return (
        <div id='main' className="container">
            {this.renderRedirect()}
            <div className="row">
                <MapComponent
                    stations={this.state.stations}
                    openStationPopUp={this.openStationPopUp}
                    closeStationPopUp={this.closeStationPopUp}
                    selectStation={this.props.selectStation}
                />
                <ListComponent
                    stations={this.state.stations}
                    filterStations={this.filterStations}
                    animateStation={this.animateStation}
                    unAnimateStation={this.unAnimateStation}
                    selectStation={this.props.selectStation}
                />
            </div>
        </div>
    );
}
```

Ilustración 80 Vista del framework

- Estado: Si las props eran la característica principal de los componentes de React, se podría decir que el estado o state es la segunda en ese ranking. Gracias a este mecanismo, un determinado componente puede definir un estado para guardar la información que sea necesaria (según el caso) en cada momento, teniendo la peculiaridad que la modificación de cualquier propiedad de este estado (se modela como un objeto) desencadena un nuevo render.

  El estado inicial se acostumbra a definir en el constructor del componente, y a modificarse dentro de otras funciones que típicamente son llamadas como callbacks a determinados eventos, ya sean del propio componente o de cualquiera de los que se compone (renderiza).

```
class Login extends React.Component {

    constructor(props) {
        super(props);
        this.state = {username: '', password: ''};

        this.handleUsernameChange = this.handleUsernameChange.bind(this);
        this.handlePasswordChange = this.handlePasswordChange.bind(this);
        this.handleSubmit = this.handleSubmit.bind(this);
    }

    handleUsernameChange(event) {
        this.setState({username: event.target.value});
    }

    handlePasswordChange(event) {
        this.setState({password: event.target.value});
    }
```

Ilustración 81 Lógica del frameworks. Ejemplo.

- Ciclo de vida: Y, para terminar, tenemos otro aspecto relacionado con los componentes de React que puede llegar a ser bastante útil, el ciclo de vida. El ciclo de vida no es más que un conjunto de etapas o momentos por los que un componente pasa (montaje, actualización, desmontaje…), pero lo realmente interesante del asunto es que cada uno de esos momentos tiene una función inherente asociada y que por lo tanto se puede aprovechar en cualquier componente para llevar a cabo las acciones necesarias. Veamos algún ejemplo:

  En este caso, la etapa del ciclo de vida que se aprovecha es la más típica, componentWillMount. Este momento hace referencia a justo antes del primer render de un componente, y es el típico momento de establecer el estado inicial en función de las props recibidas.

  Otras funciones del conjunto del ciclo de vida son componentWillUpdate (cada vez que un componente recibe props nuevas), componentWillUnmount (momento antes de que el componente desaparezca de la interfaz), componentDidMount (justo después del primer render), componentDidUpdate (justo después de una actualización), entre otros.

```
componentWillMount() {
    switch(this.props.mode) {
        case 'edit-user':
            this.setState({
                Name: this.props.user.username,
                Password: '',
                ConfirmPassword: '',
                Gender: this.props.user.gender,
                Age: this.props.user.age,
                Location: this.props.user.location
            });
            break;
        case 'edit-station':
            this.setState({
                Latitude: this.props.station.latitude,
                Longitude: this.props.station.longitude,
                Location: this.props.station.location,
                Zip: this.props.station.zip
            });
            break;
        default:
            break;
    }
}
```

Ilustración 82 React Componente WillMount

6.7.2.- Symfony

Para el desarrollo de la API, el componente Middleware de la aplicación de MeteoSalle, se ha usado otro de las principales herramientas de referencia, teniendo en cuenta que el lenguaje elegido para el desarrollo de esta capa ha sido PHP.

Así pues, después de ReactJS tenemos Symfony, un frameworks muy usado y, como veremos a continuación, muy extenso para el desarrollo de aplicaciones web. Éste es básicamente el frameworks que se trabajó en el transcurso de la asignatura que lleva este nombre, por lo cual se ha podido aprovechar mucho conocimiento de la asignatura impartida en el máster.

Como decíamos, Symfony es un frameworks muy grande, y nos referimos a grande en cuanto al tamaño por la gran cantidad de módulos, componentes y librerías que integra, cada uno destinado a satisfacer alguna necesidad, más o menos básica, más o menos fundamental, de la implementación de una web.

Al tratarse del desarrollo de una API, los módulos principales y necesarios para el desarrollo de este Middleware han ido enfocados en esta dirección. A continuación, se enumeran y describen

brevemente dichos módulos o librerías usadas:

- http-foundation: Éste es el componente principal y el más importante y usado para la capa intermedia de MeteoSalle, pues es el que permite gestionar las comunicaciones de la API desde el lado del servidor. La principal característica de este módulo es la capa de abstracción orientada a objetos que ofrece para tratar las peticiones HTTP (acceso a parámetros, cabeceras, etc.) así como las respuestas (formato, códigos de respuesta, etc.).

  En la siguiente captura se pueden observar sus librerías en funcionamiento:

```
public function index(Request $request): JsonResponse
{
    try {

        $station = Station::buildStation($request);

        $repository = new StationRepositoryMysql($_SERVER['HOST_M
        $tails = new TailsRepositoryRabbit($_SERVER['HOST_RABBIT'
        try {
            $createStation = new CreateStation($repository, $tail
            $createStation($station);

        } catch (Exception $exception) {
            throw new StationErrorException($exception->getMessag
        }

        return new JsonResponse(
            ['Message' => 'Station ' . $station . ' created'],
            201,
```

Ilustración 83 Symfony request

- http-client: Y si el anterior componente permitía gestionar peticiones desde el lado del servidor, este módulo ofrece lo mismo, pero desde el lado del cliente. Es decir, una capa de abstracción similar, pero para generar peticiones y consumir de una API. Este módulo ha sido necesario para comunicarse con el tercer y último componente Backend y consumir de sus endpoints, los cuales prestan servicio para recibir datos de estaciones y predicciones meteorológicas:

```
private const ADDRESS_API = 'http://34.66.90.82:9000/v1/';
private $httpClient;

public function __construct()
{

    $this->httpClient = HttpClient::create(
        [
            'http_version' => '2.0',
            'headers' =>
                [
                    'User-Agent' => 'MeteoSalleMiddel',
                    'Content-Type' => 'application/json'
                ]
        ], 10);

}
```

Ilustración 84 Symfony Http-Client

- maker-bundle: Vistos los dos módulos principales para la comunicación bidireccional hacia y desde la API del Middleware, tenemos otro módulo interesante para facilitar la tarea de creación e implementación de elementos específicos como controladores, comandos, clases, etc. El nombre, pues, precisamente viene por la característica principal de este componente, que es la creación (vía línea de comandos) del elemento que necesitemos, así como la configuración de sus parámetros y especificaciones.

```
~/Sites/os/maker-bundle/tests/tmp/cache/maker_52afa2ca25$ ./bin/console make:user

The name of the security user class (e.g. User) [User]:
>

Do you want to store user data in the database (via Doctrine)? (yes/no) [yes]:
>

Enter a property name that will be the unique "display" name for the user (e.g. email, u
>

Will this app need to hash/check user passwords? Choose No if passwords are not needed o
system (e.g. a single sign-on server).

Do this app need to hash/check user passwords? (yes/no) [yes]:
>
```

Ilustración 85 Symfony bundle maker

- dotenv: Para terminar, tenemos otra útil librería que nos permite gestionar diferentes variables que, en función del entorno (desarrollo, producción, test) tomen diferentes valores. Las llamadas variables de entorno, que con este módulo se almacenan en diferentes ficheros, para así poder tener el mismo nombre e ir pivotando en la fuente de datos desde donde se toman según el entorno sobre el que se trabaje en cada momento.

6.7.3.- Revel

Para la última capa de la aplicación, se ha hecho uso de uno de los frameworks más populares en la comunidad de Golang, Revel.

Como ya hemos comentado GO es un lenguaje moderno y flexible, para crear aplicaciones compiladas de manera fácil.

Aunque el uso de un frameworks no es imprescindible en un lenguaje de programación como Go, con multitud de paquetes, e incluso, podríamos empezar a programar una aplicación web desde cero, del mismo modo que sucede con otros lenguajes. La utilización de un frameworks nos facilita enormemente esta tarea.

Tras haber empezado el desarrollo de esta capa sin frameworks y tras algún que otro tropiezo, se optó por realizar una búsqueda exhaustiva de frameworks en el ecosistema GO, para de este modo, determinar qué frameworks era el más completo, fácil de usar y con mejor documentación. Después de algunas pruebas, se determinó que ese frameworks era Revel.

Para utilizar Revel, basta con instalar las siguientes dependencias:

go get github.com/revel/revel, que instalará el frameworks
get github.com/revel/cmd/revel, que ejecutará un intérprete de comandos.

Además, se ha hecho uso de algunos módulos que describiremos a continuación.

Los módulos en Revel son paquetes que se pueden conectar directamente a una aplicación y permiten, compartir controladores, vistas y otro tipo de código, entre múltiples aplicaciones propiamente de Revel, o de terceros.

Un módulo, por lo tanto, debe tener el mismo diseño que el diseño de una aplicación Revel. Todas las plantillas module/app/views, se agregarán a la ruta de búsqueda del cargador de plantillas. Cualquier controlador en module/app/controllers, será tratado como si estuviera en su aplicación. Las rutas se pueden incluir en su aplicación con la línea de ruta de module:modulename- ver ruta

Revel viene con algunos módulos integrados, y en nuestro caso hemos hecho uso del módulo de testing, que nos permite realizar pruebas unitarias de manera integrada y sencilla, y módulo de Jobs, que nos permite ejecutar código de manera asíncrona y agendada. Ambos, los describiremos en profundidad en el apartado técnico.

6.8.- SEO, Analítica web y Emprendeduría

Esta es otra de las asignaturas que está más estrechamente relacionada con la capa Frontend de la aplicación, en especial la parte más puramente técnica de SEO. En este apartado se van a

desarrollar diferentes aspectos relacionados con esta disciplina, y en última instancia se comentará cuál es el modelo de negocio que hay detrás de MeteoSalle.

6.8.1.- Arquitectura

A nivel de arquitectura hay diferentes aspectos a comentar sobre la toma de decisiones referente a el enfoque y la estructuración que se ha querido plasmar en nuestra aplicación web.

6.8.1.1.- URLs, Dominios y ccTLDs

La URL básica de la web de MeteoSalle es la siguiente: https://www.meteosalle.com. Debido a que, en el momento actual, MeteoSalle centre su alcance o ámbito de información a nivel España, no hemos visto necesario incluir versiones con ccTLDs para diferentes países.

En cuanto a la estructuración de páginas en la propia URL:

- Login: https://www.meteosalle.com/login
- Registro: https://www.meteosalle.com/register
- Página principal: https://www.meteosalle.com/home
- Página de usuario: https://www.meteosalle.com/user
- Página de estación: https://www.meteosalle.com/station

6.8.1.2.- Tipologías

Las tipologías de búsquedas identificadas, y a las cuales se ha correspondido con una página acorde son las siguientes:

- Búsqueda de **estaciones por población** o código postal -> Página principal, resultados tanto a nivel de mapa como de lista.
- Búsqueda de **estaciones del usuario** con sesión iniciada -> Página principal (en primera instancia, al iniciar sesión) y en la página de usuario en todo momento.
- Búsqueda de **información del usuario** con sesión iniciada -> Página de usuario.
- Búsqueda de **información de una estación** en concreto -> Página de estación, accesible tanto desde la página principal como la de usuario.

6.8.1.3.- Sitemap

Finalmente, para poder conceptualizar mejor la estructura de la página web, aquí se incluye su sitemap:

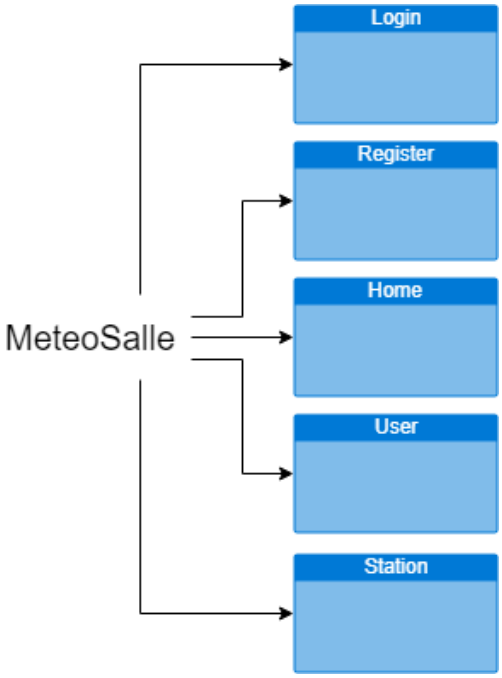


Ilustración 86 Site map MeteoSalle

6.8.2.- Keywords

El proyecto MeteoSalle se encuentra claramente en fase de arranque, por lo que la metodología principal para encontrar posibles palabras clave se basa en el análisis de la competencia. Se ha usado tanto la herramienta SimilarWeb como los propios sitios web de la competencia para ello, de la misma forma que se ha tirado un poco de sentido común.

A continuación, se adjuntan las keywords que se han identificado como pertenecientes al proyecto, clasificadas en las tres categorías principales:

- Head tail: weather, temperature, humidity, pressure, prediction.
- Mid tail: maximum/minimum temperature/humidity/pressure.
- Long tail: next days temperature/humidity/pressure prediction.

6.8.3.- Optimizaciones OnPage

Para conseguir un buen SEO es importante también seguir diferentes recomendaciones/medidas que ayudan a un buen posicionamiento en los buscadores (Google, en este caso).

Éste es un listado de algunas iniciativas o medidas tomadas en esa dirección:

6.8.3.1.- T's & M's (Titles & Metadescriptions)

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Share and meet the weather values of your own station, see information from other
  places and even know the forecasts and historics that interest you most"/>
  <link type="text/css" rel="stylesheet" href="styles.css">
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css"
        integrity="sha384-Gn5384xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/dAiS6JXm" crossorigin="anonymous">
  <link rel="stylesheet" href="https://use.fontawesome.com/releases/v5.5.0/css/all.css"
      integrity="sha384-B4dIYHKNBt8Bc12p+WXckhzcICo0wtJAoU8YZTY5qE0Id1GSseTk6S+L3B1XeVIU" crossorigin="anonymous">
  <title>MeteoSalle, a community to share and consult weather data</title>
```

Ilustración 87 Optimización OnPage

- Se recomienda utilizar un título de entre 50 y 60 caracteres.
- Se recomienda que el título esté presente en la propia página.
- Se recomienda que el metadescription tenga entre 150 y 160 caracteres.
- Se recomienda que el metadescription contenga la keyword principal/objetivo.

6.8.3.2.- Contenidos

```
<div id='user-info' className="container col-12 col-sm-12 col-md-5 col-lg-5 col-xl-4">
    <div id="user-profile-image">
        <img id='profile-image' alt='Avatar' src={avatar}/>
    </div>
    <div id="user-info-data">
        <p className="user-info-line">
            <span className="user-info-line-title">{this.props.t('Name')}: </span>{this.props.user.username}
        </p>
```

```
<form id='login-form' onSubmit={this.handleSubmit}>
    <div id='login-header'>
        <h3 id='login-title'>{this.props.t('Login')}</h3>
        <img id='login-logo' alt='Logo' src={logo}/>
    </div>
```

- Se cree que el contenido de la página es **relevante** y aporta **valor añadido**, pues los datos de las estaciones meteorológicas son o bien pertenecientes a un usuario que se entiende que es un entusiasta del tema, o bien sus datos han sido sacados realizando una media entre varias fuentes (contrastadas).
- Es un contenido **enriquecido** por el hecho de usar marcado semántico, así como por el uso del atributo alt en las imágenes:

6.8.3.3.- Enlazado

- Contiene **enlaces** a sitios web de calidad, teniendo los enlaces un anchor text correspondiente al contenido enlazado.

```
</a>
<a
    id="vendor-link"
    className="footer-link"
    href="https://developer.here.com/do
    hreflang="en">
    Here
</a>
<a
    id="vendor-link"
    className="footer-link"
    href="https://www.salleurl.edu/en"
    hreflang="en">
    La Salle
```

Ilustración 88 Enlaces externos

6.8.3.4.- Geolocalización

- Uso de atributos como lang y hreflang para indicar los idiomas de la página y de los enlaces, respectivamente.

6.8.4.- Conversión y modelo de negocio

Y finalmente, el aspecto más relevante o a dónde se quiere llegar con todo lo visto hasta ahora, la **conversión**.

Éstas son las vías de conversión identificadas para el proyecto de MeteoSalle, acompañadas de las optimizaciones y medidas que se han tomado para potenciarlas, así como sus criterios de medición (ROI, Return of Investment).

Vías de conversión	Optimizaciones	Criterios de medición
Registros	Presencia en RRSS, publicidad	Número de registros
Visitas	Información actualizada	Número de visitas
Duración de las visitas	Información completa y relevante	Tiempo medio de las visitas

Tabla 89 Conversión y modelo de negocio

Y para terminar el apartado de esta asignatura, se incluye el diagrama del modelo de negocio de MeteoSalle:

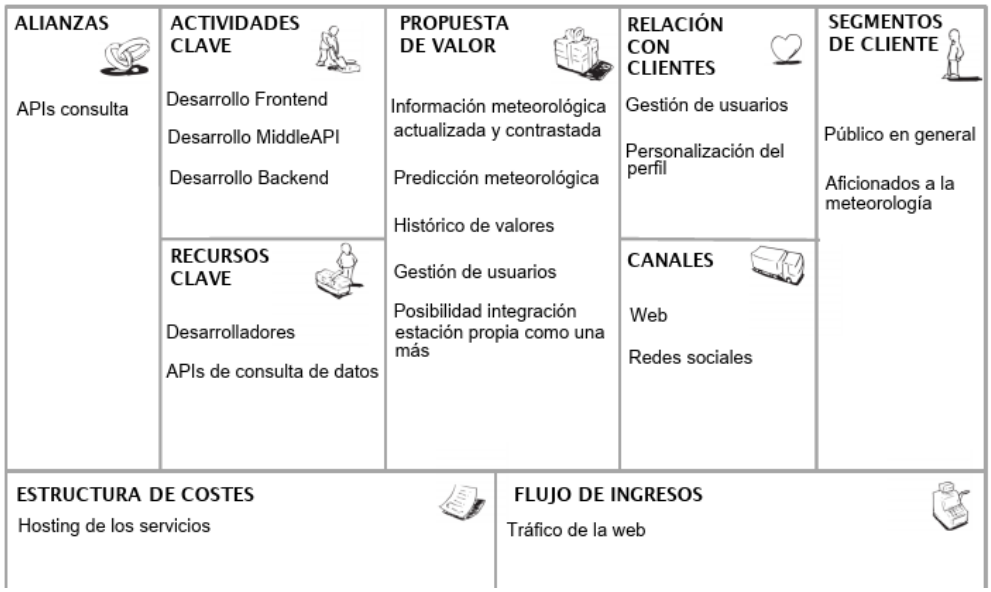


Ilustración 90 Diagrama del modelo de negocio.

6.9.- Rendimiento web

En esta asignatura, también transversal en las diferentes capas de la aplicación, se hablará de las herramientas, tecnologías y métodos utilizados para mejorar la respuesta de la web en términos de rendimiento.

6.9.1.- Cache

El tratamiento de los datos en caché se ha llevado a cabo con Redis. Redis es un motor de base de datos basado en almacenamiento con tablas de hashes(clave/valor) que principalmente utiliza la memoria RAM para el almacenamiento, pero de manera opcional, permite realizar operaciones de persistencia y alta disponibilidad, utilizando discos.

Está escrito en ANSI C por Salvatore Sanfilippo y liberado bajo licencia BSD. Su primera versión vio la luz en el año 2009 y la última versión estable (4.0.11) apareció en agosto de 2018. Su aplicación principal radica en sistemas de caché y deriva de la necesidad de mejorar los tiempos de respuesta.

Las principales características y funcionalidades de Redis son:

- Multiplataforma: Cuenta con soporte para multitud de lenguajes, de entre los que podemos destacar: ActionScript, C, C++, C#, Go, Haskell, haXe, Io, Java, JavaScript (Node.js), Lua, ObjectiveC, Perl, PHP, Pure Data, ruby y Python, entre otros.
- Rendimiento: Cómo comentábamos al inicio de este documento Redis utiliza la memoria principal para almacenar información. Esta característica le confiere un alto rendimiento y le permite acceder a los datos de manera eficiente.
- Tipos de datos: Redis permite modelar la solución más adecuada en cada caso, ya que es capaz de manejar hasta cinco estructuras de datos distintas: Strings, Listas, Sets, Sorted Sets, Hashes.
- Persistencia: Como complemento y cuando los datos a tratar superan el máximo de memoria disponible o por configuración y para poder disponer de ellos en caso de fallo del servidor, existen dos tipos de configuración sobre discos rígidos: RDB (intervalos específicos de conjuntos de datos) y Append-only-file (cada operación es escrita en un log recibida por el servidor).
- Caducidad de las claves: Después de algún tiempo y cuando los datos no sen requeridos, pueden ser eliminados de manera manual o de forma automatizada mediante la asignación de tiempos de vida.
- Replicación: Los sistemas redis cuentan con copias exactas de los datos, replicando una arquitectura Maestro-Esclavo, donde un servidor maestro, puede tener varios esclavos y un esclavo, a su vez, puede ser maestro para otros nodos con función de esclavo.

El uso de Redis es altamente aconsejable cuando los tiempos de respuesta y la velocidad de acceso a los datos, son críticos para una solución de negocio o cuando se trabaja en soluciones con respuestas en tiempo real, lo que demanda datos con acceso inmediato por eso se ha determinado que el uso de Redis en este caso está altamente justificado.

Como se ha detallado en el apartado de diseño del componente Middleware (la capa del proyecto donde se utiliza caché), se ha utilizado esta herramienta para cachear todas las operaciones

relacionadas con peticiones lanzadas desde el Frontend. Éstas pueden estar relacionadas con una estación (petición, creación, actualización y eliminación), muchas estaciones (consulta solamente) y usuarios (todas las operaciones también).

En la siguiente tabla se puede observar el formato de almacenaje en Redis y los casos en que se tratan dichos datos, de forma más sintetizada:

Clave	Valor	Escenarios
ID estación	Objeto estación	GET, POST, PUT, DELETE
ID usuario	Objeto usuario	GET, POST, PUT, DELETE
ID estaciones Middleware	Array objetos estaciones	GET
ID estaciones Backend	Array objetos estaciones	GET

Ilustración 91 Estructura de datos a ser guardados en Redis

6.9.2.- Paralelismo y Concurrencia

A pesar de que en la actualidad la mayoría de las aplicaciones trabajan con múltiples hilos de ejecución y que los lenguajes de programación actuales, tratan de manera sencilla la implementación de la ejecución múltiple. Hay dos conceptos fundamentales para llevar a cabo esta tarea, como la concurrencia y el paralelismo, que vamos a describir a continuación:

6.9.2.1.- Concurrencia

La concurrencia sucede cuando los procesos son ejecutados en una CPU única. En este caso, podemos dividir el tiempo de proceso de esta CPU de manera que las diferentes tareas, dan la sensación de que se ejecutan de manera concurrente, pero en realidad, se está ejecutando un único proceso de manera consecutiva.

Para llevar a cabo esta tarea, la CPU está priorizando los tiempos de ejecución de cada tarea e intercalando una mínima fracción de tiempo a cada una de estas. La coordinación de estos tiempos de ejecución es clave para aumentar el rendimiento y es en esta tarea que GO está realmente optimizado.

6.9.2.2.- Paralelismo

Se basa en la capacidad actual de los ordenadores que cuentan con microprocesadores de varios núcleos de ejecución y la capacidad de estos, de realizar varias tareas de manera simultánea para dividir, problemas complejos en problemas de menor tamaño. Por lo tanto y por esclarecer las diferencias entre ambos términos, la concurrencia se trata de perfeccionar la sincronización en la ejecución de procesos de manera eficiente.

Mientras que el paralelismo, trata de subdividir los problemas complejos aprovechando la capacidad técnica de los procesadores actuales, que permiten la ejecución de tareas de manera simultánea, en diferentes hilos de ejecución (multi-threading) y que además, estos a su vez, podrán organizarse de manera concurrente, en el proyecto se ha utilizado RabbitMQ para crear un proceso paralelo que actualice la base de datos de Redis cuando se realiza alguna petición de una estación, de esta forma se logró disminuir las respuesta al frontend de 3,8 segundos a 1500 milisegundos.

6.9.3.- Profiling

Para finalizar se hablará sobre el análisis de profiling realizado sobre la aplicación web de MeteoSalle. Se entiende por un análisis de profiling una especie de examen al que se somete una página web en el cual se evalúan diferentes aspectos relacionados con el rendimiento.

Ilustración 92 Profiling

Lighthouse ha sido la metodología elegida para realizar el análisis de profiling de nuestro proyecto. Se trata de una herramienta open source que además de proveernos de un informe después de cada examen con valoraciones sobre 100 en cuatro categorías distintas (rendimiento, accesibilidad, buenas prácticas y SEO) también ofrece consejos y tácticas para mejorar dichas evaluaciones. Veamos un ejemplo, con relación a los resultados obtenidos en un primer análisis sobre una versión build antigua:

- Pobre contraste de colores entre los enlaces del header y el footer y su color de fondo. (Accesibilidad)
- Elementos input de formularios sin labels asociados. (Accesibilidad)
- No uso de HTTPS. (Buenas prácticas)
- Imágenes sin el atributo alt. (SEO)
- Ausencia de meta-description en la página. (SEO)

Así pues, simplemente aplicando los fixes relacionados a los comentarios obtenidos en el informe, obtenemos rápidamente unos resultados bastante decentes:

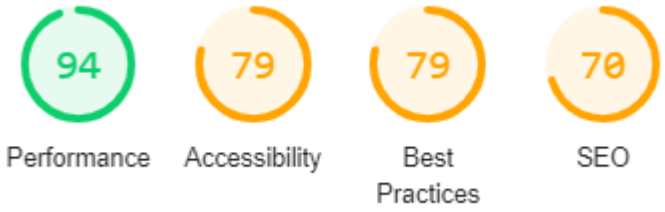


Ilustración 93 Análisis estadístico de rendimiento frontal

6.10.- Seguridad web

Para terminar, se hablará de los diferentes métodos y buenas prácticas extraídas de cursar la asignatura de seguridad y aplicadas en los diferentes componentes del proyecto, aportando ejemplos y justificando cada uno de ellos.

6.10.1.- Inyecciones

Primero de todo se verán las precauciones tomadas para evitar que se produzca algún tipo de inyección de código (ya sea HTML, JavaScript, PHP, SQL…) a nuestro sistema a través de las diferentes entradas de texto que se ofrecen en la capa frontal. Básicamente se han empleado dos métodos:

6.10.1.1.- Expresiones regulares

Las expresiones regulares son representaciones de conjuntos determinados de caracteres que conforman patrones de validación para permitir o no la introducción de determinados valores a través de inputs de formularios. Éstos son los que se han usado en este proyecto:

- Alfanumérico. Para el nombre de usuario: **[a-zA-Z0-9]+**
- Contraseña. 1 carácter en minúscula, 1 en mayúscula y 1 número como mínimo, con una longitud entre 8 y 12 caracteres: **^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9]).{8,12}$**
- Numérico entero. Para valores como la edad del usuario o un código postal: **[0-9]+**
- Numérico positivo. Para valores como la humedad y la presión: **[0-9]+([\.,][0-9]+)?**
- Numérico general: Para valores como las coordenadas (longitud y latitud) y la temperatura: **(-?)+[0-9]+([\.,][0-9]+)?**

6.10.1.2.- Filtrado de caracteres

Para otros campos menos específicos se ha optado por una manipulación más directa de lo introducido por el usuario antes de mandar información a la API del componente Middleware, que básicamente ha consistido en escapar los siguientes caracteres (reemplazarlos por nada, es decir, suprimirlos):

- [; & |]: Para evitar posibles inyecciones de código JavaScript y PHP.
- [< >]: Para evitar posibles inyecciones de código HTML.
- [‘ #]: Para evitar posibles inyecciones de código SQL.

Todos estos tratamientos se han aplicado de forma secuencial a los campos relativos a datos que no tenían una expresión regular tan definida, como es por ejemplo el nombre de una población o localidad.

6.10.2.- Autenticación y sesiones

En segundo lugar se hablará del funcionamiento del método de autenticación de MeteoSalle, en el cual están involucrados los componentes Frontend i Middleware. Y para ello, se utilizará un sistema basado en tokens. Concretamente, se utilizará la librería PHP-JWT, la cual, como indican

las siglas del nombre, utiliza tokens basados en JSON (JSON Web Tokens).

El elemento token no deja de ser un objeto que de alguna forma sirve para validar o confirmar la validez de una petición o respuesta, siendo representativo de una sesión autorizada en un sistema o aplicación.

Para ello, en el momento del login (en el Middleware), cuando se comprueba que las credenciales son correctas, se genera un token para la sesión del usuario (se acostumbra a generar a partir de datos del usuario y codificándolo en base a una key), el cual es enviado juntamente con el mensaje correspondiente de respuesta del login, típicamente un 200 OK.

```
$key = "example_key";
$token = array(
    'username' => $username,
    'password' => $password,
);

$jwt = JWT::encode($token, $key);

$response = new Response(
    json_encode([
        'login' => $this->logIn->logIn($username, $password),
        'token' => $jwt
    ], 200)
);
```

Ilustración 94 Uso de JWT

```
▾ Request Headers
  ⚠ Provisional headers are shown
  Authorization: eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.
  Origin: http://localhost:3000
  Referer: http://localhost:3000/login
```

Ilustración 95 Petición con la cabecera de autenticación

Por su lado, cuando el Frontend recibe el token de sesión, éste se almacena en el navegador (típicamente en la propia sesión o en las cookies), para así, en cada futura petición a la API, adjuntarlo en la cabecera de la request y que el Middleware sepa reconocer la petición como válida y proceda a tratarla:

De este modo, en los endpoints de datos queda por ver la validación del token (en base a la key con la que se ha codificado) para o bien ofrecer la respuesta que se pide o bien mandar un Unauthorized, o incluso otro mensaje conforme el token no figura en la petición, en caso de que se diese esto último:

```
if (isset($headers['Authorization'])) {

    $jwt = $headers['Authorization'];
    try {
        $key = "example_key";
        $decoded = JWT::decode($jwt, $key, array('HS256'));
        $response = new Response(json_encode($this->stationsBrowser-
        return $response;
    } catch (SignatureInvalidException $e){
        $response = new Response(json_encode('Unauthorized', 401));
        return $response;
    }
}
else {
    $response = new Response(json_encode(['msg' => 'Missing token',
    return $response;
}
```

Ilustración 96 Ejemplo de implementación JWT

6.10.3.- Datos sensibles

Para terminar, se verá cómo se ha llevado a cabo la protección de los datos sensibles de los usuarios, es decir, la encriptación de las contraseñas. Para ello, se ha utilizado el algoritmo BCRYPT, uno de los más utilizados y recomendados hoy en día para tal finalidad.

Afortunadamente, PHP dispone de una función incorporada que ahorra tener que buscar librerías o componentes externos para llevar a cabo la encriptación o hashing de contraseñas, de modo que basta con aplicar este proceso antes de guardar la contraseña en base de datos:

```php
final class UserCreator
{
    private $usersRepository;

    public function __construct(UsersRepository $usersRepository)
    {
        $this->usersRepository = $usersRepository;
    }

    public function __invoke(string $userId, string $username, string $password): void
    {
        $user = new User($userId);
        $user->setUsername($username);
        $user->setPassword(password_hash($password, PASSWORD_BCRYPT, array('cost' => 12)));
        $this->usersRepository->save($user);
    }
}
```

Ilustración 97 Ejemplo de CreatUser

Y para el proceso inverso, el de validar una contraseña para el login, basta con usar la función complementaria (incorporada también en PHP) que valida la contraseña introducida respecto a la encriptada y almacenada en base de datos:

```php
if (password_verify($passwordInput, $password)) {

    $user = $user['id'];

} else {

    $user = -1;

}
```

Ilustración 98 Verificación de la contraseña

7.- COSTE TEMPORAL

Entrando ya en los apartados de clausura de esta memoria, en este apartado se hablará del tiempo que ha tomado la construcción de la aplicación. Este punto tiene un poco de relación con el de planificación y desarrollo, en el sentido que trata sobre el proyecto desde un punto de vista más de gestión, aunque como se verá a continuación, ahora se hace con una perspectiva posterior a la implementación.

Para contextualizar cómo ha sido el desarrollo de MeteoSalle, se han identificado 4 etapas o tipos de tareas por las que se ha pasado hasta obtener el resultado actual, y son las siguientes:

- **Análisis**: Etapa de identificación de componentes, repartición de tareas, investigación y estudio de tecnologías y alcance, etc.

- **Diseño**: Tareas relacionadas con la validación de tecnologías, elaboración de pruebas de concepto, identificación de los casos de uso, etc.

- **Implementación**: La construcción de la aplicación en sí.

- **Documentación**: Básicamente el redactado de este documento (memoria), así como informes, emails, etc.

En el siguiente gráfico podemos ver cómo se ha desplegado el desarrollo de las diferentes tareas comentadas a lo largo del periodo que empieza con la asignación del proyecto y termina con la entrega y presentación de este:

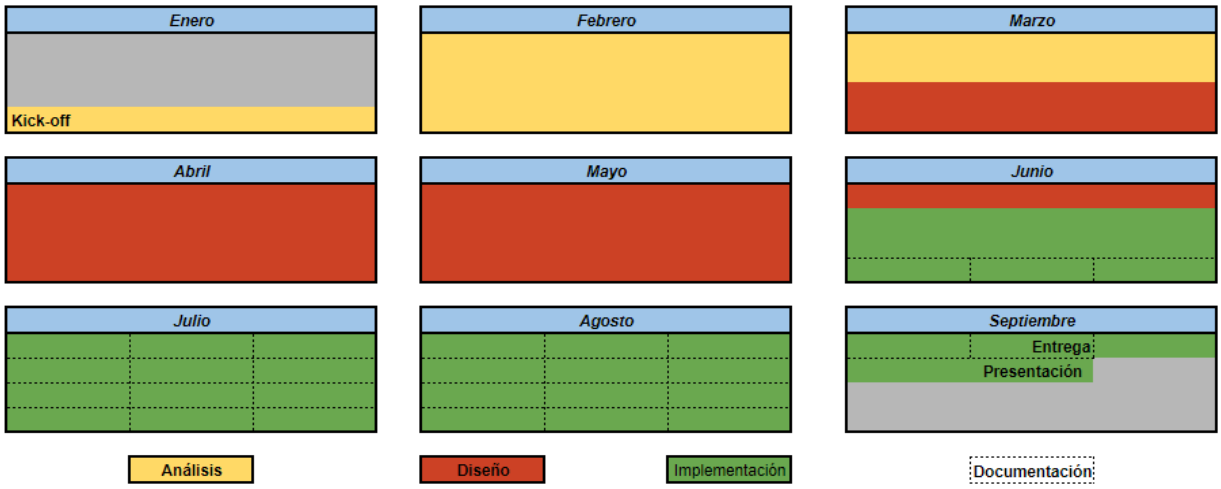
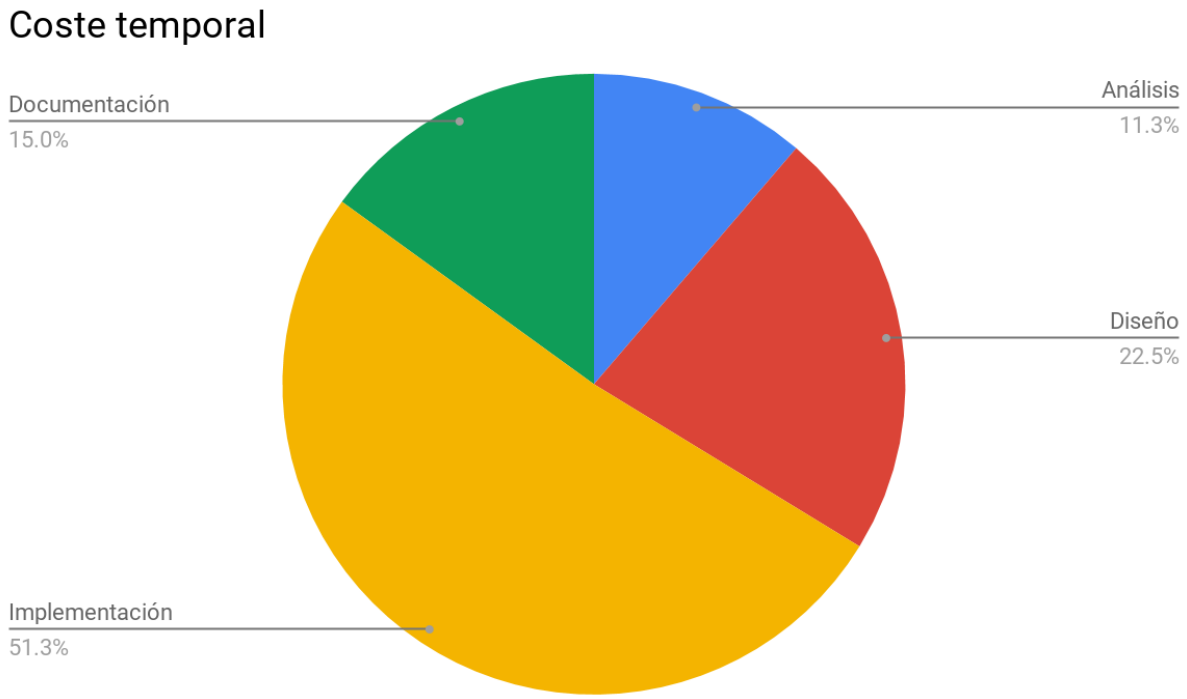


Ilustración 99 Tiempo de desarrollo

No obstante, para figurar mejor el volumen de trabajo que ha supuesto cada etapa o grupo de tareas que ha conformado el proyecto, se añaden también los siguientes gráficos, en formato de tabla y gráfico de sectores respectivamente, trasladan mejor el número de horas dedicadas a cada componente:



Etapa o grupo de tareas	Número de horas dedicadas
Análisis	90
Diseño	180
Implementación	410
Documentación	120

Ilustración 100 Etapas y número de horas

8.- CONCLUSIONES

En este apartado se van a enumerar y a reflexionar sobre las valoraciones que se sacan después de haber llevado a cabo la implementación de MeteoSalle. En primer lugar, se incluirá una relación de las conclusiones más globales y sacadas más desde un punto de vista colectivo, para luego ya, según la versión de la memoria que se trate, incluir las conclusiones más personales e individuales de cada miembro del equipo.

8.1.- Conclusiones comunes

Sobre las conclusiones comunes se va a seguir un formato de lista con una breve reflexión para cada una:

- Aportación de las materias del máster: La opinión general del equipo es que toda asignatura ha aportado su granito de arena para el desarrollo de este proyecto. Además, ha sido en el momento del redactado de esta memoria (y más concretamente del apartado relacionado con las materias) cuando ha sido más notorio este hecho, pues en un principio pudiera parecer que solamente englobaba Maquetación, JavaScript, PHP, Frameworks y Bases de Datos. No obstante, dando un tiempo para reflexionar y pensar el equipo se ha dado cuenta de que todo ha sumado, y se cree que esto ha sido altamente positivo.
- Conocimientos adquiridos: Como siempre, después de la implementación de un proyecto relativamente grande, es de valorar el aprendizaje adquirido. En el apartado de conclusiones individuales se especificará más qué conocimientos valora más o dónde ha profundizado más cada uno, pero creemos que es bastante importante resaltar este hecho, obviamente resultando también en una conclusión positiva.
- Valoración del encaje de piezas: En un principio pudiera parecer que casi la totalidad del trabajo que suponía este proyecto radicaba en la implementación por separado de los tres componentes o capas que lo integran. Pero el momento de juntar todas las piezas ha resultado ser también un buen pico de trabajo, por lo que, en general, se piensa que éste es otro aprendizaje que resulta de este proyecto, la valoración de la unión de varios componentes y la importancia de reservar y dedicar el tiempo que merece a este proceso.
- Importancia de la comunicación: Bastante en relación con la reflexión anterior, otra de las enseñanzas extraídas de este trabajo es la importancia de la comunicación, sobre todo, más que de la comunicación en sí, de la constancia de ésta. El equipo se ha dado cuenta que al trabajar cada uno por su cuenta uno termina, de alguna manera, aislándose y dando por sentado que el pensar general del equipo equivale al individual y personal, y esto no siempre es así. Se piensa que es muy importante contrastar y compartir continuamente los avances para evitar sorpresas en etapas avanzadas de la implementación, y asegurarse en todo momento que todo el equipo trabaja en la misma línea para evitar tener que realizar modificaciones innecesarias.

8.2.- Conclusión individual

Con la elaboración de este proyecto diseño e implementación de la **API Rest Middleware**, para el trabajo final de máster, podemos concluir que tanto el diseño, como la implementación, no solo de la API sino de cualquier proyecto en general, son muy importantes, la arquitectura definida en el diseño; asienta las pautas a seguir luego en nuestro código, elegir y construir con una

adecuada arquitectura de software nos conllevara a una aplicación mantenible en el tiempo y legible por otros desarrolladores, nos permite centrarnos en lo que realmente interesa, que es nuestro dominio de aplicación, nos brinda un plan "visual" del desarrollo del software y donde limar las complejidades que se pueden generar, la aplicación de la arquitectura hexagonal como pilar fundamental en la concepción del software permitió precisamente eliminar esas complejidades que se generan ineludiblemente cuando **no** se aplica una arquitectura, como las llama "Robert C. Martin" (Uncle Bob) "Clean Arquitecture"[4] en la cual está inspirada la ilustración **API Middleware**.

Tanto el diseño como la implementación responden a los requisitos y necesidades de la aplicación en general del sitio social Meteosalle, brindando una capa de abstracción a la aplicación cliente y coordinando la integración con el backend la hace que en cierto modo sea el "orquestador" de las funcionalidades, sin duda alguna ha sido un reto manejar y unir los datos de ambas bases de datos y brindar una respuesta coordinada, la solución han sido las excepciones, que brindan la posibilidad de declarar funciones puras; ante una misma llamada, siempre habrá una única respuesta.

[4] Block de Robert C Martin https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

9.- LÍNEAS DE FUTURO

A continuación, se muestra una relación de posibles mejoras y añadidos que, o por motivos de tiempo o de alcance del proyecto, no se han implementado en esta versión pero que podrían resultar interesantes de incorporar en posibles versiones futuras:

- Ampliación radio de alcance: Pensamos que podría ser interesante realizar alguna ampliación de territorio para que la aplicación pudiese funcionar a una escala superior, por ejemplo, incluir nuevos países, por no hablar de nivel Europa o mundial. Evidentemente, para dar este paso sería necesario revisar el alcance de las fuentes de datos del Backend, y actualizarlas si fuera necesario.

- Funcionalidades sociales: Por ahora a nivel social la aplicación es bastante simple, solamente se “comparten” las estaciones de los usuarios, pero podría estar bien añadir funcionalidades más típicas de una red social como poder conectar con otros usuarios, ver sus perfiles, etc.

- Incorporar nuevas fuentes de datos: Por ahora son tres fuentes sobre las que se trabajan y se calculan las medias. Ampliar este espectro podría aumentar la calidad de los datos y que los muestreos fuesen más ricos. Además, dar este paso no tendría que suponer mucho trabajo más allá de revisar los formatos de los datos a la hora de llevar a cabo la integración

- Introducción automática de datos: Por ahora los usuarios deben introducir registros en sus estaciones de forma manual, pero podría ser interesante incorporar, de alguna forma, la posibilidad de que las estaciones (las que puedan) se comunicaran directamente con la aplicación y de que los datos se registraran automáticamente.

- Integración con RRSS: Para reforzar aún más el aspecto social, podría resultar interesante conectar el perfil de diferentes redes sociales con el usuario de la aplicación, para poder compartir de manera fácil las estaciones también en otras áreas, o notificar el añadido de un nuevo registro meteorológico, por ejemplo.

10.- BIBLIOGRAFIA

[1] React, a JavaScript library for building interfaces:
https://reactjs.org/

[2] Create React App - Set up a modern web app by running one command:
https://facebook.github.io/create-react-app/

[3] Redux - A Predictable State Container for JavaScript apps:
https://redux.js.org/

[4] Integrate redux into your existing react app in 5 steps:
https://medium.com/@vishmimoney/integrate-redux-into-your-existing-react-app-in-5-steps-d4cd59090e9f

[5] React Redux, Official React bindings for Redux:
https://react-redux.js.org/

[6] React Router: Declarative Routing for React.js: https://reacttraining.com/react-router/web/guides/quick-start

[7] react-google-maps, React.js Google Maps integration component:
https://github.com/tomchentw/react-google-maps

[8] React Google Maps Style Guide:
https://tomchentw.github.io/react-google-maps/

[9] Overview - Maps JavaScript API - Google Developers:
https://developers.google.com/maps/documentation/javascript/tutorial

[10] react-google-charts, A tiny, typed, React wrapper for Google Charts:
https://www.npmjs.com/package/react-google-charts

[11] selenium-webdriver, Selenium is a browser automation library:
https://seleniumhq.github.io/selenium/docs/api/javascript/index.html

[12] Mocha - the fun, simple, flexible JavaScript test framework:
https://mochajs.org/

[13] Chai, BDD & TDD assertion library:
https://www.chaijs.com/

[14] Selenium Tests with Mocha and Chai in JavaScript, august 2018:
https://nehalist.io/selenium-tests-with-mocha-and-chai-in-javascript/

[15] i18next: learn once - translate everywhere:
https://github.com/i18next/i18next

[16] react-i18next / A powerful internationalization framework for React.js:

https://react.i18next.com/

[17] react-i18next:
https://github.com/i18next/react-i18next

[18] i18next-browser-languageDetector / Language detector used in browser environment for i18next:
https://github.com/i18next/i18next-browser-languageDetector

[19] Frequently Asked Questions (FAQ)
https://golang.org/doc/faq#

[20] PHP-JWT - A simple library to encode and decode JSON Web Tokens
https://github.com/firebase/php-jwt

[21] PHP JWT & REST API Authentication Tutorial: Login and Signup
https://www.techiediaries.com/php-jwt-authentication-tutorial/

[22] REST API Authentication Example in PHP - JWT Tutorial
https://www.codeofaninja.com/2018/09/rest-api-authentication-example-php-jwt-tutorial.html

[23] PHP: Password hash - Manual
https://www.php.net/manual/en/function.password-hash.php

[24] Domain-Driven Design in PHP - A highly Practical Guide- Carlos Buenosvinos Christian Soronellas y Keyvan Akbary junio 2017

[25] The Api checklist
https://mathieu.fenniak.net/the-api-checklist/

[26] The Clean arquitecture
https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

[27] Everithing cURL
https://www.gitbook.com/download/pdf/book/bagder/everything-curl

[28] Http Foundation
https://symfony.com/doc/current/components/http_foundation.html

[29] Api-design
https://docs.microsoft.com/es-es/azure/architecture/best-practices/api-design

[30] Códigos de estado de respuesta HTTP
https://developer.mozilla.org/es/docs/Web/HTTP/Status

[31] Componentes de consola
https://symfony.com/doc/current/components/console.html

[32] Principio HATEOAS

https://docs.microsoft.com/es-es/azure/architecture/best-practices/api-design#use-hateoas-to-enable-navigation-to-related-resources

[33] Principios SOLID
https://profile.es/blog/principios-solid-desarrollo-software-calidad

[34] Hexagonal-architecture
https://fideloper.com/hexagonal-architecture

[35] Routing
https://symfony.com/doc/current/components/routing.html

[36] Bundle Maker
https://symfony.com/doc/current/bundles/SymfonyMakerBundle/index.html

[37] NelmioCorsBundle
https://github.com/nelmio/NelmioCorsBundle

Ilustración 1 Lluvia de ideas 6
Ilustración 2 Sobre el trabajo 6
Ilustración 3 Objetivos 7
Ilustración 4 Trello 9
Ilustración 5 Requerimientos 10
Ilustración 6 Diagrama de comunicaciones 11
Ilustración 7 PHP 7 13
Ilustración 8 Symfony 13
Ilustración 9 Composer 15
Ilustración 10 Mysql 15
Ilustración 11 Diagrama relacional de la base de datos meteosalle 16
Ilustración 12 RabbitMq 17
Ilustración 13 Redis 19
Ilustración 14 Docker 20
Ilustración 15 Diagrama de flujo entre los servicios 22
Ilustración 16 Json 23
Ilustración 17 Controladores 31
Ilustración 18 Aplicación 33
Ilustración 19 Infraestructura 34
Ilustración 20 Comandos 34
Ilustración 21 Dominio 35
Ilustración 22 Diagrama de arquitectura de la API 37
Ilustración 23 Estructura de carpetas 1 38
Ilustración 24 Estructura de carpetas 2 38
Ilustración 25 Estructura de carpetas 3 39
Ilustración 26 Docker 1 41
Ilustración 27 GitHub 41
Ilustración 28 Roles Ansible 42
Ilustración 29 Vista del html 43
Ilustración 30 Ejemplo de componentes de React 43
Ilustración 31 Formulario de login 44
Ilustración 32 CSS muestra 44
Ilustración 33 Html muestra 44
Ilustración 34 Vista de registro 45
Ilustración 35 CSS Grid muestra 45
Ilustración 36 Vista de registro, otro dispositivo 45
Ilustración 37 CSS de otros dispositivos 45
Ilustración 38 Vista de estaciones pequeños dispositivos 46
Ilustración 39 Vista de estaciones 46
Ilustración 40 Dimensiones de las vistas 47
Ilustración 41 Página de inicio. Tamaño “Desktop” 47
Ilustración 42 Página de inicio. Tamaño “Smart phone” 48
Ilustración 43 Página de inicio. Tamaño “Tablet” 48

Ilustración 44 Asociación Html-Javascript 49
Ilustración 45 Comportamiento 49
Ilustración 46 Javascript import/export 50
Ilustración 47 Javascript objetos 50
Ilustración 48 Javascript promesas 50
Ilustración 49 Arrow functions 51
Ilustración 50 Fetch Api....51
Ilustración 51 Fetch Api, método POST 51
Ilustración 52 Web Storage....52
Ilustración 53 Web Storage 1....53
Ilustración 54 Web Storage 2....53
Ilustración 55 Diagrama de componentes 54
Ilustración 56 Dependencias web 55
Ilustración 57 Excepción de conexiones externas 56
Ilustración 58 Ejemplo de código de respuesta http 56
Ilustración 59 Conexión a base de datos....56
Ilustración 60 Predis librería externa instalada con composer....57
Ilustración 61 Petición de Api externa 58
Ilustración 62 Componentes pantalla principal 59
Ilustración 63 Gestión de idiomas 60
Ilustración 64 Ejemplo de implementación de Liskov 60
Ilustración 65 Interfaz de desacople del repositorio de usuarios 61
Ilustración 66 Ejemplo de inversión de dependencias....61
Ilustración 67 Capa de insfraestructura Controladores....62
Ilustración 68 Capa de insfraestructura....62
Ilustración 69 Capa de aplicación 62
Ilustración 70 Capa de dominio....62
Ilustración 71 Test....63
Ilustración 72 Test 1....63
Ilustración 73 Test 2....64
Ilustración 74 Test 3....64
Ilustración 75 Test 4....64
Ilustración 76 Ejemplo de consulta a base de datos....66
Ilustración 77 Base de datos búsqueda por ID....66
Ilustración 78 Base de datos actualización....66
Ilustración 79 Framework React....68
Ilustración 80 Vista del framework....69
Ilustración 81 Lógica del frameworks. Ejemplo....69
Ilustración 82 React Componente WillMount....70
Ilustración 83 Symfony request 71
Ilustración 84 Symfony Http-Client 72
Ilustración 85 Symfony bundle maker 72
Ilustración 86 Site map MeteoSalle 75

Ilustración 87 Optimización OnPage 76
Ilustración 88 Enlaces externos 76
Tabla 89 Conversión y modelo de negocio 77
Ilustración 90 Diagrama del modelo de negocio. 77
Ilustración 91 Estructura de datos a ser guardados en Redis....79
Ilustración 92 Profiling....80
Ilustración 93 Análisis estadístico de rendimiento frontal 80
Ilustración 94 Uso de JWT 82
Ilustración 95 Petición con la cabecera de autenticación 82
Ilustración 96 Ejemplo de implementación JWT 82
Ilustración 97 Ejemplo de CreatUser....83
Ilustración 98 Verificación de la contraseña 83
Ilustración 99 Tiempo de desarrollo 84
Ilustración 100 Etapas y número de horas 85

11.- ANEXOS

11.1.- Anexo 1 Api Middleware

Recursos

- **Servicio**
  Comprueba el estado de la API

  - **Request**

  curl -X GET \
  http://meteosalle.local/apiv1/status \

  - **Response header code http 200**


```
-   {
-       "message": "Success, the api status is up!!!",
-       "links": [
-           {
-               "href": "http://meteosalle.local/apiv1/stations/",
-               "action": "GET ",
-               "types": ['headers: Authorization']
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
-               "action": "DELETE ",
-               "types": ['headers: Authorization']
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
-               "action": "PUT ",
-               "types": [
-                   "multipart/form-data",
-                   "application/x-www-form-urlencoded",
-                   “'headers: Authorization'”
-               ]
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
-               "action": "GET ",
-               "types": ['headers: Authorization']
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
-               "action": "POST ",
-               "types": [
-                   'headers: Authorization',
-                   "multipart/form-data",
-                   "application/x-www-form-urlencoded"
```

```
-            ]
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/stations/{uuidStation}/history",
-            "action": "POST ",
-            "types": [
-                'headers: Authorization',
-                "multipart/form-data",
-                "application/x-www-form-urlencoded"
-            ]
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/stations/postalcode/{postalCode}",
-            "action": "GET ",
-            "types": ['headers: Authorization']
-    ]
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/predictions/{postalCode}",
-            "action": "GET ",
-            "types": ['headers: Authorization']
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-            "action": "GET ",
-            "types": ['headers: Authorization']
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-            "action": "DELETE ",
-            "types": ['headers: Authorization']
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-            "action": "PUT ",
-            "types": [
-                'headers: Authorization',
-                "multipart/form-data",
-                "application/x-www-form-urlencoded"
-            ]
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-            "action": "GET ",
-            "types": ['headers: Authorization']
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-            "action": "POST ",
-            "types": [
```

```
-                'headers: Authorization'
-                "multipart/form-data",
-                "application/x-www-form-urlencoded"
-            ]
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/{uuidUser}/stations",
-            "action": "GET ",
-            "types": ['headers: Authorization']
-        }
-    ]
-  }
```


- **Servicio**
  Crea una estación meteorológica de un usuario

  - **Request**

```
curl -X POST \
  http://meteosalle.local/apiv1/stations/ \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'
  -F uuidStation=t2t33 \
  -F uuidUser=u22 \
  -F latitud=44.666 \
  -F longitud=55.7657 \
  -F postalCode=08720 \
  -F temp=30 \
  -F humidity=89 \
  -F presion=14.7 \
  -F location=VilaFranca \
  -F state=Barcelona
```

  - **Response header code http 201**


```
-  {
-    "Message": "Station t2t711 created",
-    "links": [
-        {
-            "href": "http://meteosalle.local/apiv1/stations/t2t711",
-            "action": "DELETE ",
-            "types": ['headers: Authorization']
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/stations/t2t711",
-            "action": "PUT ",
-            "types": [
-                "multipart/form-data",
-                "application/x-www-form-urlencoded",
-                 'headers: Authorization'
-            ]
-        },
```

```
-         {
-            "href": "http://meteosalle.local/apiv1/stations/t2t711",
-            "action": "GET ",
-            "types": ['headers: Authorization']
-         },
-         {
-            "href": "http://meteosalle.local/apiv1/stations/t2t711/history",
-            "action": "POST ",
-            "types": [
-               'headers: Authorization'
-               "multipart/form-data",
-               "application/x-www-form-urlencoded"
-            ]
-         },
-         {
-            "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-            "action": "GET ",
-            "types": ['headers: Authorization']
-         }
-      ]
-   }
```


- **Response on fail header code 400**


```
{
   "Message": "Station already exists try update, use method PUT instead"
}

{
   "Message": "Missing argument uuidStation in request "
}
```


- **Servicio**

Encuentra una estación meteorológica con toda su información, datos históricos y las predicciones

- **Request**

curl -X GET \
http://meteosalle.local/apiv1/stations/t2t3 \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

- **Response header code http 200**


```
-   {
-      "uuidStation": "t2t7",
-      "uuidUser": "uu5",
-      "latitud": "44.66600000",
-      "longitud": "55.765700000",
-      "postalCode": "08720",
-      "temp": "30.00",
-      "humidity": "89.00",
-      "state": "Barcelona",
```

```
-        "presion": "14.70",
-        "location": "VilaFranca",
-        "historic": {
-            "0": {
-                "temp": "12.00",
-                "humidity": "90.00",
-                "presion": "33.00",
-                "timestamp": "1567202654"
-            },
-            "1": {
-                "temp": "12.00",
-                "humidity": "90.00",
-                "presion": "33.00",
-                "timestamp": "1567207496"
-            }
-        },
-        "predictions": {
-            "0": {
-                "latitud": 41.347212,
-                "longitud": 1.697747,
-                "temp": 22.689999999999998,
-                "humidity": 67.66666666666667,
-                "state": 8,
-                "presion": 1018.4933333333333,
-                "location": "Vilafranca del Penedès",
-                "timeStamp": 1567512000
-            },
-            "1": {
-                "latitud": 41.347212,
-                "longitud": 1.697747,
-                "temp": 25.452,
-                "humidity": 59.2,
-                "state": 8,
-                "presion": 1013.628,
-                "location": "Vilafranca del Penedès",
-                "timeStamp": 1567339200
-            },
-            "2": {
-                "latitud": 41.347212,
-                "longitud": 1.697747,
-                "temp": 26.2,
-                "humidity": 64,
-                "state": 8,
-                "presion": 1016,
-                "location": "Vilafranca del Penedès",
-                "timeStamp": 1566993600
-            },
-            "3": {
-                "latitud": 41.347212,
-                "longitud": 1.697747,
```

```
-            "temp": 23.742,
-            "humidity": 72.4,
-            "state": 8,
-            "presion": 1015.494,
-            "location": "Vilafranca del Penedès",
-            "timeStamp": 1567425600
-        },
-        "4": {
-            "latitud": 41.347212,
-            "longitud": 1.697747,
-            "temp": 28.5,
-            "humidity": 54,
-            "state": 8,
-            "presion": 1016.5,
-            "location": "Vilafranca del Penedès",
-            "timeStamp": 1567080000
-        },
-
-    },
-    "timestamp": 1567202623,
-    "links": {
-        "0": {
-            "href": "http://meteosalle.local/apiv1/stations/t2t7",
-            "action": "DELETE ",
-            "types": ['headers: Authorization']
-        },
-        "1": {
-            "href": "http://meteosalle.local/apiv1/stations/t2t7",
-            "action": "PUT ",
-            "types": {
-                "0": "multipart/form-data",
-                "1": "application/x-www-form-urlencoded",
-                “2”: “headers: Authorization”
-            }
-        },
-        "2": {
-            "href": "http://meteosalle.local/apiv1/stations/t2t7/history",
-            "action": "POST ",
-            "types": {
-                "0": "multipart/form-data",
-                "1": "application/x-www-form-urlencoded",
-                “2”: “headers: Authorization”
-
-            }
-        },
-        "3": {
-            "href": "http://meteosalle.local/apiv1/predictions/{postalCode}",
-            "action": "GET ",
-            "types": { “headers: Authorization”}
-
```

```
-           },
-           "4": {
-               "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-               "action": "GET ",
-               "types": { “headers: Authorization”}        }
-       }
-   }
```


- **Response on fail header code 404**


```
{
   "Message": "Station t2t333 not Found"
}
```


- **Servicio**

  Añade información histórica de una estación meteorológica a la base de datos

  - **Request**

```
curl -X POST \
  http://meteosalle.local/apiv1/stations/t2t3/history \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'
  -F uuidUser=u2 \
  -F temp=34 \
  -F humidity=90 \
  -F presion=123
```

  - **Response header code http 201**


```
-   {
-       "Message": "history for the station t2t7 updated",
-       "links": [
-           {
-               "href": "http://meteosalle.local/apiv1/stations/t2t7",
-               "action": "DELETE ",
-               "types": { “headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/t2t7",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/t2t7/history",
-               "action": "POST ",
-               "types": [
-                   "multipart/form-data",
-                   "application/x-www-form-urlencoded",
-                   “headers: Authorization”
-               ]
```

```
-         }
-      ]
-   }
```


- **Response on fail header code 400**
  {
  "Message": "Could not insert value, check your request; station most exist. Values type temp=decimal(4,2), humidity=decimal(4,2), presion=decimal(6,2),lat=decimal(10,8), longitud=decimal(11,9) "
  }

- **Servicio**
  Encuentra las predicciones meteorológicas dado un código postal

  - **Request**
    curl -X GET \
    http://meteosalle.local/apiv1/predictions/8500 \
    -H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

  - **Response header code http 200**


```
-   {
-      "0": {
-         "latitud": 41.347212,
-         "longitud": 1.697747,
-         "temp": 22.689999999999998,
-         "humidity": 67.66666666666667,
-         "state": 8,
-         "presion": 1018.4933333333333,
-         "location": "Vilafranca del Penedès",
-         "timeStamp": 1567512000
-      },
-      "1": {
-         "latitud": 41.347212,
-         "longitud": 1.697747,
-         "temp": 25.452,
-         "humidity": 59.2,
-         "state": 8,
-         "presion": 1013.628,
-         "location": "Vilafranca del Penedès",
-         "timeStamp": 1567339200
-      },
-      "2": {
-         "latitud": 41.347212,
-         "longitud": 1.697747,
-         "temp": 26.2,
-         "humidity": 64,
-         "state": 8,
-         "presion": 1016,
```

```
-           "location": "Vilafranca del Penedès",
-           "timeStamp": 1566993600
-       },
-       "3": {
-           "latitud": 41.347212,
-           "longitud": 1.697747,
-           "temp": 23.742,
-           "humidity": 72.4,
-           "state": 8,
-           "presion": 1015.494,
-           "location": "Vilafranca del Penedès",
-           "timeStamp": 1567425600
-       },
-       "4": {
-           "latitud": 41.347212,
-           "longitud": 1.697747,
-           "temp": 28.5,
-           "humidity": 54,
-           "state": 8,
-           "presion": 1016.5,
-           "location": "Vilafranca del Penedès",
-           "timeStamp": 1567080000
-       },
-       "links": {
-           "0": {
-               "href": "http://meteosalle.local/apiv1/stations/",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           "1": {
-               "href": "http://meteosalle.local/apiv1/stations/{uuidStation}/history",
-               "action": "POST ",
-               "types": {
-                   "0": "multipart/form-data",
-                   "1": "application/x-www-form-urlencoded",
-                   “2”: “headers: Authorization”
-               }
-           },
-           "2": {
-               "href": "http://meteosalle.local/apiv1/stations/postalcode/08720",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           "3": {
-               "href": "http://meteosalle.local/apiv1/predictions/08720",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           "4": {
-               "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
```

```
-            "action": "GET ",
-            "types": { “headers: Authorization”}
-         }
-      }
-   }
```

- **Response on fail header code 400 404 500 503**

En este punto no se mostrarán todos los posibles errores debido a que esta petición es externa y se manejan muchos errores en dependencia de la respuesta del servidor remoto y de la petición del usuario, los errores pueden ser de tipo transport exception, client exception, redirection exception, Server exception, LocationCodeErrorException o cuando no se encuentra una predicción dado un código postal el código de error en la respuesta del encabezado varía según la excepción. Estas excepciones remotas son convertidas a una excepción de dominio llamada ApiConectionErrorException que es la que se lanza dentro de la aplicación middellware en la interfaz de comunicación externa, luego es capturada en el caso de uso y se lanza otra excepción de dominio que es la que finalmente captura el controlador que estandariza la respuesta al cliente, un ejemplo de respuesta sería:

header code 404

```
{
   "Message": "No location code found for this postal code"
}
```

- **Servicio**

Encontrar todas las estaciones radicadas en un área geográfica identificada por un código postal, devuelve la unión de las estaciones externas y las internas en el caso que alguna falle devuelve lo que haya encontrado

- **Request**

curl -X GET \
http://meteosalle.local/apiv1/stations/postalcode/08500 \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

- **Response header code http 200**

```
-   {
-   {
-      "0": {
-         "uuidStation": 746,
-         "uuidUser": "external",
-         "latitud": 41.928384,
-         "longitud": 2.255736,
-         "postalCode": "08500",
-         "temp": 22.72,
-         "humidity": 73,
-         "state": 8,
-         "presion": 1018.0966666666667,
```

```
-           "location": "Vic",
-           "historic": null,
-           "predictions": null,
-           "timestamp": 1567208750
-       },
-       "links": {
-           "0": {
-               "href": "http://meteosalle.local/apiv1/stations/",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           "1": {
-               "href": "http://meteosalle.local/apiv1/stations/{uuidStation}/history",
-               "action": "POST ",
-               "types": {
-                   "0": "multipart/form-data",
-                   "1": "application/x-www-form-urlencoded",
-                    “2”: “headers: Authorization”
-               }
-           },
-           "2": {
-               "href": "http://meteosalle.local/apiv1/stations/postalcode/08500",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           "3": {
-               "href": "http://meteosalle.local/apiv1/predictions/08500",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           "4": {
-               "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           }
-       }
-   }
```


- **Response on fail header code 404**


```
{
    "Message": "No location code found for this postal code"
}
```


- **Servicio**
  Encuentra todas las estaciones disponibles y las devuelve, tanto las estaciones externas como las internas.

- **Request**

curl -X GET \
http://meteosalle.local/apiv1/stations/ \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

- **Response header code http 200**

Muestra de la respuesta


```
"2": {
        "uuidStation": "t2t333",
        "uuidUser": "u22",
        "latitud": "44.66600000",
        "longitud": "55.765700000",
        "postalCode": "08720",
        "temp": "30.00",
        "humidity": "89.00",
        "state": "Barcelona",
        "presion": "14.70",
        "location": "VilaFranca",
        "historic": null,
        "predictions": null,
        "timestamp": 1567158397
    },
    "3": {
        "uuidStation": 1655,
        "uuidUser": "external",
        "latitud": 41.625529,
        "longitud": 1.415415,
        "postalCode": "8717",
        "temp": 29.933333333333334,
        "humidity": 51,
        "state": 8,
        "presion": 1019.9666666666667,
        "location": "Montmaneu",
        "historic": null,
        "predictions": null,
        "timestamp": 1567162130
    },
"links": {
        "0": {
            "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
            "action": "DELETE ",
            "types": {}
        },
        "1": {
            "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
            "action": "PUT ",
            "types": {
```

```
            "0": "multipart/form-data",
            "1": "application/x-www-form-urlencoded"
             “2”:  “headers: Authorization”
        }
    },
    "2": {
        "href": "http://meteosalle.local/apiv1/stations/{uuidStation}/history",
        "action": "POST ",
        "types": {
            "0": "multipart/form-data",
            "1": "application/x-www-form-urlencoded",
            “2”: “headers: Authorization”

        }
    },
    "3": {
        "href": "http://meteosalle.local/apiv1/predictions/{postalCode}",
        "action": "GET ",
        "types": { “headers: Authorization”}
    },
    "4": {
        "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
        "action": "GET ",
        "types": { “headers: Authorization”}
    }
}
```

- **Response on fail header code 500**

```
{
    "Message": "We could not find any stations sorry"
}
```

- **Servicio**
  Actualiza los datos de una estación
  - **Parámetros requeridos**
    1. uuidStation
    2. token en el encabezado de la petición.
  - **Parámetros opcionales**
    1. temp
    2. latitud
    3. longitud
    4. uuidUser
    5. location
    6. postalCode
    7. state
    8. presion
  - **Request**

  curl -X PUT \

'http://meteosalle.local/apiv1/stations/{uuidStation}?temp=2&latitud=-0.333&uuidUser=u2&longitud=33.33&location=Vila%20Franca%20del%20penedes&postalCode=08720&state=Barcelona&presion=33' \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

- **Response header code http 200**

```
-   {
-       "Message": "Station t2t7 updated",
-       "links": [
-           {
-               "href": "http://meteosalle.local/apiv1/stations/t2t7",
-               "action": "DELETE ",
-               "types": { “headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/t2t7",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/t2t7/history",
-               "action": "POST ",
-               "types": [
-                   "multipart/form-data",
-                   "application/x-www-form-urlencoded",
-                   “headers: Authorization”
-               ]
-           }
-       ]
-   }
```

- **Response on fail header code 404**

```
{
    "Message": "Station t2t43 not Found"
}
```

- **Servicio**
  Elimina una estación
  - **Parámetros requeridos**
    1. uuidStation

  - **Request**

curl -X DELETE \
http://meteosalle.local/apiv1/stations/t2t2 \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

  - **Response header code http 200**

```
-   {
-       "Message": "Station deleted",
-       "links": [
-           {
-               "href": "http://meteosalle.local/apiv1/stations/",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/stations/t2t5",
-               "action": "GET ",
-               "types": { “headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-               "action": "GET ",
-               "types": { “headers: Authorization”}        }
-       ]
-   }
```

- **Response on fail header code 500**

```
{
    "Message": "Could not delete station."
}
```

- **Servicio**

  Crea un usuario de la aplicación

  - **Parámetros requeridos**
    1. uuidUser
    2. name
    3. lastname
    4. password
    5. userName

  - **Parámetros opcionales**
    1. Age
    2. gender
  - **Request**

```
curl -X POST \
  http://meteosalle.local/apiv1/user/ \
  -F uuidUser=uu \
  -F name=david \
  -F lastname=espinosa \
  -F password=a \
  -F userName=despinosa \
  -F age=29 \
  -F gender=male
```

  - **Response header code http 201**

```
-       "Message": "User 1d1dd created",
-       "links": [
-           {
```

```
-            "href": "http://meteosalle.local/apiv1/user/1d1dd",
-            "action": "DELETE ",
-            "types": { “headers: Authorization”}
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/1d1dd",
-            "action": "PUT ",
-            "types": [
-                "multipart/form-data",
-                "application/x-www-form-urlencoded",
-                “headers: Authorization”
-            ]
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/1d1dd",
-            "action": "GET ",
-            "types": { “headers: Authorization”}
-        },
-        {
-            "href": "http://meteosalle.local/apiv1/user/1d1dd/stations",
-            "action": "GET ",
-            "types": { “headers: Authorization”}
-        }
-    ],
-    "token":
    "JDJ5JDEyJHJ0ZE1WdU9Qc1MvWlBpY0ZnM0s4dy55ZjIxaWhhckRiTGVxbzY5RXB
    PdElESnRDM0EvYjVl"
-   }
```


- **Response on fail header code 400**


```
{
    "Message": " User already exists try update, use method PUT instead"
}
```


- **Servicio**

Busca un usuario

- **Parámetros requeridos**
  1. uuidUser
  2. header token
- **Request**

curl -X GET \
http://meteosalle.local/apiv1/user/uu2 \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

- **Response header code http 200**


```
-   {
-       "uuidUser": "9",
```

```
-       "name": "david",
-       "lastname": "espinosa",
-       "password": "a",
-       "userName": "despinosa",
-       "age": "29",
-       "gender": "male",
-       "links": {
-           "0": {
-               "href": "http://meteosalle.local/apiv1/user/9",
-               "action": "DELETE ",
-               "types": {}
-           },
-           "1": {
-               "href": "http://meteosalle.local/apiv1/user/9",
-               "action": "PUT ",
-               "types": {
-                   "0": "multipart/form-data",
-                   "1": "application/x-www-form-urlencoded",
-                   “2”: “headers: Authorization”
-               }
-           },
-           "2": {
-               "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-               "action": "POST ",
-               "types": {
-                   "0": "multipart/form-data",
-                   "1": "application/x-www-form-urlencoded",
-                   “2”: “headers: Authorization”
-
-               }
-           },
-           "3": {
-               "href": "http://meteosalle.local/apiv1/user/9/stations",
-               "action": "GET ",
-               "types": {“headers: Authorization”}
-           }
-       }
-   }
```


- **Response on fail header code 404**


```
{
    "Message": "User uu223 not Found"
}
```


- **Servicio**

  Busca las estaciones que pertenecen a un usuario

  - **Parámetros requeridos**
    1. uuidUser=u2
    2. header token
  - **Request**

```
curl -X GET \
  http://meteosalle.local/apiv1/user/u2/stations \
-H 'Authorization:
JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvS
DlvaVdTZVAyRlJ5'
```

Response header code http 200


```
{
    "0": {
        "uuidStation": "99",
        "uuidUser": "9",
        "latitud": "44.66600000",
        "longitud": "55.765700000",
        "postalCode": "08720",
        "temp": "30.00",
        "humidity": "89.00",
        "state": "Barcelona",
        "presion": "14.70",
        "location": "VilaFranca",
        "historic": {
            "0": {
                "temp": "12.00",
                "humidity": "90.00",
                "presion": "33.00",
                "timestamp": "1567209093"
            }
        },
        "predictions": {
            "0": {
                "latitud": 41.347212,
                "longitud": 1.697747,
                "temp": 22.689999999999998,
                "humidity": 67.66666666666667,
                "state": 8,
                "presion": 1018.4933333333333,
                "location": "Vilafranca del Penedès",
                "timeStamp": 1567512000
            },
            "1": {
                "latitud": 41.347212,
                "longitud": 1.697747,
                "temp": 25.452,
                "humidity": 59.2,
                "state": 8,
                "presion": 1013.628,
                "location": "Vilafranca del Penedès",
                "timeStamp": 1567339200
            },
```

```
        "2": {
            "latitud": 41.347212,
            "longitud": 1.697747,
            "temp": 26.2,
            "humidity": 64,
            "state": 8,
            "presion": 1016,
            "location": "Vilafranca del Penedès",
            "timeStamp": 1566993600
        },
        "3": {
            "latitud": 41.347212,
            "longitud": 1.697747,
            "temp": 23.742,
            "humidity": 72.4,
            "state": 8,
            "presion": 1015.494,
            "location": "Vilafranca del Penedès",
            "timeStamp": 1567425600
        },
        "4": {
            "latitud": 41.347212,
            "longitud": 1.697747,
            "temp": 28.5,
            "humidity": 54,
            "state": 8,
            "presion": 1016.5,
            "location": "Vilafranca del Penedès",
            "timeStamp": 1567080000
        }
    },
    "timestamp": 1567209085
},
"links": {
    "0": {
        "href": "http://meteosalle.local/apiv1/stations/",
        "action": "GET ",
        "types": {“headers: Authorization”}
    },
    "1": {
        "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
        "action": "DELETE ",
        "types": {“headers: Authorization”}
    },
    "2": {
        "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
        "action": "PUT ",
        "types": {
            "0": "multipart/form-data",
            "1": "application/x-www-form-urlencoded",
```

```
            "2": “headers: Authorization”
        }
    },
    "3": {
        "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
        "action": "GET ",
        "types": {“headers: Authorization”}
    },
    "4": {
        "href": "http://meteosalle.local/apiv1/stations/{uuidStation}",
        "action": "POST ",
        "types": {
            "0": "multipart/form-data",
            "1": "application/x-www-form-urlencoded",
            "2": “headers: Authorization”

        }
    },
    "5": {
        "href": "http://meteosalle.local/apiv1/stations/{uuidStation}/history",
        "action": "POST ",
        "types": {
            "0": "multipart/form-data",
            "1": "application/x-www-form-urlencoded"
            "2": “headers: Authorization”
        }
    },
    "6": {
        "href": "http://meteosalle.local/apiv1/stations/postalcode/{postalCode}",
        "action": "GET ",
        "types": {“headers: Authorization”}
    },
    "7": {
        "href": "http://meteosalle.local/apiv1/predictions/{postalCode}",
        "action": "GET ",
        "types": {“headers: Authorization”}
    },
    "8": {
        "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
        "action": "GET ",
        "types": {“headers: Authorization”}
    }
  }
}
```


- **Response on fail header code 404**


```
{
    "Message": "Stations not found"
}
{
```

```
    "Message": "User u2332 not Found"
}
```

- **Servicio**
  Actualizar los datos de un usuario
  - **Parámetros requeridos**
    1. uuidUser=u2
    2. header token

  - **Parámetros opcionales**
    1. lastname
    2. password
    3. userName
    4. age
    5. gender
    6. name
  - **Request**

curl -X PUT \
'http://meteosalle.local/apiv1/user/uqw2?lastname=uyasgd&password=password&userName=nm&age=30&gender=other&name=user23' \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

  - **Response header code http 200**

```
-   {
-       "Message": "User 9 updated",
-       "links": [
-           {
-               "href": "http://meteosalle.local/apiv1/user/9",
-               "action": "DELETE ",
-               "types": {“headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/user/9",
-               "action": "GET ",
-               "types": {“headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/user/9/stations",
-               "action": "GET ",
-               "types": {“headers: Authorization”}
-           }
-       ]
-   }
```

  - **Response on fail header code 404**

```
{
    "Message": "User u652 not Found"
```

}

- **Servicio**
  Elimina un usuario de la aplicación

  - **Parámetros requeridos**
    1. uuidUser=9
    2. header token

  - **Request**

curl -X DELETE \
http://meteosalle.local/apiv1/user/u2 \
-H 'Authorization: JDJ5JDEyJGQ5dTBrSXJPVHhncFV1dkFHb3o5Li5DamFMUDNEYkRJcm0wUjlhdEQvSDlvaVdTZVAyRlJ5'

  - **Response header code http 200**

```
-   {
-       "Message": "User deleted",
-       "0": [
-           {
-               "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-               "action": "POST ",
-               "types": [
-                   "multipart/form-data",
-                   "application/x-www-form-urlencoded"
-                    “headers: Authorization”
-
-               ]
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/user/{uuidUser}",
-               "action": "GET ",
-               "types": {“headers: Authorization”}
-           },
-           {
-               "href": "http://meteosalle.local/apiv1/user/{uuidUser}/stations",
-               "action": "GET ",
-               "types": {“headers: Authorization”}
-           }
-       ]
-   }
```

  - **Response on fail header code 404**

```
{
    "Message": "User u2t not Found"
}
```

- **Servicio**
  Inicia la sesión de un usuario

- **Parámetros requeridos**
  1. uuidUser=9

- **Request**

```
curl -X POST \
  http://meteosalle.local/apiv1/login/ \

  -F userName=id1d \
  -F password=a
```

- **Response header code 200**


```
-   {
-     "Message": "Login Correct",
-     "token":
    "JDJ5JDEyJHZCME1qTkpPTHk5V2RRVjdGOHU3QS5DTHp1QnZldEM1Tk8zc0JQb
    DdnNTRCOUVtUk9pNk5D",
-     "uuidUser": "id1d"
-   }
```


- **Response header code 403**


```
-   {
-     "Message": " Autentication denied”
-   }
```